

SOLUTIONS MANUAL

OPERATING SYSTEMS: INTERNALS AND DESIGN PRINCIPLES

SIXTH EDITION

Do Not Post on Web

WILLIAM STALLINGS

Copyright 2008: William Stallings

© 2008 by William Stallings

All rights reserved. No part of this document may be reproduced, in any form or by any means, or posted on the Internet, without permission in writing from the author. Selected solutions may be shared with students, provided that they are not available, unsecured, on the Web.

NOTICE

This manual contains solutions to the review questions and homework problems in *Operating Systems, Sixth Edition*. If you spot an error in a solution or in the wording of a problem, I would greatly appreciate it if you would forward the information via email to ws@shore.net. An errata sheet for this manual, if needed, is available at <http://www.box.net/public/ig0eifhfxu> . File name is S-OS6e-mmyy

W.S.

Please
Post on We

TABLE OF CONTENTS

Chapter 1 Computer System Overview.....	5
Chapter 2 Operating System Overview.....	11
Chapter 3 Process Description and Control.....	14
Chapter 4 Threads, SMP and Microkernels	19
Chapter 5 Concurrency: Mutual Exclusion and Synchronization	24
Chapter 6 Concurrency: Deadlock and Starvation	37
Chapter 7 Memory Management	46
Chapter 8 Virtual Memory	51
Chapter 9 Uniprocessor Scheduling.....	59
Chapter 10 Multiprocessor and Real-Time Scheduling	72
Chapter 11 I/O Management and Disk Scheduling.....	77
Chapter 12 File Management	83
Chapter 13 Embedded Operating Systems	87
Chapter 14 Computer Security Threats	92
Chapter 15 Computer Security Techniques	94
Chapter 16 Distributed Processing, Client/Server, and Clusters.....	101
Chapter 17 Networking	104
Chapter 18 Distributed Process Management	107
Appendix A Topics in Concurrency	110

CHAPTER 1 COMPUTER SYSTEM OVERVIEW

ANSWERS TO QUESTIONS

- 1.1 A **main memory**, which stores both data and instructions; an **arithmetic and logic unit** (ALU) capable of operating on binary data; a **control unit**, which interprets the instructions in memory and causes them to be executed; and **input and output (I/O) equipment** operated by the control unit.
- 1.2 **User-visible registers:** Enable the machine- or assembly-language programmer to minimize main memory references by optimizing register use. For high-level languages, an optimizing compiler will attempt to make intelligent choices of which variables to assign to registers and which to main memory locations. Some high-level languages, such as C, allow the programmer to suggest to the compiler which variables should be held in registers. **Control and status registers:** Used by the processor to control the operation of the processor and by privileged, operating system routines to control the execution of programs.
- 1.3 These actions fall into four categories: **Processor-memory:** Data may be transferred from processor to memory or from memory to processor. **Processor-I/O:** Data may be transferred to or from a peripheral device by transferring between the processor and an I/O module. **Data processing:** The processor may perform some arithmetic or logic operation on data. **Control:** An instruction may specify that the sequence of execution be altered.
- 1.4 An interrupt is a mechanism by which other modules (I/O, memory) may interrupt the normal sequencing of the processor.
- 1.5 Two approaches can be taken to dealing with multiple interrupts. The first is to disable interrupts while an interrupt is being processed. A second approach is to define priorities for interrupts and to allow an interrupt of higher priority to cause a lower-priority interrupt handler to be interrupted.
- 1.6 The three key characteristics of memory are cost, capacity, and access time.
- 1.7 Cache memory is a memory that is smaller and faster than main memory and that is interposed between the processor and main memory. The cache acts as a buffer for recently used memory locations.
- 1.8 **Programmed I/O:** The processor issues an I/O command, on behalf of a process, to an I/O module; that process then busy-waits for the operation to be completed before proceeding. **Interrupt-driven I/O:** The processor issues an I/O command on behalf of a process, continues to execute subsequent instructions, and is interrupted

by the I/O module when the latter has completed its work. The subsequent instructions may be in the same process, if it is not necessary for that process to wait for the completion of the I/O. Otherwise, the process is suspended pending the interrupt and other work is performed. **Direct memory access (DMA):** A DMA module controls the exchange of data between main memory and an I/O module. The processor sends a request for the transfer of a block of data to the DMA module and is interrupted only after the entire block has been transferred.

1.9 Spatial locality refers to the tendency of execution to involve a number of memory locations that are clustered. **Temporal locality** refers to the tendency for a processor to access memory locations that have been used recently.

1.10 Spatial locality is generally exploited by using larger cache blocks and by incorporating prefetching mechanisms (fetching items of anticipated use) into the cache control logic. **Temporal locality** is exploited by keeping recently used instruction and data values in cache memory and by exploiting a cache hierarchy.

ANSWERS TO PROBLEMS

1.1 Memory (contents in hex): 300: 3005; 301: 5940; 302: 7006

Step 1: 3005 → IR; **Step 2:** 3 → AC

Step 3: 5940 → IR; **Step 4:** $3 + 2 = 5$ → AC

Step 5: 7006 → IR; **Step 6:** AC → Device 6

- 1.2**
1.
 - a. The PC contains 300, the address of the first instruction. This value is loaded into the MAR.
 - b. The value in location 300 (which is the instruction with the value 1940 in hexadecimal) is loaded into the MBR, and the PC is incremented. These two steps can be done in parallel.
 - c. The value in the MBR is loaded into the IR.
 2.
 - a. The address portion of the IR (940) is loaded into the MAR.
 - b. The value in location 940 is loaded into the MBR.
 - c. The value in the MBR is loaded into the AC.
 3.
 - a. The value in the PC (301) is loaded into the MAR.
 - b. The value in location 301 (which is the instruction with the value 5941) is loaded into the MBR, and the PC is incremented.
 - c. The value in the MBR is loaded into the IR.
 4.
 - a. The address portion of the IR (941) is loaded into the MAR.
 - b. The value in location 941 is loaded into the MBR.
 - c. The old value of the AC and the value of location MBR are added and the result is stored in the AC.
 5.
 - a. The value in the PC (302) is loaded into the MAR.
 - b. The value in location 302 (which is the instruction with the value 2941) is loaded into the MBR, and the PC is incremented.
 - c. The value in the MBR is loaded into the IR.
 6.
 - a. The address portion of the IR (941) is loaded into the MAR.
 - b. The value in the AC is loaded into the MBR.
 - c. The value in the MBR is stored in location 941.

- 1.3 a. $2^{24} = 16 \text{ MBytes}$
- b. (1) If the local address bus is 32 bits, the whole address can be transferred at once and decoded in memory. However, since the data bus is only 16 bits, it will require 2 cycles to fetch a 32-bit instruction or operand.
- (2) The 16 bits of the address placed on the address bus can't access the whole memory. Thus a more complex memory interface control is needed to latch the first part of the address and then the second part (since the microprocessor will end in two steps). For a 32-bit address, one may assume the first half will decode to access a "row" in memory, while the second half is sent later to access a "column" in memory. In addition to the two-step address operation, the microprocessor will need 2 cycles to fetch the 32 bit instruction/operand.
- c. The program counter must be at least 24 bits. Typically, a 32-bit microprocessor will have a 32-bit external address bus and a 32-bit program counter, unless on-chip segment registers are used that may work with a smaller program counter. If the instruction register is to contain the whole instruction, it will have to be 32-bits long; if it will contain only the op code (called the op code register) then it will have to be 8 bits long.

- 1.4 In cases (a) and (b), the microprocessor will be able to access $2^{16} = 64\text{K}$ bytes; the only difference is that with an 8-bit memory each access will transfer a byte, while with a 16-bit memory an access may transfer a byte or a 16-byte word. For case (c), separate input and output instructions are needed, whose execution will generate separate "I/O signals" (different from the "memory signals" generated with the execution of memory-type instructions); at a minimum, one additional output pin will be required to carry this new signal. For case (d), it can support $2^8 = 256$ input and $2^8 = 256$ output byte ports and the same number of input and output 16-bit ports; in either case, the distinction between an input and an output port is defined by the different signal that the executed input or output instruction generated.

1.5 Clock cycle = $\frac{1}{8 \text{ MHz}} = 125 \text{ ns}$

Bus cycle = $4 \times 125 \text{ ns} = 500 \text{ ns}$

2 bytes transferred every 500 ns; thus transfer rate = 4 MBytes/sec

Doubling the frequency may mean adopting a new chip manufacturing technology (assuming each instructions will have the same number of clock cycles); doubling the external data bus means wider (maybe newer) on-chip data bus drivers/latches and modifications to the bus control logic. In the first case, the speed of the memory chips will also need to double (roughly) not to slow down the microprocessor; in the second case, the "word length" of the memory will have to double to be able to send/receive 32-bit quantities.

- 1.6 a. Input from the Teletype is stored in INPR. The INPR will only accept data from the Teletype when FGI=0. When data arrives, it is stored in INPR, and FGI is set to 1. The CPU periodically checks FGI. If FGI =1, the CPU transfers the contents of INPR to the AC and sets FGI to 0.

When the CPU has data to send to the Teletype, it checks FGO. If FGO = 0, the CPU must wait. If FGO = 1, the CPU transfers the contents of the AC to OUTF and sets FGO to 0. The Teletype sets FGI to 1 after the word is printed.

- b. The process described in (a) is very wasteful. The CPU, which is much faster than the Teletype, must repeatedly check FGI and FGO. If interrupts are used, the Teletype can issue an interrupt to the CPU whenever it is ready to accept or send data. The IEN register can be set by the CPU (under programmer control)

1.7 If a processor is held up in attempting to read or write memory, usually no damage occurs except a slight loss of time. However, a DMA transfer may be to or from a device that is receiving or sending data in a stream (e.g., disk or tape), and cannot be stopped. Thus, if the DMA module is held up (denied continuing access to main memory), data will be lost.

1.8 Let us ignore data read/write operations and assume the processor only fetches instructions. Then the processor needs access to main memory once every microsecond. The DMA module is transferring characters at a rate of 1200 characters per second, or one every 833 μ s. The DMA therefore "steals" every 833rd cycle. This slows down the processor approximately $\frac{1}{833} \times 100\% = 0.12\%$

- 1.9 a. The processor can only devote 5% of its time to I/O. Thus the maximum I/O instruction execution rate is $10^6 \times 0.05 = 50,000$ instructions per second. The I/O transfer rate is therefore 25,000 words/second.
b. The number of machine cycles available for DMA control is

$$10^6(0.05 \times 5 + 0.95 \times 2) = 2.15 \times 10^6$$

If we assume that the DMA module can use all of these cycles, and ignore any setup or status-checking time, then this value is the maximum I/O transfer rate.

- 1.10 a. A reference to the first instruction is immediately followed by a reference to the second.
b. The ten accesses to $a[i]$ within the inner for loop which occur within a short interval of time.

1.11 Define

C_i = Average cost per bit, memory level i

S_i = Size of memory level i

T_i = Time to access a word in memory level i

H_i = Probability that a word is in memory i and in no higher-level memory

B_i = Time to transfer a block of data from memory level $(i + 1)$ to memory level i

Let cache be memory level 1; main memory, memory level 2; and so on, for a total of N levels of memory. Then

$$C_s = \frac{\sum_{i=1}^N C_i S_i}{\sum_{i=1}^N S_i}$$

The derivation of T_s is more complicated. We begin with the result from probability theory that:

$$\text{Expected Value of } x = \sum_{i=1}^N i \Pr[x = i]$$

We can write:

$$T_s = \sum_{i=1}^N T_i H_i$$

We need to realize that if a word is in M_1 (cache), it is read immediately. If it is in M_2 but not M_1 , then a block of data is transferred from M_2 to M_1 and then read.

Thus:

$$T_2 = B_1 + T_1$$

Further

$$T_3 = B_2 + T_2 = B_1 + B_2 + T_1$$

Generalizing:

$$T_i = \sum_{j=1}^{i-1} B_j + T_1$$

So

$$T_s = \sum_{i=2}^N \sum_{j=1}^{i-1} (B_j H_i) + T_1 \sum_{i=1}^N H_i$$

But

$$\sum_{i=1}^N H_i = 1$$

Finally

$$T_s = \sum_{i=2}^N \sum_{j=1}^{i-1} (B_j H_i) + T_1$$

1.12 a. $\text{Cost} = C_m \times 8 \times 10^6 = 8 \times 10^3 \text{ ¢} = \80

b. $\text{Cost} = C_c \times 8 \times 10^6 = 8 \times 10^4 \text{ ¢} = \800

c. From Equation 1.1 : $1.1 \times T_1 = T_1 + (1 - H)T_2$
 $(0.1)(100) = (1 - H)(1200)$
 $H = 1190/1200$

1.13 There are three cases to consider:

Location of referenced word	Probability	Total time for access in ns
In cache	0.9	20
Not in cache, but in main memory	$(0.1)(0.6) = 0.06$	$60 + 20 = 80$
Not in cache or main memory	$(0.1)(0.4) = 0.04$	$12\text{ms} + 60 + 20 = 12,000,080$

So the average access time would be:

$$\text{Avg} = (0.9)(20) + (0.06)(80) + (0.04)(12000080) = 480026 \text{ ns}$$

1.14 Yes, if the stack is only used to hold the return address. If the stack is also used to pass parameters, then the scheme will work only if it is the control unit that removes parameters, rather than machine instructions. In the latter case, the processor would need both a parameter and the PC on top of the stack at the same time.

CHAPTER 2 OPERATING SYSTEM OVERVIEW

ANSWERS TO QUESTIONS

- 2.1 **Convenience:** An operating system makes a computer more convenient to use. **Efficiency:** An operating system allows the computer system resources to be used in an efficient manner. **Ability to evolve:** An operating system should be constructed in such a way as to permit the effective development, testing, and introduction of new system functions without interfering with service.
- 2.2 The kernel is a portion of the operating system that includes the most heavily used portions of software. Generally, the kernel is maintained permanently in main memory. The kernel runs in a privileged mode and responds to calls from processes and interrupts from devices.
- 2.3 Multiprogramming is a mode of operation that provides for the interleaved execution of two or more computer programs by a single processor.
- 2.4 A process is a program in execution. A process is controlled and scheduled by the operating system.
- 2.5 The **execution context**, or **process state**, is the internal data by which the operating system is able to supervise and control the process. This internal information is separated from the process, because the operating system has information not permitted to the process. The context includes all of the information that the operating system needs to manage the process and that the processor needs to execute the process properly. The context includes the contents of the various processor registers, such as the program counter and data registers. It also includes information of use to the operating system, such as the priority of the process and whether the process is waiting for the completion of a particular I/O event.
- 2.6 **Process isolation:** The operating system must prevent independent processes from interfering with each other's memory, both data and instructions. **Automatic allocation and management:** Programs should be dynamically allocated across the memory hierarchy as required. Allocation should be transparent to the programmer. Thus, the programmer is relieved of concerns relating to memory limitations, and the operating system can achieve efficiency by assigning memory to jobs only as needed. **Support of modular programming:** Programmers should be able to define program modules, and to create, destroy, and alter the size of modules dynamically. **Protection and access control:** Sharing of memory, at any level of the memory hierarchy, creates the potential for one program to address the memory space of another. This is desirable when sharing is needed by particular applications. At other times, it threatens the integrity of programs and even of the

operating system itself. The operating system must allow portions of memory to be accessible in various ways by various users. **Long-term storage:** Many application programs require means for storing information for extended periods of time, after the computer has been powered down.

- 2.7 A **virtual address** refers to a memory location in virtual memory. That location is on disk and at some times in main memory. A real address is an address in main memory.
- 2.8 Round robin is a scheduling algorithm in which processes are activated in a fixed cyclic order; that is, all processes are in a circular queue. A process that cannot proceed because it is waiting for some event (e.g. termination of a child process or an input/output operation) returns control to the scheduler.
- 2.9 A **monolithic kernel** is a large kernel containing virtually the complete operating system, including scheduling, file system, device drivers, and memory management. All the functional components of the kernel have access to all of its internal data structures and routines. Typically, a monolithic kernel is implemented as a single process, with all elements sharing the same address space. A **microkernel** is a small privileged operating system core that provides process scheduling, memory management, and communication services and relies on other processes to perform some of the functions traditionally associated with the operating system kernel.
- 2.10 Multithreading is a technique in which a process, executing an application, is divided into threads that can run concurrently.

ANSWERS TO PROBLEMS

- 2.1 The answers are the same for (a) and (b). Assume that although processor operations cannot overlap, I/O operations can.

1 Job:	TAT = NT	Processor utilization	= 50%
2 Jobs:	TAT = NT	Processor utilization	= 100%
4 Jobs:	TAT = (4N - 2)T	Processor utilization	= 100%

- 2.2 I/O-bound programs use relatively little processor time and are therefore favored by the algorithm. However, if a processor-bound process is denied processor time for a sufficiently long period of time, the same algorithm will grant the processor to that process since it has not used the processor at all in the recent past. Therefore, a processor-bound process will not be permanently denied access.
- 2.3 With time sharing, the concern is turnaround time. Time-slicing is preferred because it gives all processes access to the processor over a short period of time. In a batch system, the concern is with throughput, and the less context switching, the more processing time is available for the processes. Therefore, policies that minimize context switching are favored.

- 2.4 A system call is used by an application program to invoke a function provided by the operating system. Typically, the system call results in transfer to a system program that runs in kernel mode.
- 2.5 The system operator can review this quantity to determine the degree of "stress" on the system. By reducing the number of active jobs allowed on the system, this average can be kept high. A typical guideline is that this average should be kept above 2 minutes [IBM86]. This may seem like a lot, but it isn't.

Please Do Not
Post on Web

CHAPTER 3 PROCESS DESCRIPTION AND CONTROL

ANSWERS TO QUESTIONS

- 3.1 An instruction trace for a program is the sequence of instructions that execute for that process.
- 3.2 New batch job; interactive logon; created by OS to provide a service; spawned by existing process. See Table 3.1 for details.
- 3.3 **Running:** The process that is currently being executed. **Ready:** A process that is prepared to execute when given the opportunity. **Blocked:** A process that cannot execute until some event occurs, such as the completion of an I/O operation. **New:** A process that has just been created but has not yet been admitted to the pool of executable processes by the operating system. **Exit:** A process that has been released from the pool of executable processes by the operating system, either because it halted or because it aborted for some reason.
- 3.4 Process preemption occurs when an executing process is interrupted by the processor so that another process can be executed.
- 3.5 Swapping involves moving part or all of a process from main memory to disk. When none of the processes in main memory is in the Ready state, the operating system swaps one of the blocked processes out onto disk into a suspend queue, so that another process may be brought into main memory to execute.
- 3.6 There are two independent concepts: whether a process is waiting on an event (blocked or not), and whether a process has been swapped out of main memory (suspended or not). To accommodate this 2×2 combination, we need two Ready states and two Blocked states.
- 3.7 1. The process is not immediately available for execution. 2. The process may or may not be waiting on an event. If it is, this blocked condition is independent of the suspend condition, and occurrence of the blocking event does not enable the process to be executed. 3. The process was placed in a suspended state by an agent; either itself, a parent process, or the operating system, for the purpose of preventing its execution. 4. The process may not be removed from this state until the agent explicitly orders the removal.
- 3.8 The OS maintains tables for entities related to memory, I/O, files, and processes. See Table 3.10 for details.

- 3.9 Process identification, processor state information, and process control information.
- 3.10 The user mode has restrictions on the instructions that can be executed and the memory areas that can be accessed. This is to protect the operating system from damage or alteration. In kernel mode, the operating system does not have these restrictions, so that it can perform its tasks.
- 3.11 1. Assign a unique process identifier to the new process. 2. Allocate space for the process. 3. Initialize the process control block. 4. Set the appropriate linkages. 5. Create or expand other data structures.
- 3.12 An interrupt is due to some sort of event that is external to and independent of the currently running process, such as the completion of an I/O operation. A trap relates to an error or exception condition generated within the currently running process, such as an illegal file access attempt.
- 3.13 Clock interrupt, I/O interrupt, memory fault.
- 3.14 A mode switch may occur without changing the state of the process that is currently in the Running state. A process switch involves taking the currently executing process out of the Running state in favor of another process. The process switch involves saving more state information.

ANSWERS TO PROBLEMS

- 3.1
- **Creation and deletion of both user and system processes.** The processes in the system can execute concurrently for information sharing, computation speedup, modularity, and convenience. Concurrent execution requires a mechanism for process creation and deletion. The required resources are given to the process when it is created, or allocated to it while it is running. When the process terminates, the OS needs to reclaim any reusable resources.
 - **Suspension and resumption of processes.** In process scheduling, the OS needs to change the process's state to waiting or ready state when it is waiting for some resources. When the required resources are available, OS needs to change its state to running state to resume its execution.
 - **Provision of mechanism for process synchronization.** Cooperating processes may share data. Concurrent access to shared data may result in data inconsistency. OS has to provide mechanisms for processes synchronization to ensure the orderly execution of cooperating processes, so that data consistency is maintained.
 - **Provision of mechanism for process communication.** The processes executing under the OS may be either independent processes or cooperating processes. Cooperating processes must have the means to communicate with each other.
 - **Provision of mechanisms for deadlock handling.** In a multiprogramming environment, several processes may compete for a finite number of resources. If a deadlock occurs, all waiting processes will never change their waiting state to running state again, resources are wasted and jobs will never be completed.
- 3.2 a. There is no limit in principle to the number of processes that can be in the Ready and Blocked states. The OS may impose a limit based on resource

constraints related to the amount of memory devoted to the process state for each process. At most N processes can be in the Running state, one for each processor.

- b. Zero is the minimum number of processes in each state. It is possible for all processes to be in either the Running state (up to N processes) or the Ready state, with no process blocked. It is possible for all processes to be blocked waiting for I/O operations, with zero processes in the Ready and Running states.

3.3 a. **New → Ready or Ready/Suspend:** covered in text

Ready → Running or Ready/Suspend: covered in text

Ready/Suspend → Ready: covered in text

Blocked → Ready or Blocked/Suspend: covered in text

Blocked/Suspend → Ready /Suspend or Blocked: covered in text

Running → Ready, Ready/Suspend, or Blocked: covered in text

Any State → Exit: covered in text

- b. **New → Blocked, Blocked/Suspend, or Running:** A newly created process remains in the new state until the processor is ready to take on an additional process, at which time it goes to one of the Ready states.

Ready → Blocked or Blocked/Suspend: Typically, a process that is ready cannot subsequently be blocked until it has run. Some systems may allow the OS to block a process that is currently ready, perhaps to free up resources committed to the ready process.

Ready/Suspend → Blocked or Blocked/Suspend: Same reasoning as preceding entry.

Ready/Suspend → Running: The OS first brings the process into memory, which puts it into the Ready state.

Blocked → Ready /Suspend: this transition would be done in 2 stages. A blocked process cannot at the same time be made ready and suspended, because these transitions are triggered by two different causes.

Blocked → Running: When a process is unblocked, it is put into the Ready state. The dispatcher will only choose a process from the Ready state to run

Blocked/Suspend → Ready: same reasoning as Blocked → Ready /Suspend

Blocked/Suspend → Running: same reasoning as Blocked → Running

Running → Blocked/Suspend: this transition would be done in 2 stages

Exit → Any State: Can't turn back the clock

3.4 The following example is used in [PINK89] to clarify their definition of block and suspend:

Suppose a process has been executing for a while and needs an additional magnetic tape drive so that it can write out a temporary file. Before it can initiate a write to tape, it must be given permission to use one of the drives. When it makes its request, a tape drive may not be available, and if that is the case, the process will be placed in the blocked state. At some point, we assume the system will allocate the tape drive to the process; at that time the process will be moved back to the active state. When the process is placed into the execute state again it will request a write operation to its newly acquired tape drive. At this point, the process will be move to the suspend state, where it waits for the completion of the current write on the tape drive that it now owns.

The distinction made between two different reasons for waiting for a device could be useful to the operating system in organizing its work. However, it is no substitute for a knowledge of which processes are swapped out and which processes are swapped in. This latter distinction is a necessity and must be reflected in some fashion in the process state.

- 3.5 Figure 9.3 in Chapter 9 shows the result for a single blocked queue. The figure readily generalizes to multiple blocked queues.
- 3.6 Penalize the Ready, suspend processes by some fixed amount, such as one or two priority levels, so that a Ready, suspend process is chosen next only if it has a higher priority than the highest-priority Ready process by several levels of priority.
- 3.7 a. A separate queue is associated with each wait state. The differentiation of waiting processes into queues reduces the work needed to locate a waiting process when an event occurs that affects it. For example, when a page fault completes, the scheduler know that the waiting process can be found on the Page Fault Wait queue.
- b. In each case, it would be less efficient to allow the process to be swapped out while in this state. For example, on a page fault wait, it makes no sense to swap out a process when we are waiting to bring in another page so that it can execute.
- c. The state transition diagram can be derived from the following state transition table:

Current State	Next State				
	Currently Executing	Computable (resident)	Computable (outswapped)	Variety of wait states (resident)	Variety of wait states (outswapped)
Currently Executing		Rescheduled		Wait	
Computable (resident)	Scheduled		Outswap		
Computable (outswapped)		Inswap			
Variety of wait states (resident)		Event satisfied	Outswap		
Variety of wait states (outswapped)			Event satisfied		

- 3.8 a. The advantage of four modes is that there is more flexibility to control access to memory, allowing finer tuning of memory protection. The disadvantage is complexity and processing overhead. For example, procedures running at each of the access modes require separate stacks with appropriate accessibility.
- b. In principle, the more modes, the more flexibility, but it seems difficult to justify going beyond four.

- 3.9 a.** With $j < i$, a process running in D_i is prevented from accessing objects in D_j . Thus, if D_j contains information that is more privileged or is to be kept more secure than information in D_i , this restriction is appropriate. However, this security policy can be circumvented in the following way. A process running in D_j could read data in D_j and then copy that data into D_i . Subsequently, a process running in D_i could access the information.
- b.** An approach to dealing with this problem, known as a trusted system, is discussed in Chapter 16.
- 3.10 a.** An application may be processing data received from another process and storing the results on disk. If there is data waiting to be taken from the other process, the application may proceed to get that data and process it. If a previous disk write has completed and there is processed data to write out, the application may proceed to write to disk. There may be a point where the process is waiting both for additional data from the input process and for disk availability.
- b.** There are several ways that could be handled. A special type of either/or queue could be used. Or the process could be put in two separate queues. In either case, the operating system would have to handle the details of alerting the process to the occurrence of both events, one after the other.
- 3.11** This technique is based on the assumption that an interrupted process A will continue to run after the response to an interrupt. But, in general, an interrupt may cause the basic monitor to preempt a process A in favor of another process B . It is now necessary to copy the execution state of process A from the location associated with the interrupt to the process description associated with A . The machine might as well have stored them there in the first place. Source: [BRIN73].
- 3.12** Because there are circumstances under which a process may not be preempted (i.e., it is executing in kernel mode), it is impossible for the operating system to respond rapidly to real-time requirements.

CHAPTER 4 THREADS, SMP AND MICROKERNELS

ANSWERS TO QUESTIONS

- 4.1 This will differ from system to system, but in general, resources are owned by the process and each thread has its own execution state. A few general comments about each category in Table 3.5: **Identification:** the process must be identified but each thread within the process must have its own ID. **Processor State Information:** these are generally process-related. **Process control information:** scheduling and state information would mostly be at the thread level; data structuring could appear at both levels; interprocess communication and interthread communication may both be supported; privileges may be at both levels; memory management would generally be at the process level; and resource info would generally be at the process level.
- 4.2 Less state information is involved.
- 4.3 Resource ownership and scheduling/execution.
- 4.4 Foreground/background work; asynchronous processing; speedup of execution by parallel processing of data; modular program structure.
- 4.5 Address space, file resources, execution privileges are examples.
- 4.6 1. Thread switching does not require kernel mode privileges because all of the thread management data structures are within the user address space of a single process. Therefore, the process does not switch to the kernel mode to do thread management. This saves the overhead of two mode switches (user to kernel; kernel back to user). 2. Scheduling can be application specific. One application may benefit most from a simple round-robin scheduling algorithm, while another might benefit from a priority-based scheduling algorithm. The scheduling algorithm can be tailored to the application without disturbing the underlying OS scheduler. 3. ULTs can run on any operating system. No changes are required to the underlying kernel to support ULTs. The threads library is a set of application-level utilities shared by all applications.
- 4.7 1. In a typical operating system, many system calls are blocking. Thus, when a ULT executes a system call, not only is that thread blocked, but also all of the threads within the process are blocked. 2. In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing. A kernel assigns one process

to only one processor at a time. Therefore, only a single thread within a process can execute at a time.

- 4.8 Jacketing converts a blocking system call into a nonblocking system call by using an application-level I/O routine which checks the status of the I/O device.
- 4.9 **SIMD:** A single machine instruction controls the simultaneous execution of a number of processing elements on a lockstep basis. Each processing element has an associated data memory, so that each instruction is executed on a different set of data by the different processors.. **MIMD:** A set of processors simultaneously execute different instruction sequences on different data sets. **Master/slave:** The operating system kernel always runs on a particular processor. The other processors may only execute user programs and perhaps operating system utilities. **SMP:** the kernel can execute on any processor, and typically each processor does self-scheduling from the pool of available processes or threads. **Cluster:** Each processor has a dedicated memory, and is a self-contained computer.
- 4.10 Simultaneous concurrent processes or threads; scheduling; synchronization; memory management; reliability and fault tolerance.
- 4.11 Device drivers, file systems, virtual memory manager, windowing system, and security services.
- 4.12 **Uniform interfaces:** Processes need not distinguish between kernel-level and user-level services because all such services are provided by means of message passing. **Extensibility:** facilitates the addition of new services as well as the provision of multiple services in the same functional area. **Flexibility:** not only can new features be added to the operating system, but existing features can be subtracted to produce a smaller, more efficient implementation. **Portability:** all or at least much of the processor-specific code is in the microkernel; thus, changes needed to port the system to a new processor are fewer and tend to be arranged in logical groupings. **Reliability:** A small microkernel can be rigorously tested. Its use of a small number of application programming interfaces (APIs) improves the chance of producing quality code for the operating-system services outside the kernel. **Distributed system support:** the message orientation of microkernel communication lends itself to extension to distributed systems. **Support for object-oriented operating system (OOOS):** An object-oriented approach can lend discipline to the design of the microkernel and to the development of modular extensions to the operating system.
- 4.13 It takes longer to build and send a message via the microkernel, and accept and decode the reply, than to make a single service call.
- 4.14 These functions fall into the general categories of low-level memory management, inter-process communication (IPC), and I/O and interrupt management.
- 4.15 Messages.

ANSWERS TO PROBLEMS

- 4.1 Yes, because more state information must be saved to switch from one process to another.
- 4.2 Because, with ULTs, the thread structure of a process is not visible to the operating system, which only schedules on the basis of processes.
- 4.3 a. The use of sessions is well suited to the needs of an interactive graphics interface for personal computer and workstation use. It provides a uniform mechanism for keeping track of where graphics output and keyboard/mouse input should be directed, easing the task of the operating system.
b. The split would be the same as any other process/thread scheme, with address space and files assigned at the process level.
- 4.4 The issue here is that a machine spends a considerable amount of its waking hours waiting for I/O to complete. In a multithreaded program, one KLT can make the blocking system call, while the other KLTs can continue to run. On uniprocessors, a process that would otherwise have to block for all these calls can continue to run its other threads. Source: [LEWI96]
- 4.5 No. When a process exits, it takes everything with it—the KLTs, the process structure, the memory space, everything—including threads. Source: [LEWI96]
- 4.6 As much information as possible about an address space can be swapped out with the address space, thus conserving main memory.
- 4.7 a. If a conservative policy is used, at most $20/4 = 5$ processes can be active simultaneously. Because one of the drives allocated to each process can be idle most of the time, at most 5 drives will be idle at a time. In the best case, none of the drives will be idle.
b. To improve drive utilization, each process can be initially allocated with three tape drives. The fourth one will be allocated on demand. In this policy, at most $\lfloor 20/3 \rfloor = 6$ processes can be active simultaneously. The minimum number of idle drives is 0 and the maximum number is 2. Source: *Advanced Computer Architecture*, K. Hwang, 1993.
- 4.8 a. The function counts the number of positive elements in a list.
b. This should work correctly, because `count_positives` in this specific case does not update `global_positives`, and hence the two threads operate on distinct global data and require no locking. Source: Boehn, H. et al. "Multithreading in C and C++." ;login, February 2007.
- 4.9 This transformation is clearly consistent with the C language specification, which addresses only single-threaded execution. In a single-threaded environment, it is indistinguishable from the original. The pthread specification also contains no clear prohibition against this kind of transformation. And since it is a library and not a language specification, it is not clear that it could. However, in a multithreaded environment, the transformed version is quite different, in that it assigns to `global_positives`, even if the list contains only negative elements. The original program is now broken, because the update of `global_positives` by thread B may be lost, as a result of thread A writing back an earlier value of `global_positives`. By pthread rules, a thread-unaware compiler has turned a

perfectly legitimate program into one with undefined semantics. Source: Boehn, H. et al. "Multithreading in C and C++." ;login, February 2007.

- 4.10 a. This program creates a new thread. Both the main thread and the new thread then increment the global variable `myglobal` 20 times.
- b. Quite unexpected! Because `myglobal` starts at zero, and both the main thread and the new thread each increment it by 20, we should see `myglobal` equaling 40 at the end of the program. But `myglobal` equals 21, so we know that something fishy is going on here. In `thread_function()`, notice that we copy `myglobal` into a local variable called `j`. The program increments `j`, then sleeps for one second, and only then copies the new `j` value into `myglobal`. That's the key. Imagine what happens if our main thread increments `myglobal` just after our new thread copies the value of `myglobal` into `j`. When `thread_function()` writes the value of `j` back to `myglobal`, it will overwrite the modification that the main thread made. Source: Robbins, D. "POSIX Threads Explained." *IBM Developer Works*, July 2000. ibm.com/developerworks/library/l-posix1.html
- 4.11 Every call that can possibly change the priority of a thread or make a higher-priority thread runnable will also call the scheduler, and it in turn will preempt the lower-priority active thread. So there will never be a runnable, higher-priority thread. Source: [LEWI96]
- 4.12 a. Some programs have logical parallelism that can be exploited to simplify and structure the code but do not need hardware parallelism. For example, an application that employs multiple windows, only one of which is active at a time, could with advantage be implemented as a set of ULTs on a single LWP. The advantage of restricting such applications to ULTs is efficiency. ULTs may be created, destroyed, blocked, activated, and so on. without involving the kernel. If each ULT were known to the kernel, the kernel would have to allocate kernel data structures for each one and perform thread switching. Kernel-level thread switching is more expensive than user-level thread switching.
- b. The execution of user-level threads is managed by the threads library whereas the LWP is managed by the kernel.
- c. An unbound thread can be in one of four states: runnable, active, sleeping, or stopped. These states are managed by the threads library. A ULT in the active state is currently assigned to a LWP and executes while the underlying kernel thread executes. We can view the LWP state diagram as a detailed description of the ULT active state, because an thread only has an LWP assigned to it when it is in the Active state. The LWP state diagram is reasonably self-explanatory. An active thread is only executing when its LWP is in the Running state. When an active thread executes a blocking system call, the LWP enters the Blocked state. However, the ULT remains bound to that LWP and, as far as the threads library is concerned, that ULT remains active.
- 4.13 As the text describes, the Uninterruptible state is another blocked state. The difference between this and the Interruptible state is that in an uninterruptible state, a process is waiting directly on hardware conditions and therefore will not handle any signals. This is used in situations where the process must wait without interruption or when the event is expected to occur quite quickly. For

example, this state may be used when a process opens a device file and the corresponding device driver starts probing for a corresponding hardware device. The device driver must not be interrupted until the probing is complete, or the hardware device could be left in an unpredictable state.

Please Do Not
Post on Web

CHAPTER 5 CONCURRENCY: MUTUAL EXCLUSION AND SYNCHRONIZATION

ANSWERS TO QUESTIONS

- 5.1 Communication among processes, sharing of and competing for resources, synchronization of the activities of multiple processes, and allocation of processor time to processes.
- 5.2 Multiple applications, structured applications, operating-system structure.
- 5.3 The ability to enforce mutual exclusion.
- 5.4 **Processes unaware of each other:** These are independent processes that are not intended to work together. **Processes indirectly aware of each other:** These are processes that are not necessarily aware of each other by their respective process IDs, but that share access to some object, such as an I/O buffer. **Processes directly aware of each other:** These are processes that are able to communicate with each other by process ID and which are designed to work jointly on some activity.
- 5.5 **Competing processes** need access to the same resource at the same time, such as a disk, file, or printer. **Cooperating processes** either share access to a common object, such as a memory buffer or are able to communicate with each other, and cooperate in the performance of some application or activity.
- 5.6 **Mutual exclusion:** competing processes can only access a resource that both wish to access one at a time; mutual exclusion mechanisms must enforce this one-at-a-time policy. **Deadlock:** if competing processes need exclusive access to more than one resource then deadlock can occur if each processes gained control of one resource and is waiting for the other resource. **Starvation:** one of a set of competing processes may be indefinitely denied access to a needed resource because other members of the set are monopolizing that resource.
- 5.7 1. Mutual exclusion must be enforced: only one process at a time is allowed into its critical section, among all processes that have critical sections for the same resource or shared object. 2. A process that halts in its non-critical section must do so without interfering with other processes. 3. It must not be possible for a process requiring access to a critical section to be delayed indefinitely: no deadlock or starvation. 4. When no process is in a critical section, any process that requests entry to its critical section must be permitted to enter without delay. 5. No assumptions are made about relative process speeds or number of processors. 6. A process remains inside its critical section for a finite time only.

- 5.8 1. A semaphore may be initialized to a nonnegative value. 2. The *wait* operation decrements the semaphore value. If the value becomes negative, then the process executing the *wait* is blocked. 3. The *signal* operation increments the semaphore value. If the value is not positive, then a process blocked by a *wait* operation is unblocked.
- 5.9 A binary semaphore may only take on the values 0 and 1. A general semaphore may take on any integer value.
- 5.10 A strong semaphore requires that processes that are blocked on that semaphore are unblocked using a first-in-first-out policy. A weak semaphore does not dictate the order in which blocked processes are unblocked.
- 5.11 A monitor is a programming language construct providing abstract data types and mutually exclusive access to a set of procedures
- 5.12 There are two aspects, the send and receive primitives. When a *send* primitive is executed in a process, there are two possibilities: either the sending process is blocked until the message is received, or it is not. Similarly, when a process issues a *receive* primitive, there are two possibilities: If a message has previously been sent, the message is received and execution continues. If there is no waiting message, then either (a) the process is blocked until a message arrives, or (b) the process continues to execute, abandoning the attempt to receive.
- 5.13 1. Any number of readers may simultaneously read the file. 2. Only one writer at a time may write to the file. 3. If a writer is writing to the file, no reader may read it.

ANSWERS TO PROBLEMS

- 5.1 On uniprocessors you can avoid interruption and thus concurrency by disabling interrupts. Also on multiprocessor machines another problem arises: memory ordering (multiple processors accessing the memory unit).
- 5.2 b. The read coroutine reads the cards and passes characters through a one-character buffer, *rs*, to the squash coroutine. The read coroutine also passes the extra blank at the end of every card image. The squash coroutine need know nothing about the 80-character structure of the input; it simply looks for double asterisks and passes a stream of modified characters to the print coroutine via a one-character buffer, *sp*. Finally, print simply accepts an incoming stream of characters and prints it as a sequence of 125-character lines.
- d. This can be accomplished using three concurrent processes. One of these, Input, reads the cards and simply passes the characters (with the additional trailing space) through a finite buffer, say InBuffer, to a process Squash which simply looks for double asterisks and passes a stream of modified characters through a second finite buffer, say OutBuffer, to a process Output, which extracts the characters from the second buffer and prints them in 15 column lines. A producer/consumer semaphore approach can be used.

5.3 ABCDE; ABDCE; ABDEC; ADBCE; ADBEC; ADEBC;
DEABC; DAEBC; DABEC; DABCE

5.4 a. On casual inspection, it appears that `tally` will fall in the range $50 \leq \text{tally} \leq 100$ since from 0 to 50 increments could go unrecorded due to the lack of mutual exclusion. The basic argument contends that by running these two processes concurrently we should not be able to derive a result lower than the result produced by executing just one of these processes sequentially. But consider the following interleaved sequence of the load, increment, and store operations performed by these two processes when altering the value of the shared variable:

1. Process A loads the value of `tally`, increments *tally*, but then loses the processor (it has incremented its register to 1, but has not yet stored this value).
2. Process B loads the value of `tally` (still zero) and performs forty-nine complete increment operations, losing the processor after it has stored the value 49 into the shared variable `tally`.
3. Process A regains control long enough to perform its first store operation (replacing the previous `tally` value of 49 with 1) but is then immediately forced to relinquish the processor.
4. Process B resumes long enough to load 1 (the current value of *tally*) into its register, but then it too is forced to give up the processor (note that this was B's final load).
5. Process A is rescheduled, but this time it is not interrupted and runs to completion, performing its remaining 49 load, increment, and store operations, which results in setting the value of `tally` to 50.
6. Process B is reactivated with only one increment and store operation to perform before it terminates. It increments its register value to 2 and stores this value as the final value of the shared variable.

Some thought will reveal that a value lower than 2 cannot occur. Thus, the proper range of final values is $2 \leq \text{tally} \leq 100$.

- b. For the generalized case of N processes, the range of final values is $2 \leq \text{tally} \leq (N \times 50)$, since it is possible for all other processes to be initially scheduled and run to completion in step (5) before Process B would finally destroy their work by finishing last.

Source: [RUDO90]. A slightly different formulation of the same problem appears in [BEN98]

5.5 On average, yes, because busy-waiting consumes useless instruction cycles.

However, in a particular case, if a process comes to a point in the program where it must wait for a condition to be satisfied, and if that condition is already satisfied, then the busy-wait will find that out immediately, whereas, the blocking wait will consume OS resources switching out of and back into the process.

5.6 Consider the case in which `turn` equals 0 and `P(1)` sets `blocked[1]` to `true` and then finds `blocked[0]` set to `false`. `P(0)` will then set `blocked[0]` to `true`, find `turn = 0`, and enter its critical section. `P(1)` will then assign 1 to `turn` and will also enter its critical section. The error was pointed out in [RAYN86].

- 5.7 a. When a process wishes to enter its critical section, it is assigned a ticket number. The ticket number assigned is calculated by adding one to the largest of the ticket numbers currently held by the processes waiting to enter their critical section and the process already in its critical section. The process with the smallest ticket number has the highest precedence for entering its critical section. In case more than one process receives the same ticket number, the process with the smallest numerical name enters its critical section. When a process exits its critical section, it resets its ticket number to zero.
- b. If each process is assigned a unique process number, then there is a unique, strict ordering of processes at all times. Therefore, deadlock cannot occur.
- c. To demonstrate mutual exclusion, we first need to prove the following lemma: if P_i is in its critical section, and P_k has calculated its number[k] and is attempting to enter its critical section, then the following relationship holds:

$$(\text{number}[i], i) < (\text{number}[k], k)$$

To prove the lemma, define the following times:

- T_{w1} P_i reads choosing[k] for the last time, for $j = k$, in its first wait, so we have choosing[k] = false at T_{w1} .
- T_{w2} P_i begins its final execution, for $j = k$, of the second **while** loop. We therefore have $T_{w1} < T_{w2}$.
- T_{k1} P_k enters the beginning of the **repeat** loop.
- T_{k2} P_k finishes calculating number[k].
- T_{k3} P_k sets choosing[k] to false. We have $T_{k1} < T_{k2} < T_{k3}$.

Since at T_{w1} , choosing[k] = false, we have either $T_{w1} < T_{k1}$ or $T_{k3} < T_{w1}$. In the first case, we have number[i] < number[k], since P_i was assigned its number prior to P_k ; this satisfies the condition of the lemma.

In the second case, we have $T_{k2} < T_{k3} < T_{w1} < T_{w2}$, and therefore $T_{k2} < T_{w2}$. This means that at T_{w2} , P_i has read the current value of number[k]. Moreover, as T_{w2} is the moment at which the final execution of the second **while** for $j = k$ takes place, we have $(\text{number}[i], i) < (\text{number}[k], k)$, which completes the proof of the lemma.

It is now easy to show the mutual exclusion is enforced. Assume that P_i is in its critical section and P_k is attempting to enter its critical section. P_k will be unable to enter its critical section, as it will find number[i] $\neq 0$ and $(\text{number}[i], i) < (\text{number}[k], k)$.

- 5.8 Suppose we have two processes just beginning; call them p_0 and p_1 . Both reach line 3 at the same time. Now, we'll assume both read number[0] and number[1] before either addition takes place. Let p_1 complete the line, assigning 1 to number[1], but p_0 block before the assignment. Then p_1 gets through the while loop at line 5 and enters the critical section. While in the critical section, it blocks; p_0 unblocks, and assigns 1 to number[0] at line 3. It proceeds to the while loop at line 5. When it goes through that loop for $j = 1$, the first condition on line 5 is true. Further, the second condition on line 5 is false, so p_0 enters the critical section. Now p_0 and p_1 are both in the critical section, violating mutual exclusion. The

reason for choosing is to prevent the while loop in line 5 from being entered when process j is setting its $\text{number}[j]$. Note that if the loop is entered and then process j reaches line 3, one of two situations arises. Either $\text{number}[j]$ has the value 0 when the first test is executed, in which case process i moves on to the next process, or $\text{number}[j]$ has a non-zero value, in which case at some point $\text{number}[j]$ will be greater than $\text{number}[i]$ (since process i finished executing statement 3 before process j began). Either way, process i will enter the critical section before process j , and when process j reaches the while loop, it will loop at least until process i leaves the critical section. Source: Matt Bishop, UC Davis

- 5.9 This is a program that provides mutual exclusion for access to a critical resource among N processes, which can only use the resource one at a time. The unique feature of this algorithm is that a process need wait no more than $N - 1$ turns for access. The values of $\text{control}[i]$ for process i are interpreted as follows: 0 = outside the critical section and not seeking entry; 1 = wants to access critical section; 2 = has claimed precedence for entering the critical section. The value of k reflects whose turn it is to enter the critical section. **Entry algorithm:** Process i expresses the intention to enter the critical section by setting $\text{control}[i] = 1$. If no other process between k and i (in circular order) has expressed a similar intention then process i claims its precedence for entering the critical section by setting $\text{control}[i] = 2$. If i is the only process to have made a claim, it enters the critical section by setting $k = i$; if there is contention, i restarts the entry algorithm. **Exit algorithm:** Process i examines the array control in circular fashion beginning with entry $i + 1$. If process i finds a process with a nonzero control entry, then k is set to the identifier of that process.

The original paper makes the following observations: First observe that no two processes can be simultaneously processing between their statements L3 and L6. Secondly, observe that the system cannot be blocked; for if none of the processes contending for access to its critical section has yet passed its statement L3, then after a point, the value of k will be constant, and the first contending process in the cyclic ordering ($k, k + 1, \dots, N, 1, \dots, k - 1$) will meet no resistance. Finally, observe that no single process can be blocked. Before any process having executed its critical section can exit the area protected from simultaneous processing, it must designate as its unique successor the first contending process in the cyclic ordering, assuring the choice of any individual contending process within $N - 1$ turns. Original paper: Eisenberg, A., and McGuire, M. "Other Comments on Dijkstra's Concurrent Programming Control Problem." *Communications of the ACM*, November 1972.

- 5.10 a. `exchange(keyi, bolt)`
 b. The statement `bolt = 0` is preferable. An atomic statement such as `exchange` will use more resources.


```

5.11  var   j: 0..n-1;
        key: boolean;
    while (true) {
        waiting[i] = true;
        key := true;
        while (waiting[i] && key)
            key = (compare_and_swap(lock, 0, 1) == 0);
        waiting[i] = false;
        < critical section >
        j = i + 1 mod n;
        while (j != i && !waiting[j]) j = j + 1 mod n;
        if (j == i) lock := false
        else waiting = false;
        < remainder section >
    }

```

The algorithm uses the common data structures

```

    var   waiting: array [0..n - 1] of boolean
        lock: boolean

```

These data structures are initialized to false. When a process leaves its critical section, it scans the array *waiting* in the cyclic ordering ($i + 1, i + 2, \dots, n - 1, 0, \dots, i - 1$). It designates the first process in this ordering that is in the entry section ($waiting[j] = \text{true}$) as the next one to enter the critical section. Any process waiting to enter its critical section will thus do so within $n - 1$ turns.

```

5.12 /* program mutualexclusion */
    const int n = /* number of processes */;
    int bolt;
    void P(int i)
    {
        while (true) {
            while (!testset (bolt)) /* do nothing */;
            /* critical section */;
            bolt = 0;
            /* remainder */;
        }
    }
    void main()
    {
        bolt = 0;
        parbegin (P(1), P(2), . . . ,P(n));
    }

```

5.13 The two are equivalent. In the definition of Figure 5.3, when the value of the semaphore is negative, its value tells you how many processes are waiting. With the definition of this problem, you don't have that information readily available. However, the two versions function the same.

- 5.14 This problem and the next two are based on examples in; Reek, K. "Design Patterns for Semaphores." ACM SIGCSE'04, March 2004.
- a. We quote the explanation in Reek's paper. There are two problems. First, because unblocked processes must reenter the mutual exclusion (line 10) there is a chance that newly arriving processes (at line 5) will beat them into the critical section. Second, there is a time delay between when the waiting processes are unblocked and when they resume execution and update the counters. The waiting processes must be accounted for as soon as they are unblocked (because they might resume execution at any time), but it may be some time before the processes actually do resume and update the counters to reflect this. To illustrate, consider the case where three processes are blocked at line 9. The last active process will unblock them (lines 25-28) as it departs. But there is no way to predict when these processes will resume executing and update the counters to reflect the fact that they have become active. If a new process reaches line 6 before the unblocked ones resume, the new one should be blocked. But the status variables have not yet been updated so the new process will gain access to the resource. When the unblocked ones eventually resume execution, they will also begin accessing the resource. The solution has failed because it has allowed four processes to access the resource together.
 - b. This forces unblocked processes to recheck whether they can begin using the resource. But this solution is more prone to starvation because it encourages new arrivals to "cut in line" ahead of those that were already waiting.
- 5.15
- a. This approach is to eliminate the time delay. If the departing process updates the waiting and active counters as it unblocks waiting processes, the counters will accurately reflect the new state of the system before any new processes can get into the mutual exclusion. Because the updating is already done, the unblocked processes need not reenter the critical section at all. Implementing this pattern is easy. Identify all of the work that would have been done by an unblocked process and make the unblocking process do it instead.
 - b. Suppose three processes arrived when the resource was busy, but one of them lost its quantum just before blocking itself at line 9 (which is unlikely, but certainly possible). When the last active process departs, it will do three `semSignal` operations and set `must_wait` to true. If a new process arrives before the older ones resume, the new one will decide to block itself. However, it will breeze past the `semWait` in line 9 without blocking, and when the process that lost its quantum earlier runs it will block itself instead. This is not an error—the problem doesn't dictate which processes access the resource, only how many are allowed to access it. Indeed, because the unblocking order of semaphores is implementation dependent, the only portable way to ensure that processes proceed in a particular order is to block each on its own semaphore.
 - c. The departing process updates the system state on behalf of the processes it unblocks.
- 5.16
- a. After you unblock a waiting process, you leave the critical section (or block yourself) without opening the mutual exclusion. The unblocked process doesn't reenter the mutual exclusion—it takes over your ownership of it. The process can therefore safely update the system state on its own. When it is finished, it reopens the mutual exclusion. Newly arriving processes can no longer cut in line because they cannot enter the mutual exclusion until the unblocked process has finished. Because the unblocked process takes care of its

own updating, the cohesion of this solution is better. However, once you have unblocked a process, you must immediately stop accessing the variables protected by the mutual exclusion. The safest approach is to immediately leave (after line 26, the process leaves without opening the mutex) or block yourself.

- b. Only one waiting process can be unblocked even if several are waiting—to unblock more would violate the mutual exclusion of the status variables. This problem is solved by having the newly unblocked process check whether more processes should be unblocked (line 14). If so, it passes the baton to one of them (line 15); if not, it opens up the mutual exclusion for new arrivals (line 17).
- c. This pattern synchronizes processes like runners in a relay race. As each runner finishes her laps, she passes the baton to the next runner. “Having the baton” is like having permission to be on the track. In the synchronization world, being in the mutual exclusion is analogous to having the baton—only one person can have it..

5.17 Suppose two processes each call `semWait(s)` when `s` is initially 0, and after the first has just done `semSignalB(mutex)` but not done `semWaitB(delay)`, the second call to `semWait(s)` proceeds to the same point. Because `s = -2` and `mutex` is unlocked, if two other processes then successively execute their calls to `semSignal(s)` at that moment, they will each do `semSignalB(delay)`, but the effect of the second `semSignalB` is not defined.

The solution is to move the **else** line, which appears just before the **end** line in `semWait` to just before the **end** line in `semSignal`. Thus, the last `semSignalB(mutex)` in `semWait` becomes unconditional and the `semSignalB(mutex)` in `semSignal` becomes conditional. For a discussion, see “A Correct Implementation of General Semaphores,” by Hemmendinger, *Operating Systems Review*, July 1988.

5.18 The program is found in [RAYN86]:

```

var a, b, m: semaphore;
    na, nm: 0 ... +∞;
a := 1; b := 1; m := 0; na := 0; nm := 0;
semWait(b); na ← na + 1; semSignal(b);
semWait(a); nm ← nm + 1;
    semWait(b); na ← na - 1;
    if na = 0 then semSignal(b); semSignal(m)
        else semSignal(b); semSignal(a)
    endif;
semWait(m); nm ← nm - 1;
<critical section>;
if nm = 0 then semSignal(a)
    else semSignal(m)
endif;

```

5.19 The code has a major problem. The `V(passenger_released)` in the car code can unblock a passenger blocked on `P(passenger_released)` that is NOT the one riding in the car that did the `V()`.

5.20

	Producer	Consumer	s	n	delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	if (n==1) (semSignalB(delay))		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		if (n==0) (semWaitB(delay))			
10	semWaitB(s)				

Both producer and consumer are blocked.

5.21 This solution is from [BEN82].

```
program    producerconsumer;
var        n: integer;
           s: (*binary*) semaphore (:= 1);
           delay: (*binary*) semaphore (:= 0);

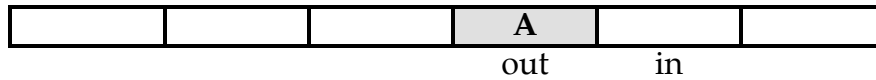
procedure producer;
begin
  repeat
    produce;
    semWaitB(s);
    append;
    n := n + 1;
    if n=0 then semSignalB(delay);
    semSignalB(s)
  forever
end;

procedure consumer;
begin
  repeat
    semWaitB(s);
    take;
    n := n - 1;
    if n = -1 then
      begin
        semSignalB(s);
        semWaitB(delay);
        semWaitB(s)
      end;
    consume;
    semSignalB(s)
  forever
end;

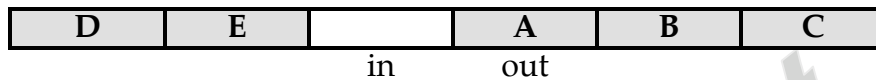
begin (*main program*)
  n := 0;
  parbegin
    producer; consumer
  parend
end.
```

5.22 Any of the interchanges listed would result in an incorrect program. The semaphore s controls access to the critical region and you only want the critical region to include the append or take function.

- 5.23 a. If the buffer is allowed to contain n entries, then the problem is to distinguish an empty buffer from a full one. Consider a buffer of six slots, with only one entry, as follows:



Then, when that one element is removed, $out = in$. Now suppose that the buffer is one element shy of being full:



Here, $out = in + 1$. But then, when an element is added, in is incremented by 1 and $out = in$, the same as when the buffer is empty.

- b. You could use an auxiliary variable, `count`, which is incremented and decremented appropriately.

Please Do Not
Post on Web

5.24

```

#define REINDEER 9 /* max # of reindeer */
#define ELVES 3 /* size of elf group */
/* Semaphores */
only_elves = 3, /* 3 go to Santa */
emutex = 1, /* update elf_cnt */
rmutex = 1, /* update rein_ct */
rein_semaphore = 0, /* block early arrivals
back from islands */
sleigh = 0, /* all reindeer
semWait
around the sleigh */
done = 0, /* toys all delivered */
santa_semaphore = 0, /* 1st 2 elves semWait
on
this outside Santa's shop
*/
santa = 0, /* Santa sleeps on this
blocked semaphore
*/
problem = 0, /* semWait to pose
the
question to Santa */
elf_done = 0; /* receive reply */
/* Shared Integers */
rein_ct = 0; /* # of reindeer back
*/
elf_ct = 0; /* # of elves with problem
*/
/* Reindeer Process */
for (;;) {
tan on the beaches in the Pacific until
Christmas is close
semWait (rmutex)
rein_ct++
if (rein_ct == REINDEER) {
semSignal (rmutex)
semSignal (santa)
}
else {
semSignal (rmutex)
semWait (rein_semaphore)
}
}
/* all reindeer semWaiting to be attached to
sleigh */
semWait (sleigh)
fly off to deliver toys
semWait (done)
}

/* Elf Process */
for (;;) {
semWait (only_elves) /* only 3 elves
"in" */
semWait (emutex)
elf_ct++
if (elf_ct == ELVES) {
semSignal (emutex)
semSignal (santa) /* 3rd elf wakes
Santa */
}
else {
semSignal (emutex)
semWait (santa_semaphore) /*
semWait outside
Santa's shop door */
}
semWait (problem)
ask question /* Santa woke elf up */
semWait (elf_done)
semSignal (only_elves)
} /* end "forever" loop */
}
/* Santa Process */
for (;;) {
semWait (santa) /* Santa "rests" */
/* mutual exclusion is not needed on rein_ct
because if it is not equal to REINDEER,
then elves woke up Santa */
if (rein_ct == REINDEER) {
semWait (rmutex)
rein_ct = 0 /* reset while blocked */
semSignal (rmutex)
for (i = 0; i < REINDEER - 1; i++)
semSignal (rein_semaphore)
for (i = 0; i < REINDEER; i++)
semSignal (sleigh)
deliver all the toys and return
for (i = 0; i < REINDEER; i++)
semSignal (done)
}
else {
/* 3 elves have arrive */
for (i = 0; i < ELVES - 1; i++)
semSignal (santa_semaphore)
semWait (emutex)
elf_ct = 0
semSignal (emutex)
for (i = 0; i < ELVES; i++) {
semSignal (problem)
}
}
}

```

```

head back to the Pacific islands
} /* end "forever" loop */

                                answer that question
                                semSignal (elf_done)
                                }
                                }
                                } /* end "forever" loop */

```

- 5.25 a. There is an array of message slots that constitutes the buffer. Each process maintains a linked list of slots in the buffer that constitute the mailbox for that process. The message operations can be implemented as:

send (message, dest)	
semWait (mbuf)	semWait for message buffer available
semWait (mutex)	mutual exclusion on message queue
acquire free buffer slot	
copy message to slot	
link slot to other messages	
semSignal (dest.sem)	wake destination process
semSignal (mutex)	release mutual exclusion
receive (message)	
semWait (own.sem)	semWait for message to arrive
semWait (mutex)	mutual exclusion on message queue
unlink slot from own.queue	
copy buffer slot to message	
add buffer slot to freelist	
semSignal (mbuf)	indicate message slot freed
semSignal (mutex)	release mutual exclusion

where mbuf is initialized to the total number of message slots available; own and dest refer to the queue of messages for each process, and are initially zero.

- b. This solution is taken from [TANE97]. The synchronization process maintains a counter and a linked list of waiting processes for each semaphore. To do a WAIT or SIGNAL, a process calls the corresponding library procedure, *wait* or *signal*, which sends a message to the synchronization process specifying both the operation desired and the semaphore to be used. The library procedure then does a RECEIVE to get the reply from the synchronization process.

When the message arrives, the synchronization process checks the counter to see if the required operation can be completed. SIGNALs can always complete, but WAITs will block if the value of the semaphore is 0. If the operation is allowed, the synchronization process sends back an empty message, thus unblocking the caller. If, however, the operation is a WAIT and the semaphore is 0, the synchronization process enters the caller onto the queue and does not send a reply. The result is that the process doing the WAIT is blocked, just as it should be. Later, when a SIGNAL is done, the synchronization process picks one of the processes blocked on the semaphore, either in FIFO order, priority order, or some other order, and sends a reply. Race conditions are avoided here because the synchronization process handles only one request at a time.

CHAPTER 6 CONCURRENCY: DEADLOCK AND STARVATION

ANSWERS TO QUESTIONS

- 6.1 Examples of reusable resources are processors, I/O channels, main and secondary memory, devices, and data structures such as files, databases, and semaphores. Examples of consumable resources are interrupts, signals, messages, and information in I/O buffers.
- 6.2 **Mutual exclusion.** Only one process may use a resource at a time. **Hold and wait.** A process may hold allocated resources while awaiting assignment of others. **No preemption.** No resource can be forcibly removed from a process holding it.
- 6.3 The above three conditions, plus: **Circular wait.** A closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain.
- 6.4 The hold-and-wait condition can be prevented by requiring that a process request all of its required resources at one time, and blocking the process until all requests can be granted simultaneously.
- 6.5 First, if a process holding certain resources is denied a further request, that process must release its original resources and, if necessary, request them again together with the additional resource. Alternatively, if a process requests a resource that is currently held by another process, the operating system may preempt the second process and require it to release its resources.
- 6.6 The circular-wait condition can be prevented by defining a linear ordering of resource types. If a process has been allocated resources of type R, then it may subsequently request only those resources of types following R in the ordering.
- 6.7 **Deadlock prevention** constrains resource requests to prevent at least one of the four conditions of deadlock; this is either done indirectly, by preventing one of the three necessary policy conditions (mutual exclusion, hold and wait, no preemption), or directly, by preventing circular wait. **Deadlock avoidance** allows the three necessary conditions, but makes judicious choices to assure that the deadlock point is never reached. With **deadlock detection**, requested resources are granted to processes whenever possible.; periodically, the operating system performs an algorithm that allows it to detect the circular wait condition.

ANSWERS TO PROBLEMS

- 6.1 Mutual exclusion:** Only one car can occupy a given quadrant of the intersection at a time. **Hold and wait:** No car ever backs up; each car in the intersection waits until the quadrant in front of it is available. **No preemption:** No car is allowed to force another car out of its way. **Circular wait:** Each car is waiting for a quadrant of the intersection occupied by another car.
- 6.2 Prevention:** Hold-and-wait approach: Require that a car request both quadrants that it needs and blocking the car until both quadrants can be granted. No preemption approach: releasing an assigned quadrant is problematic, because this means backing up, which may not be possible if there is another car behind this car. Circular-wait approach: assign a linear ordering to the quadrants.
Avoidance: The algorithms discussed in the chapter apply to this problem. Essentially, deadlock is avoided by not granting requests that might lead to deadlock.
Detection: The problem here again is one of backup.
- 6.3**
1. Q acquires B and A, and then releases B and A. When P resumes execution, it will be able to acquire both resources.
 2. Q acquires B and A. P executes and blocks on a request for A. Q releases B and A. When P resumes execution, it will be able to acquire both resources.
 3. Q acquires B and then P acquires and releases A. Q acquires A and then releases B and A. When P resumes execution, it will be able to acquire B.
 4. P acquires A and then Q acquires B. P releases A. Q acquires A and then releases B. P acquires B and then releases B.
 5. P acquires and then releases A. P acquires B. Q executes and blocks on request for B. P releases B. When Q resumes execution, it will be able to acquire both resources.
 6. P acquires A and releases A and then acquires and releases B. When Q resumes execution, it will be able to acquire both resources.
- 6.4** If Q acquires B and A before P requests A, then Q can use these two resources and then release them, allowing A to proceed. If P acquires A before Q requests A, then at most Q can proceed to the point of requesting A and then is blocked. However, once P releases A, Q can proceed. Once Q releases B, A can proceed.
- 6.5**
- a. 0 0 0 0
 0 7 5 0
 6 6 2 2
 2 0 0 2
 0 3 2 0
- b. to d. Running the banker's algorithm, we see processes can finish in the order p1, p4, p5, p2, p3.
- e. Change available to (2,0,0,0) and p3's row of "still needs" to (6,5,2,2). Now p1, p4, p5 can finish, but with available now (4,6,9,8) neither p2 nor p3's "still needs" can be satisfied. So it is not safe to grant p3's request.
- 6.6**
1. $W = (2\ 1\ 0\ 0)$
 2. Mark P3; $W = (2\ 1\ 0\ 0) + (0\ 1\ 2\ 0) = (2\ 2\ 2\ 0)$

3. Mark P2; $W = (2\ 2\ 2\ 0) + (2\ 0\ 0\ 1) = (4\ 2\ 2\ 1)$
 4. Mark P1; no deadlock detected

- 6.7 A deadlock occurs when process I has filled the disk with input ($i = \max$) and process i is waiting to transfer more input to the disk, while process P is waiting to transfer more output to the disk and process O is waiting to transfer more output from the disk. Source: [BRIN73].
- 6.8 Reserve a minimum number of blocks (called *reso*) permanently for output buffering, but permit the number of output blocks to exceed this limit when disk space is available. The resource constraints now become:

$$\begin{aligned} i + o &\leq \max \\ i &\leq \max - \text{reso} \end{aligned}$$

where

$$0 < \text{reso} < \max$$

If process P is waiting to deliver output to the disk, process O will eventually consume all previous output and make at least *reso* pages available for further output, thus enabling P to continue. So P cannot be delayed indefinitely by O. Process I can be delayed if the disk is full of I/O; but sooner or later, all previous input will be consumed by P and the corresponding output will be consumed by O, thus enabling I to continue. Source: [BRIN73].

- 6.9
$$\begin{aligned} i + o + p &\leq \max - \\ i + o &\leq \max - \text{resp} \\ i + p &\leq \max - \text{reso} \\ i &\leq \max - (\text{reso} + \text{resp}) \end{aligned}$$

 Source: [BRIN73].

- 6.10 a.
 1. $i \leftarrow i + 1$
 2. $i \leftarrow i - 1; p \leftarrow p + 1$
 3. $p \leftarrow p - 1; o \leftarrow o + 1$
 4. $o \leftarrow o - 1$
 5. $p \leftarrow p + 1$
 6. $p \leftarrow p - 1$
- b. By examining the resource constraints listed in the solution to problem 6.7, we can conclude the following:
6. Procedure returns can take place immediately because they only release resources.
 5. Procedure calls may exhaust the disk ($p = \max - \text{reso}$) and lead to deadlock.
 4. Output consumption can take place immediately after output becomes available.
 3. Output production can be delayed temporarily until all previous output has been consumed and made at least *reso* pages available for further output.
 2. Input consumption can take place immediately after input becomes available.
 1. Input production can be delayed until all previous input and the corresponding output has been consumed. At this point, when $i = o = 0$,

input can be produced provided the user processes have not exhausted the disk ($p < \text{max} - \text{reso}$).

Conclusion: the uncontrolled amount of storage assigned to the user processes is the only possible source of a storage deadlock. Source: [BRIN73].

6.11 a. Creating the process would result in the state:

Process	Max	Hold	Claim	Free
1	70	45	25	25
2	60	40	20	
3	60	15	45	
4	60	25	35	

There is sufficient free memory to guarantee the termination of either P1 or P2. After that, the remaining three jobs can be completed in any order.

b. Creating the process would result in the trivially unsafe state:

Process	Max	Hold	Claim	Free
1	70	45	25	15
2	60	40	20	
3	60	15	45	
4	60	35	25	

6.12 It is unrealistic: don't know max demands in advance, number of processes can change over time, number of resources can change over time (something can break). Most OS's ignore deadlock. But Solaris only lets the superuser use the last process table slot.

- 6.13 a. The buffer is declared to be an array of shared elements of type T. Another array defines the number of input elements *available* to each process. Each process keeps track of the index *j* of the buffer element it is referring to at the moment.

```

var buffer: array 0..max-1 of shared T;
    available: shared array 0..n-1 of 0..max;

"Initialization"
var K: 1..n-1;
region available do
begin
    available(0) := max;
    for every k do available (k) := 0;
end

"Process i"
var j: 0..max-1; succ: 0..n-1;
begin
    j := 0; succ := (i+1) mod n;
    repeat
        region available do
            await available (i) > 0;
            region buffer(j) do consume element;
            region available do
                begin
                    available (i) := available(i) - 1;
                    available (succ) := available (succ) + 1;
                end
            j := (j+1) mod max;
        forever
    end

```

In the above program, the construct **region** defines a critical region using some appropriate mutual-exclusion mechanism. The notation

region v do S

means that at most one process at a time can enter the critical region associated with variable *v* to perform statement *S*.

b. A deadlock is a situation in which:

P_0 waits for P_{n-1} AND
 P_1 waits for P_0 AND
 $\dots\dots$
 P_{n-1} waits for P_{n-2}

because

(available (0) = 0) AND
 (available (1) = 0) AND
 $\dots\dots$
 (available (n-1) = 0)

But if $\max > 0$, this condition cannot hold because the critical regions satisfy the following invariant:

$$\sum_{i=1}^N \text{claim}(i) < N \sum_{i=0}^{n-1} \text{available}(i) = \max$$

Source: [BRIN73].

- 6.14 a.** Deadlock occurs if all resource units are reserved while one or more processes are waiting indefinitely for more units. But, if all 4 units are reserved, at least one process has acquired 2 units. Therefore, that process will be able to complete its work and release both units, thus enabling another process to continue.
- b.** Using terminology similar to that used for the banker's algorithm, define $\text{claim}[i]$ = total amount of resource units needed by process i ; $\text{allocation}[i]$ = current number of resource units allocated to process i ; and $\text{deficit}[i]$ = amount of resource units still needed by i . Then we have:

$$\sum_i^N \text{claim}[i] = \sum_i^N \text{deficit}[i] + \sum_i^N \text{allocation}[i] < M + N$$

In a deadlock situation, all resource units are reserved:

$$\sum_i^N \text{allocation}[i] = M$$

and some processes are waiting for more units indefinitely. But from the two preceding equations, we find

$$\sum_i^N \text{deficit}[i] < N$$

This means that at least one process j has acquired all its resources ($\text{deficit}[j] = 0$) and will be able to complete its task and release all its resources again, thus ensuring further progress in the system. So a deadlock cannot occur.

6.15 The number of available units required for the state to be safe is 3, making a total of 10 units in the system. In the state shown in the problem, if one additional unit is available, P2 can run to completion, releasing its resources, making 2 units available. This would allow P1 to run to completion making 3 units available. But at this point P3 needs 6 units and P4 needs 5 units. If to begin with, there had been 3 units available instead of 1 unit, there would now be 5 units available. This would allow P4 to run to completion, making 7 units available, which would allow P3 to run to completion.

6.16 a. In order from most-concurrent to least, there is a rough partial order on the deadlock-handling algorithms:

1. detect deadlock and kill thread, releasing its resources
detect deadlock and roll back thread's actions
restart thread and release all resources if thread needs to wait

None of these algorithms limit concurrency before deadlock occurs, because they rely on runtime checks rather than static restrictions. Their effects after deadlock is detected are harder to characterize: they still allow lots of concurrency (in some cases they enhance it), but the computation may no longer be sensible or efficient. The third algorithm is the strangest, since so much of its concurrency will be useless repetition; because threads compete for execution time, this algorithm also prevents useful computation from advancing. Hence it is listed twice in this ordering, at both extremes.

2. banker's algorithm
resource ordering

These algorithms cause more unnecessary waiting than the previous two by restricting the range of allowable computations. The banker's algorithm prevents unsafe allocations (a proper superset of deadlock-producing allocations) and resource ordering restricts allocation sequences so that threads have fewer options as to whether they must wait or not.

3. reserve all resources in advance

This algorithm allows less concurrency than the previous two, but is less pathological than the worst one. By reserving all resources in advance, threads have to wait longer and are more likely to block other threads while they work, so the system-wide execution is in effect more linear.

4. restart thread and release all resources if thread needs to wait

As noted above, this algorithm has the dubious distinction of allowing both the most and the least amount of concurrency, depending on the definition of concurrency.

b. In order from most-efficient to least, there is a rough partial order on the deadlock-handling algorithms:

1. reserve all resources in advance
resource ordering

These algorithms are most efficient because they involve no runtime overhead. Notice that this is a result of the same static restrictions that made these rank poorly in concurrency.

2. banker's algorithm
detect deadlock and kill thread, releasing its resources

These algorithms involve runtime checks on allocations which are roughly equivalent; the banker's algorithm performs a search to verify safety which is $O(nm)$ in the number of threads and allocations, and deadlock detection performs a cycle-detection search which is $O(n)$ in the length of resource-dependency chains. Resource-dependency chains are bounded by the number of threads, the number of resources, and the number of allocations.

3. detect deadlock and roll back thread's actions

This algorithm performs the same runtime check discussed previously but also entails a logging cost which is $O(n)$ in the total number of memory writes performed.

4. restart thread and release all resources if thread needs to wait

This algorithm is grossly inefficient for two reasons. First, because threads run the risk of restarting, they have a low probability of completing. Second, they are competing with other restarting threads for finite execution time, so the entire system advances towards completion slowly if at all.

This ordering does not change when deadlock is more likely. The algorithms in the first group incur no additional runtime penalty because they statically disallow deadlock-producing execution. The second group incurs a minimal, bounded penalty when deadlock occurs. The algorithm in the third tier incurs the unrolling cost, which is $O(n)$ in the number of memory writes performed between checkpoints. The status of the final algorithm is questionable because the algorithm does not allow deadlock to occur; it might be the case that unrolling becomes more expensive, but the behavior of this restart algorithm is so variable that accurate comparative analysis is nearly impossible.

6.17 The philosophers can starve while repeatedly picking up and putting down their left forks in perfect unison. Source: [BRIN73].

6.18 a. Assume that the table is in deadlock, i.e., there is a nonempty set D of philosophers such that each P_i in D holds one fork and waits for a fork held by neighbor. Without loss of generality, assume that $P_j \in D$ is a lefty. Since P_j clutches his left fork and cannot have his right fork, his right neighbor P_k never completes his dinner and is also a lefty. Therefore, $P_k \in D$. Continuing the argument rightward around the table shows that all philosophers in D are lefties. This contradicts the existence of at least one righty. Therefore deadlock is not possible.

b. Assume that lefty P_j starves, i.e., there is a stable pattern of dining in which P_j never eats. Suppose P_j holds no fork. Then P_j 's left neighbor P_i must continually hold his right fork and never finishes eating. Thus P_i is a righty holding his right fork, but never getting his left fork to complete a meal, i.e., P_i also starves. Now P_i 's left neighbor must be a righty who continually holds his right fork. Proceeding leftward around the table with this argument shows that all philosophers are (starving) righties. But P_j is a lefty: a contradiction. Thus P_j must hold one fork.

As P_j continually holds one fork and waits for his right fork, P_j 's right neighbor P_k never sets his left fork down and never completes a meal, i.e., P_k is also a lefty who starves. If P_k did not continually hold his left fork, P_j could eat; therefore P_k holds his left fork. Carrying the argument rightward around the table shows that all philosophers are (starving) lefties: a contradiction. Starvation is thus precluded.

Source: [GING90].

- 6.19 One solution (6.14) waits on available forks; the other solutions (6.17) waits for the neighboring philosophers to be free. The logic is essentially the same. The solution of Figure 6.17 is slightly more compact.
- 6.20 Atomic operations operate on atomic data types, which have their own internal format. Therefore, a simple read operation cannot be used, but a special read operation for the atomic data type is needed.
- 6.21 This code causes a deadlock, because the writer lock will spin, waiting for all readers to release the lock, including this thread.
- 6.22 Without using the memory barriers, on some processors it is possible that *c* receives the *new* value of *b*, while *d* receives the *old* value of *a*. For example, *c* could equal 4 (what we expect), yet *d* could equal 1 (not what we expect). Using the `mb ( )` insures *a* and *b* are written in the intended order, while the `rmb ( )` insures *c* and *d* are read in the intended order. This example is from [LOVE04].

Please Do Not
Post on Web

CHAPTER 7 MEMORY MANAGEMENT

ANSWERS TO QUESTIONS

- 7.1 Relocation, protection, sharing, logical organization, physical organization.
- 7.2 Typically, it is not possible for the programmer to know in advance which other programs will be resident in main memory at the time of execution of his or her program. In addition, we would like to be able to swap active processes in and out of main memory to maximize processor utilization by providing a large pool of ready processes to execute. In both these cases, the specific location of the process in main memory is unpredictable.
- 7.3 Because the location of a program in main memory is unpredictable, it is impossible to check absolute addresses at compile time to assure protection. Furthermore, most programming languages allow the dynamic calculation of addresses at run time, for example by computing an array subscript or a pointer into a data structure. Hence all memory references generated by a process must be checked at run time to ensure that they refer only to the memory space allocated to that process.
- 7.4 If a number of processes are executing the same program, it is advantageous to allow each process to access the same copy of the program rather than have its own separate copy. Also, processes that are cooperating on some task may need to share access to the same data structure.
- 7.5 By using unequal-size fixed partitions: 1. It is possible to provide one or two quite large partitions and still have a large number of partitions. The large partitions can allow the entire loading of large programs. 2. Internal fragmentation is reduced because a small program can be put into a small partition.
- 7.6 Internal fragmentation refers to the wasted space internal to a partition due to the fact that the block of data loaded is smaller than the partition. External fragmentation is a phenomenon associated with dynamic partitioning, and refers to the fact that a large number of small areas of main memory external to any partition accumulates.
- 7.7 A **logical address** is a reference to a memory location independent of the current assignment of data to memory; a translation must be made to a physical address before the memory access can be achieved. A **relative address** is a particular example of logical address, in which the address is expressed as a location relative to some known point, usually the beginning of the program. A **physical address**, or absolute address, is an actual location in main memory.

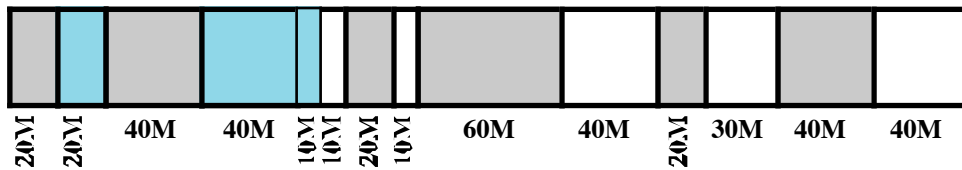
- 7.8 In a paging system, programs and data stored on disk are divided into equal, fixed-sized blocks called pages, and main memory is divided into blocks of the same size called frames. Exactly one page can fit in one frame.
- 7.9 An alternative way in which the user program can be subdivided is segmentation. In this case, the program and its associated data are divided into a number of segments. It is not required that all segments of all programs be of the same length, although there is a maximum segment length.

ANSWERS TO PROBLEMS

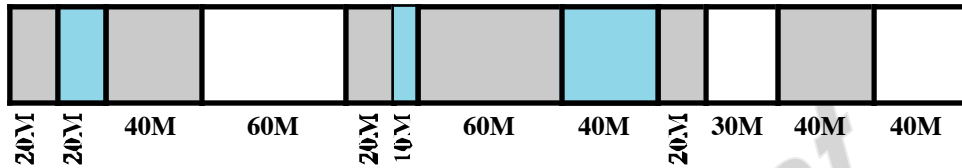
- 7.1 Here is a rough equivalence:

Relocation	≈ support modular programming
Protection	≈ process isolation; protection and access control
Sharing	≈ protection and access control
Logical Organization	≈ support of modular programming
Physical Organization	≈ long-term storage; automatic allocation and management

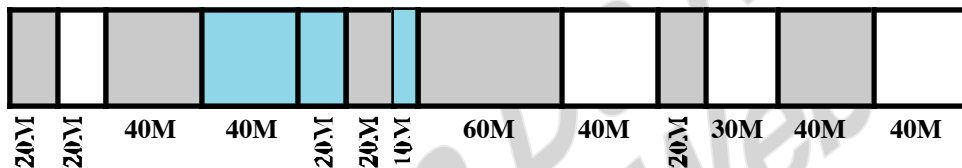
- 7.2 The number of partitions equals the number of bytes of main memory divided by the number of bytes in each partition: $2^{24}/2^{16} = 2^8$. Eight bits are needed to identify one of the 2^8 partitions.
- 7.3 Let s and h denote the average number of segments and holes, respectively. The probability that a given segment is followed by a hole in memory (and not by another segment) is 0.5, because deletions and creations are equally probable in equilibrium. so with s segments in memory, the average number of holes must be $s/2$. It is intuitively reasonable that the number of holes must be less than the number of segments because neighboring segments can be combined into a single hole on deletion.
- 7.4 By problem 7.3, we know that the average number of holes is $s/2$, where s is the number of resident segments. Regardless of fit strategy, in equilibrium, the average search length is $s/4$.
- 7.5 A criticism of the best fit algorithm is that the space remaining after allocating a block of the required size is so small that in general it is of no real use. The worst fit algorithm maximizes the chance that the free space left after a placement will be large enough to satisfy another request, thus minimizing the frequency of compaction. The disadvantage of this approach is that the largest blocks are allocated first; therefore a request for a large area is more likely to fail.
- 7.6 a. The 40 M block fits into the second hole, with a starting address of 80M. The 20M block fits into the first hole, with a starting address of 20M. The 10M block is placed at location 120M.



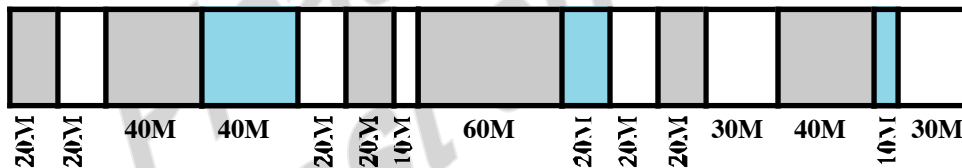
- b. The three starting addresses are 230M, 20M, and 160M, for the 40M, 20M, and 10M blocks, respectively.



- c. The three starting addresses are 80M, 120M, and 160M, for the 40M, 20M, and 10M blocks, respectively.



- d. The three starting addresses are 80M, 230M, and 360M, for the 40M, 20M, and 10M blocks, respectively.



7.7 a.

Request 70	A	128		256		512
Request 35	A	B	64	256		512
Request 80	A	B	64	C	128	512
Return A	128	B	64	C	128	512
Request 60	128	B	D	C	128	512
Return B	128	64	D	C	128	512
Return D	256			C	128	512
Return C	1024					

[illegible]

- 49-

- d. There is one entry for each page in the logical address space. Therefore there are 2^{16} entries.
- e. In addition to the valid/invalid bit, 22 bits are needed to specify the frame location in main memory, for a total of 23 bits.

7.13 The relationship is $a = pz + w$, $0 \leq w < z$, which can be stated as:

$p = \lfloor a/z \rfloor$, the integer part of a/z .

$w = R_z(a)$, the remainder obtained in dividing a by z .

7.14 a. Segment 0 starts at location 660. With the offset, we have a physical address of $660 + 198 = 858$

b. $222 + 156 = 378$

c. Segment 1 has a length of 422 bytes, so this address triggers a segment fault.

d. $996 + 444 = 1440$

e. $660 + 222 = 882$

7.15 a. Observe that a reference occurs to some segment in memory each time unit, and that one segment is deleted every t references. Because the system is in equilibrium, a new segment must be inserted every t references; therefore, the rate of the boundary's movement is s/t words per unit time. The system's operation time t_0 is then the time required for the boundary to cross the hole, i.e., $t_0 = fmr/s$, where m = size of memory. The compaction operation requires two memory references—a fetch and a store—plus overhead for each of the $(1 - f)m$ words to be moved, i.e., the compaction time t_c is at least $2(1 - f)m$. The fraction F of the time spent compacting is $F = 1 - t_0/(t_0 + t_c)$, which reduces to the expression given.

b. $k = (t/2s) - 1 = 9$; $F \geq (1 - 0.2)/(1 + 1.8) = 0.29$

CHAPTER 8 VIRTUAL MEMORY

ANSWERS TO QUESTIONS

- 8.1 **Simple paging:** all the pages of a process must be in main memory for process to run, unless overlays are used. **Virtual memory paging:** not all pages of a process need be in main memory frames for the process to run.; pages may be read in as needed
- 8.2 Thrashing is a phenomenon in virtual memory schemes, in which the processor spends most of its time swapping pieces rather than executing instructions.
- 8.3 Algorithms can be designed to exploit the principle of locality to avoid thrashing. In general, the principle of locality allows the algorithm to predict which resident pages are least likely to be referenced in the near future and are therefore good candidates for being swapped out.
- 8.4 **Frame number:** the sequential number that identifies a page in main memory; **present bit:** indicates whether this page is currently in main memory; **modify bit:** indicates whether this page has been modified since being brought into main memory.
- 8.5 The TLB is a cache that contains those page table entries that have been most recently used. Its purpose is to avoid, most of the time, having to go to disk to retrieve a page table entry.
- 8.6 With **demand paging**, a page is brought into main memory only when a reference is made to a location on that page. With **prepaging**, pages other than the one demanded by a page fault are brought in.
- 8.7 **Resident set management** deals with the following two issues: (1) how many page frames are to be allocated to each active process; and (2) whether the set of pages to be considered for replacement should be limited to those of the process that caused the page fault or encompass all the page frames in main memory. **Page replacement policy** deals with the following issue: among the set of pages considered, which particular page should be selected for replacement.
- 8.8 The clock policy is similar to FIFO, except that in the clock policy, any frame with a use bit of 1 is passed over by the algorithm.
- 8.9 (1) If a page is taken out of a resident set but is soon needed, it is still in main memory, saving a disk read. (2) Modified page can be written out in clusters rather

than one at a time, significantly reducing the number of I/O operations and therefore the amount of disk access time.

- 8.10 Because a fixed allocation policy requires that the number of frames allocated to a process is fixed, when it comes time to bring in a new page for a process, one of the resident pages for that process must be swapped out (to maintain the number of frames allocated at the same amount), which is a local replacement policy.
- 8.11 The resident set of a process is the current number of pages of that process in main memory. The working set of a process is the number of pages of that process that have been referenced recently.
- 8.12 With **demand cleaning**, a page is written out to secondary memory only when it has been selected for replacement. A **precleaning** policy writes modified pages before their page frames are needed so that pages can be written out in batches.

ANSWERS TO PROBLEMS

- 8.1 a. Split binary address into virtual page number and offset; use VPN as index into page table; extract page frame number; concatenate offset to get physical memory address
- b. (i) $1052 = 1024 + 28$ maps to VPN 1 in PFN 7, $(7 \times 1024 + 28 = 7196)$
(ii) $2221 = 2 \times 1024 + 173$ maps to VPN 2, page fault
(iii) $5499 = 5 \times 1024 + 379$ maps to VPN 5 in PFN 0, $(0 \times 1024 + 379 = 379)$
- 8.2 a. Virtual memory can hold $(2^{32} \text{ bytes of main memory}) / (2^{10} \text{ bytes/page}) = 2^{22}$ pages, so 22 bits are needed to specify a page in virtual memory. Each page table contains $(2^{10} \text{ bytes per page table}) / (4 \text{ bytes/entry}) = 2^8$ entries. Thus, each page table can handle 8 of the required 22 bits. Therefore, 3 levels of page tables are needed.
- b. Tables at two of the levels have 2^8 entries; tables at one level have 2^6 entries. $(8 + 8 + 6 = 22)$.
- c. Less space is consumed if the top level has 2^6 entries. In that case, the second level has 2^6 pages with 2^8 entries each, and the bottom level has 2^{14} pages with 2^8 entries each, for a total of $1 + 2^6 + 2^{14}$ pages = 16,449 pages. If the middle level has 26 entries, then the number of pages is $1 + 2^8 + 2^{14}$ pages = 16,641 pages. If the bottom level has 2^6 entries, then the number of tables is $1 + 2^8 + 2^{16}$ pages = 65,973 pages.
- 8.3 a. 4 Mbyte
- b. Number of rows: $26+2=128$ entries. Each entry consist of: 20 (page number) + 20 (frame number) + 8 bits (chain index) = 48 bits = 6 bytes.
Total: $128 \times 6 = 768$ bytes
- 8.4 a. PFN 3 since loaded longest ago at time 20
b. PFN 1 since referenced longest ago at time 160
c. Clear R in PFN 3 (oldest loaded), clear R in PFN 2 (next oldest loaded), victim PFN is 0 since R=0
d. Replace the page in PFN 3 since VPN 3 (in PFN 3) is used furthest in the future

e. There are 6 faults, indicated by *

		*				*	*		*	*	*	
		4	0	0	0	2	4	2	1	0	3	2
VPN of	3	4	0	0	0	2	4	2	1	0	3	2
pages in	0	3	4	4	4	0	2	4	2	1	0	
memory in	2	0	3	3			0	0	4	2	1	
LRU order	1	2								4	2	

8.5 9 and 10 page transfers, respectively. This is referred to as "Belady's anomaly," and was reported in "An Anomaly in Space-Time Characteristics of Certain Programs Running in a Paging Machine," by Belady et al, *Communications of the ACM*, June 1969.

8.6 a. LRU: Hit ratio = 16/33

1 0 2 2 1 7 6 7 0 1 2 0 3 0 4 5 1 5 2 4 5 6 7 6 7 2 4 2 7 3 3 2 3

1 1 1 1 1 1 1 1 1 1 1 1 1 1 4 4 4 4 4 4 4 4 4 4 2 2 2 2 2 2 2 2

- 0 0 0 0 0 6 6 6 6 2 2 2 2 2 5 5 5 5 5 5 5 5 5 5 4 4 4 4 4 4 4 4

- - 2 2 2 2 2 2 0 0 0 0 0 0 0 0 2 2 2 2 7 7 7 7 7 7 7 7 7 7 7 7

- - - - - 7 7 7 7 7 7 3 3 3 3 1 1 1 1 6 6 6 6 6 6 6 6 3 3 3 3

F F

b. FIFO: Hit ratio = 16/33

1 0 2 2 1 7 6 7 0 1 2 0 3 0 4 5 1 5 2 4 5 6 7 6 7 2 4 2 7 3 3 2 3

1 1 1 1 1 1 6 6 6 6 6 6 6 6 4 4 4 4 4 4 4 4 6 6 6 6 6 6 6 6 2 2

- 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 5 5 5 5 5 5 5 5 7 7 7 7 7 7 7 7

- - 2 2 2 2 2 2 2 2 2 2 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 4 4 4 4 4 4

- - - - - 7 7 7 7 7 7 3 3 3 3 3 3 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3

F F

c. These two policies are equally effective for this particular page trace.

8.7 The principal **advantage** is a savings in physical memory space. This occurs for two reasons: (1) a user page table can be paged in to memory only when it is needed. (2) The operating system can allocate user page tables dynamically, creating one only when the process is created.

Of course, there is a **disadvantage**: address translation requires extra work.

- 8.8 The machine language version of this program, loaded in main memory starting at address 4000, might appear as:

4000	(R1) $\leftarrow$ ONE	Establish index register for i
4001	(R1) $\leftarrow$ n	Establish n in R2
4002	compare R1, R2	Test i > n
4003	branch greater 4009	
4004	(R3) $\leftarrow$ B(R1)	Access B[i] using index register R1
4005	(R3) $\leftarrow$ (R3) + C(R1)	Add C[i] using index register R1
4006	A(R1) $\leftarrow$ (R3)	Store sum in A[i] using index register R1
4007	(R1) $\leftarrow$ (R1) + ONE	Increment i
4008	branch 4002	
6000-6999	storage for A	
7000-7999	storage for B	
8000-8999	storage for C	
9000	storage for ONE	
9001	storage for n	

The reference string generated by this loop is

$$494944(47484649444)^{1000}$$

consisting of over 11,000 references, but involving only five distinct pages. Source: [MAEK87].

- 8.9 The S/370 segments are fixed in size and not visible to the programmer. Thus, none of the benefits listed for segmentation are realized on the S/370, with the exception of protection. The P bit in each segment table entry provides protection for the entire segment.
- 8.10 Since each page table entry is 4 bytes and each page contains 4 Kbytes, then a one-page page table would point to $1024 = 2^{10}$ pages, addressing a total of $2^{10} \times 2^{12} = 2^{22}$ bytes. The address space however is 2^{64} bytes. Adding a second layer of page tables, the top page table would point to 2^{10} page tables, addressing a total of 2^{32} bytes. Continuing this process,

Depth	Address Space
1	2^{22} bytes
2	2^{32} bytes
3	2^{42} bytes
4	2^{52} bytes
5	2^{62} bytes
6	2^{72} bytes ($> 2^{64}$ bytes)

we can see that 5 levels do not address the full 64 bit address space, so a 6th level is required. But only 2 bits of the 6th level are required, not the entire 10 bits. So instead of requiring your virtual addresses be 72 bits long, you could mask out and ignore all but the 2 lowest order bits of the 6th level. This would give you a 64 bit

address. Your top level page table then would have only 4 entries. Yet another option is to revise the criteria that the top level page table fit into a single physical page and instead make it fit into 4 pages. This would save a physical page, which is not much.

- 8.11 a. 400 nanoseconds. 200 to get the page table entry, and 200 to access the memory location.
b. This is a familiar effective time calculation:

$$(220 \times 0.85) + (420 \times 0.15) = 250$$

- Two cases: First, when the TLB contains the entry required. In that case we pay the 20 ns overhead on top of the 200 ns memory access time. Second, when the TLB does not contain the item. Then we pay an additional 200 ns to get the required entry into the TLB.
- c. The higher the TLB hit rate is, the smaller the EMAT is, because the additional 200 ns penalty to get the entry into the TLB contributes less to the EMAT.

- 8.12 a. N
b. P

- 8.13 a. This is a good analogy to the CLOCK algorithm. Snow falling on the track is analogous to page hits on the circular clock buffer. The movement of the CLOCK pointer is analogous to the movement of the plow.
b. Note that the density of replaceable pages is highest immediately in front of the clock pointer, just as the density of snow is highest immediately in front of the plow. Thus, we can expect the CLOCK algorithm to be quite efficient in finding pages to replace. In fact, it can be shown that the depth of the snow in front of the plow is twice the average depth on the track as a whole. By this analogy, the number of pages replaced by the CLOCK policy on a single circuit should be twice the number that are replaceable at a random time. The analogy is imperfect because the CLOCK pointer does not move at a constant rate, but the intuitive idea remains.

The snowplow analogy to the CLOCK algorithm comes from [CARR84]; the depth analysis comes from Knuth, D. *The Art of Computer Programming, Volume 2: Sorting and Searching*. Reading, MA: Addison-Wesley, 1997 (page 256).

- 8.14 The processor hardware sets the reference bit to 0 when a new page is loaded into the frame, and to 1 when a location within the frame is referenced. The operating system can maintain a number of queues of page-frame tables. A page-frame table entry moves from one queue to another according to how long the reference bit from that page frame stays set to zero. When pages must be replaced, the pages to be replaced are chosen from the queue of the longest-life nonreferenced frames.

8.15 a.

Seq of page refs	Window Size, Δ					
	1	2	3	4	5	6
1	1	1	1	1	1	1
2	2	1 2	1 2	1 2	1 2	1 2
3	3	2 3	1 2 3	1 2 3	1 2 3	1 2 3
4	4	3 4	2 3 4	1 2 3 4	1 2 3 4	1 2 3 4
5	5	4 5	3 4 5	2 3 4 5	1 2 3 4 5	1 2 3 4 5
2	2	5 2	4 5 2	3 4 5 2	3 4 5 2	1 3 4 5 2
1	1	2 1	5 2 1	4 5 2 1	3 4 5 2 1	3 4 5 2 1
3	3	1 3	2 1 3	5 2 1 3	4 5 2 1 3	4 5 2 1 3
3	3	3	1 3	2 1 3	5 2 1 3	4 5 2 1 3
2	2	3 2	3 2	1 3 2	1 3 2	5 1 3 2
3	3	2 3	2 3	2 3	1 2 3	1 2 3
4	4	3 4	2 3 4	2 3 4	2 3 4	1 2 3 4
5	5	4 5	3 4 5	2 3 4 5	2 3 4 5	2 3 4 5
4	4	5 4	5 4	3 5 4	2 3 5 4	2 3 5 4
5	5	4 5	4 5	4 5	3 4 5	2 3 4 5
1	1	5 1	4 5 1	4 5 1	4 5 1	3 4 5 1
1	1	1	5 1	4 5 1	4 5 1	4 5 1
3	3	1 3	1 3	5 1 3	4 5 1 3	4 5 1 3
2	2	3 2	1 3 2	1 3 2	5 1 3 2	4 5 1 3 2
5	5	2 5	3 2 5	1 3 2 5	1 3 2 5	1 3 2 5

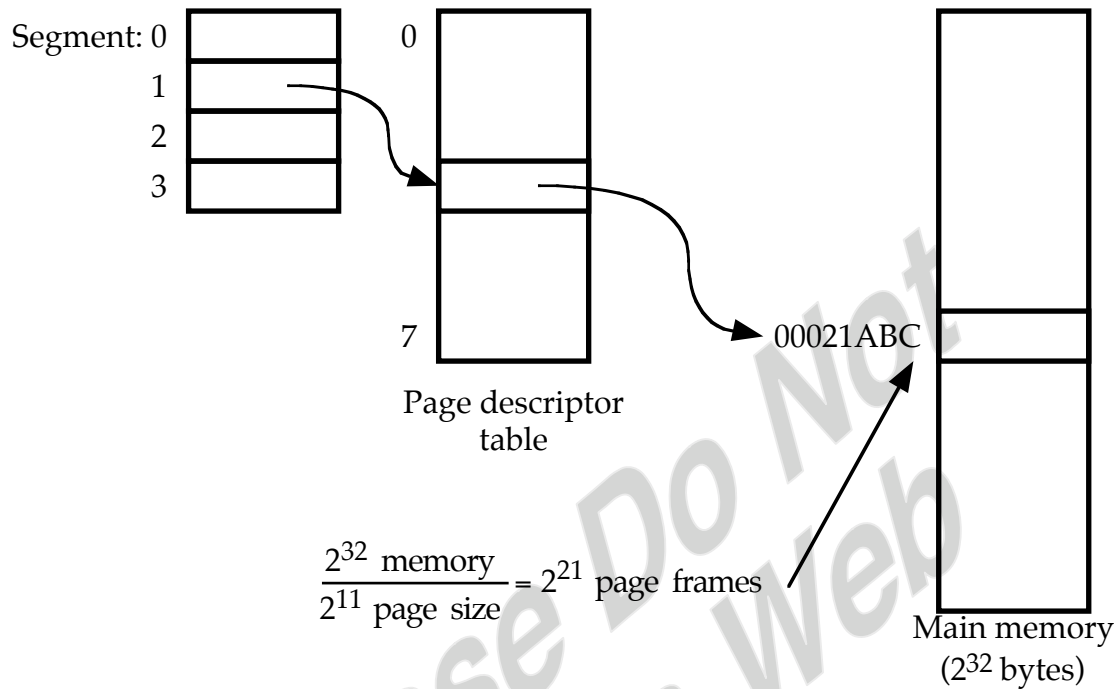
b., c.

Δ	1	2	3	4	5	6
$s_{20}(\Delta)$	1	1.85	2.5	3.1	3.55	3.9
$m_{20}(\Delta)$	0.9	0.75	0.75	0.65	0.55	0.5

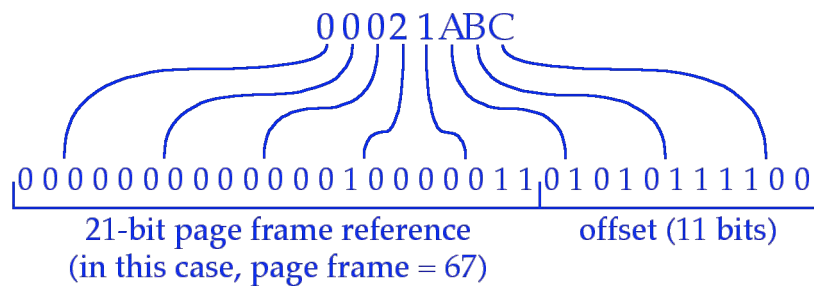
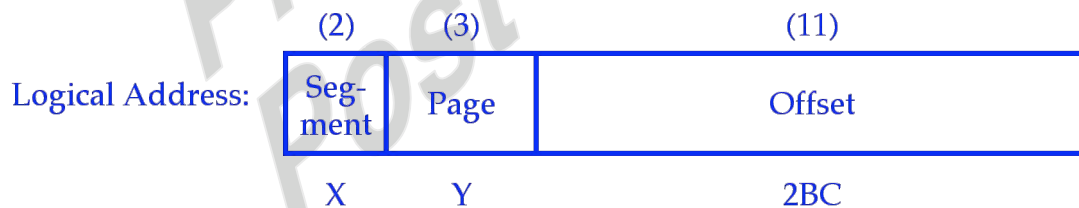
$s_{20}(\Delta)$ is an increasing function of Δ . $m_{20}(\Delta)$ is a nonincreasing function of Δ .

8.16 [PIZZ89] suggests the following strategy. Use a mechanism that adjusts the value of Q at each window time as a function of the actual page fault rate experienced during the window. The page fault rate is computed and compared with a system-wide value for "desirable" page fault rate for a job. The value of Q is adjusted upward (downward) whenever the actual page fault rate of a job is higher (lower) than the desirable value. Experimentation using this adjustment mechanism showed that execution of the test jobs with dynamic adjustment of Q consistently produced a lower number of page faults per execution and a decreased average resident set size than the execution with a constant value of Q (within a very broad range). The memory time product (MT) versus Q using the adjustment mechanism also produced a consistent and considerable improvement over the previous test results using a constant value of Q .

$$8.17 \quad \frac{2^{32} \text{ memory}}{2^{11} \text{ page size}} = 2^{21} \text{ page frames}$$



- a. $8 \times 2K = 16K$
- b. $16K \times 4 = 64K$
- c. $2^{32} = 4 \text{ GBytes}$



8.18 a.

page number (5)	offset (11)
-----------------	-------------

- b. 32 entries, each entry is 9 bits wide.
- c. If total number of entries stays at 32 and the page size does not change, then each entry becomes 8 bits wide.

8.19 It is possible to shrink a process's stack by deallocating the unused pages. By convention, the contents of memory beyond the current top of the stack are undefined. On almost all architectures, the current top of stack pointer is kept in a well-defined register. Therefore, the kernel can read its contents and deallocate any unused pages as needed. The reason that this is not done is that little is gained by the effort. If the user program will repeatedly call subroutines that need additional space for local variables (a very likely case), then much time will be wasted deallocating stack space in between calls and then reallocating it later on. If the subroutine called is only used once during the life of the program and no other subroutine will ever be called that needs the stack space, then eventually the kernel will page out the unused portion of the space if it needs the memory for other purposes. In either case, the extra logic needed to recognize the case where a stack could be shrunk is unwarranted. Source: [SCHI94].

Please Do Not
Post on Web

CHAPTER 9 UNIPROCESSOR SCHEDULING

ANSWERS TO QUESTIONS

- 9.1 **Long-term scheduling:** The decision to add to the pool of processes to be executed. **Medium-term scheduling:** The decision to add to the number of processes that are partially or fully in main memory. **Short-term scheduling:** The decision as to which available process will be executed by the processor
- 9.2 Response time.
- 9.3 **Turnaround time** is the total time that a request spends in the system (waiting time plus service time). **Response time** is the elapsed time between the submission of a request until the response begins to appear as output.
- 9.4 In UNIX and many other systems, larger priority values represent lower priority processes. Some systems, such as Windows, use the opposite convention: a higher number means a higher priority
- 9.5 **Nonpreemptive:** If a process is in the Running state, it continues to execute until (a) it terminates or (b) blocks itself to wait for I/O or to request some operating system service. **Preemptive:** The currently running process may be interrupted and moved to the Ready state by the operating system. The decision to preempt may be performed when a new process arrives, when an interrupt occurs that places a blocked process in the Ready state, or periodically based on a clock interrupt.
- 9.6 As each process becomes ready, it joins the ready queue. When the currently-running process ceases to execute, the process that has been in the ready queue the longest is selected for running.
- 9.7 A clock interrupt is generated at periodic intervals. When the interrupt occurs, the currently running process is placed in the ready queue, and the next ready job is selected on a FCFS basis.
- 9.8 This is a nonpreemptive policy in which the process with the shortest expected processing time is selected next.
- 9.9 This is a preemptive version of SPN. In this case, the scheduler always chooses the process that has the shortest expected remaining processing time. When a new process joins the ready queue, it may in fact have a shorter remaining time than the currently running process. Accordingly, the scheduler may preempt whenever a new process becomes ready.

- 9.10 When the current process completes or is blocked, choose the ready process with the greatest value of R , where $R = (w + s)/s$, with w = time spent waiting for the processor and s = expected service time.
- 9.11 Scheduling is done on a preemptive (at time quantum) basis, and a dynamic priority mechanism is used. When a process first enters the system, it is placed in RQ0 (see Figure 9.4). After its first execution, when it returns to the Ready state, it is placed in RQ1. Each subsequent time that it is preempted, it is demoted to the next lower-priority queue. A shorter process will complete quickly, without migrating very far down the hierarchy of ready queues. A longer process will gradually drift downward. Thus, newer, shorter processes are favored over older, longer processes. Within each queue, except the lowest-priority queue, a simple FCFS mechanism is used. Once in the lowest-priority queue, a process cannot go lower, but is returned to this queue repeatedly until it completes execution.

ANSWERS TO PROBLEMS

- 9.1 Each square represents one time unit; the number in the square refers to the currently-running process.

FCFS	A	A	A	B	B	B	B	B	C	C	D	D	D	D	D	E	E	E	E	E
RR, $q = 1$	A	B	A	B	C	A	B	C	B	D	B	D	E	D	E	D	E	D	E	E
RR, $q = 4$	A	A	A	B	B	B	B	C	C	B	D	D	D	D	E	E	E	E	D	E
SPN	A	A	A	C	C	B	B	B	B	B	D	D	D	D	D	E	E	E	E	E
SRT	A	A	A	C	C	B	B	B	B	B	D	D	D	D	D	E	E	E	E	E
HRRN	A	A	A	B	B	B	B	B	C	C	D	D	D	D	D	E	E	E	E	E
Feedback, $q = 1$	A	B	A	C	B	C	A	B	B	D	B	D	E	D	E	D	E	D	E	E
Feedback, $q = 2^i$	A	B	A	A	C	B	B	C	B	B	D	D	E	D	D	E	E	D	E	E

		A	B	C	D	E	
	T_a	0	1	3	9	12	
	T_s	3	5	2	5	5	
FCFS	T_f	3	8	10	15	20	
	T_r	3.00	7.00	7.00	6.00	8.00	6.20
	T_r/T_s	1.00	1.40	3.50	1.20	1.60	1.74
RR $q = 1$	T_f	6.00	11.00	8.00	18.00	20.00	
	T_r	6.00	10.00	5.00	9.00	8.00	7.60
	T_r/T_s	2.00	2.00	2.50	1.80	1.60	1.98
RR $q = 4$	T_f	3.00	10.00	9.00	19.00	20.00	
	T_r	3.00	9.00	6.00	10.00	8.00	7.20
	T_r/T_s	1.00	1.80	3.00	2.00	1.60	1.88
SPN	T_f	3.00	10.00	5.00	15.00	20.00	
	T_r	3.00	9.00	2.00	6.00	8.00	5.60
	T_r/T_s	1.00	1.80	1.00	1.20	1.60	1.32
SRT	T_f	3.00	10.00	5.00	15.00	20.00	
	T_r	3.00	9.00	2.00	6.00	8.00	5.60
	T_r/T_s	1.00	1.80	1.00	1.20	1.60	1.32
HRRN	T_f	3.00	8.00	10.00	15.00	20.00	
	T_r	3.00	7.00	7.00	6.00	8.00	6.20
	T_r/T_s	1.00	1.40	3.50	1.20	1.60	1.74
FB $q = 1$	T_f	7.00	11.00	6.00	18.00	20.00	
	T_r	7.00	10.00	3.00	9.00	8.00	7.40
	T_r/T_s	2.33	2.00	1.50	1.80	1.60	1.85
FB $q = 2^i$	T_f	4.00	10.00	8.00	18.00	20.00	
	T_r	4.00	9.00	5.00	9.00	8.00	7.00
	T_r/T_s	1.33	1.80	2.50	1.80	1.60	1.81

9.2

FCFS

RR, $q = 1$

RR, $q = 4$

SPN

SRT

HRRN

Feedback, $q = 1$

Feedback, $q = 2^i$

A	B	B	B	B	B	B	B	B	B	C	D	D	D	D	D	D	D	D	D
A	B	C	B	D	B	D	B	D	B	D	B	D	B	D	B	D	B	D	D
A	B	B	B	B	C	B	B	B	B	D	D	D	D	B	D	D	D	D	D
A	B	B	B	B	B	B	B	B	B	C	D	D	D	D	D	D	D	D	D
A	B	C	B	B	B	B	B	B	B	B	D	D	D	D	D	D	D	D	D
A	B	B	B	B	B	B	B	B	B	C	D	D	D	D	D	D	D	D	D
A	B	C	D	B	D	B	D	B	D	B	D	B	D	B	D	B	D	B	D
A	B	C	D	B	B	D	D	B	B	B	B	D	D	D	D	B	B	D	D

Please Do Not
Post on Web

		A	B	C	D	
	T_a	0	1	2	3	
	T_s	1	9	1	9	
FCFS	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22
RR $q = 1$	T_f	1.00	18.00	3.00	20.00	
	T_r	1.00	17.00	1.00	17.00	9.00
	T_r/T_s	1.00	1.89	1.00	1.89	1.44
RR $q = 4$	T_f	1.00	15.00	6.00	20.00	
	T_r	1.00	14.00	4.00	17.00	9.00
	T_r/T_s	1.00	1.56	4.00	1.89	2.11
SPN	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22
SRT	T_f	1.00	11.00	3.00	20.00	
	T_r	1.00	10.00	1.00	17.00	7.25
	T_r/T_s	1.00	1.11	1.00	1.89	1.25
HRRN	T_f	1.00	10.00	11.00	20.00	
	T_r	1.00	9.00	9.00	17.00	9.00
	T_r/T_s	1.00	1.00	9.00	1.89	3.22
FB $q = 1$	T_f	1.00	19.00	3.00	20.00	
	T_r	1.00	18.00	1.00	17.00	9.25
	T_r/T_s	1.00	2.00	1.00	1.89	1.47
FB $q = 2^i$	T_f	1.00	18.00	3.00	20.00	
	T_r	1.00	17.00	1.00	17.00	9.00
	T_r/T_s	1.00	1.89	1.00	1.89	1.44

9.3 We will prove the assertion for the case in which a batch of n jobs arrive at the same time, and ignoring further arrivals. The proof can be extended to cover later arrivals.

Let the service times of the jobs be

$$t_1 \leq t_2 \leq \dots \leq t_n$$

Then, n users must wait for the execution of job 1; $n - 1$ users must wait for the execution of job 2, and so on. Therefore, the average response time is

$$\frac{n \times t_1 + (n-1) \times t_2 + \dots + t_n}{n}$$

If we make any changes in this schedule, for example by exchanging jobs j and k (where $j < k$), the average response time is increased by the amount

$$\frac{(k-j) \times (t_k - t_j)}{n} \geq 0$$

In other words, the average response time can only increase if the SPN algorithm is not used. Source: [BRIN73].

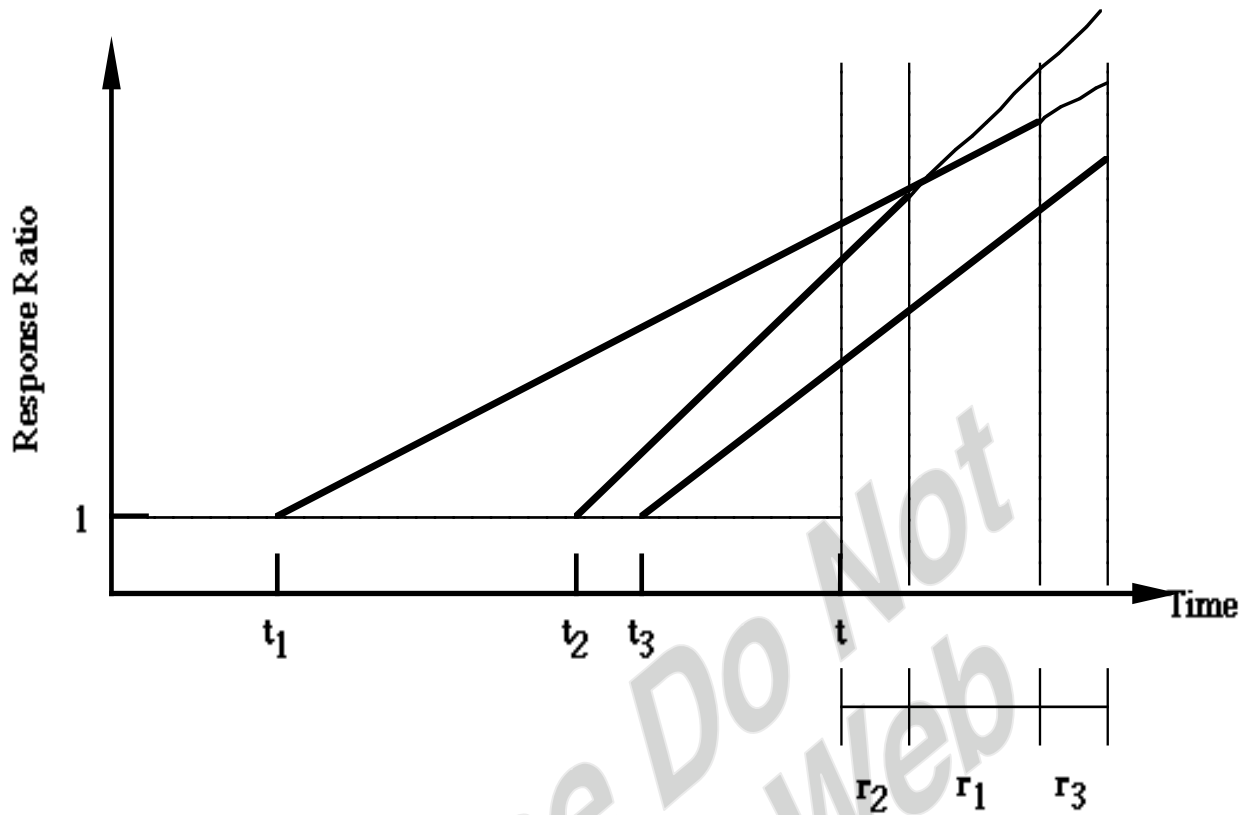
9.4 The data points for the plot:

Age of Observation	Observed Value	Simple Average	alpha = 0.8	alpha = 0.5
1	6	0.00	0.00	0.00
2	4	3.00	4.80	3.00
3	6	3.33	4.16	3.50
4	4	4.00	5.63	4.75
5	13	4.00	4.33	4.38
6	13	5.50	11.27	8.69
7	13	6.57	12.65	10.84

- 9.5 The first equation is identical to Equation 9.3, so the parameter α provides an exponential smoothing effect. The parameter β is a delay variance factor (e.g., 1.3 to 2.0). A smaller value of β will result in faster adaptation to changes in the observed times, but also more fluctuation in the estimates.

A sophisticated analysis of this type of estimation procedure is contained in *Applied Optimal Estimation*, edited by Gelb, M.I.T. Press, 1974.

- 9.6 It depends on whether you put job A in a queue after the first time unit or not. If you do, then it is entitled to 2 additional time units before it can be preempted.
- 9.7 First, the scheduler computes the response ratios at time $t + r_1 + r_2 + r_3$, when all three jobs will have been finished (see figure). At that time, job 3 will have the smallest response ratio of the three: so the scheduler decides to execute this job last and proceeds to examine jobs 1 and 2 at time $t + r_1 + r_2$, when they will both be finished. Here the response ratio of job 1 is the smaller, and consequently job 2 is selected for service at time t . This algorithm is repeated each time a job is completed to take new arrivals into account. Note that this algorithm is not quite the same as *highest response ratio next*. The latter would schedule job 1 at time t . Intuitively, it is clear that the present algorithm attempts to minimize the maximum response ratio by consistently postponing jobs that will suffer the least increase of their response ratios.



- 9.8 This proof, due to P. Mondrup, is reported in [BRIN73]. Consider the queue at time t immediately after a departure and ignore further arrivals. The waiting jobs are numbered 1 to n in the order in which they will be scheduled:

job:	1	2	...	i	...	n
arrival time:	t_1	t_2	...	t_i	...	t_n
service time:	r_1	r_2	...	r_i	...	r_n

Among these we assume that job i will reach the highest response ratio before its departure. When the jobs 1 to i have been executed, time becomes

$$T_i = t + r_1 + r_2 + \dots + r_i$$

and job i has the response ratio

$$R_i(T_i) + \frac{T_i - t_i}{r_i}$$

The reason for executing job i last in the sequence 1 to i is that its response ratio will be the lowest one among these jobs at time T_i :

$$R_i(T_i) = \min [R_1(T_i), R_2(T_i), \dots, R_i(T_i)]$$

Consider now the consequences of scheduling the same n jobs in any other sequence:

job:	a	b	. . .	j	. . .	z
arrival time:	t_a	t_b	. . .	t_j	. . .	t_z
service time:	r_a	r_b	. . .	r_j	. . .	r_z

In the new sequence, we select the smallest subsequence of jobs, a to j, that contains all the jobs, 1 to i, of the original subsequence (This implies that job j is itself one of the jobs 1 to i). When the jobs a to j have been served, time becomes

$$T_j = t + r_a + r_b + \dots + r_j$$

and job j reaches the response ratio

$$R_j(T_j) + \frac{T_j - t_j}{r_j}$$

Since the jobs 1 to i are a subset of the jobs a to j, the sum of their service times $T_i - t$ must be less than or equal to the sum of service time $T_j - t$. And since response ratios increase with time, $T_i \leq T_j$ implies

$$R_j(T_j) \geq R_j(T_i)$$

It is also known that job j is one of the jobs 1 to i, of which job j has the smallest response ratio at time T_i . The above inequality can therefore be extended as follows:

$$R_j(T_j) \geq R_j(T_i) \geq R_i(T_i)$$

In other words, when the scheduling algorithm is changed, there will always be a job j that reaches response ratio $R_j(T_j)$, which is greater than or equal to the highest response ratio $R_i(T_i)$ obtained with the original algorithm.

Notice that this proof is valid in general for priorities that are non-decreasing functions of time. For example, in a FIFO system, priorities increase linearly with waiting time at the same rate for all jobs. Therefore, the present proof shows that the FIFO algorithm minimizes the maximum waiting time for a given batch of jobs.

- 9.9** Before we begin, there is one result that is needed, as follows. Assume that an item with service time T_s has been in service for a time h. Then, the expected remaining service time $E[T/T > h] = T_s$. That is, no matter how long an item has been in service, the expected remaining service time is just the average service time for the item. This result, though counter to intuition, is correct, as we now show.

Consider the exponential probability distribution function:

$$F(x) = \Pr[X \leq x] = 1 - e^{-\mu x}$$

Then, we have $\Pr[X > x] = e^{-\mu x}$. Now let us look at the conditional probability that X is greater than $x + h$ given that X is greater than x:

$$\Pr[X > x + h | X > x] = \frac{\Pr[(X > x + h), (X > x)]}{\Pr[X > x]} = \frac{\Pr[X > x + h]}{\Pr[X > x]}$$

$$\Pr[X > x + h | X > x] = \frac{e^{-\mu(x+h)}}{e^{-\mu x}} = e^{-\mu h}$$

So,

$$\Pr[X \leq x + h | X > x] = 1 - e^{-\mu h} = \Pr[X \leq h]$$

Thus the probability distribution for service time given that there has been service of duration x is the same as the probability distribution of total service time. Therefore the expected value of the remaining service time is the same as the original expected value of service time.

With this result, we can now proceed to the original problem. When an item arrives for service, the total response time for that item will consist of its own service time plus the service time of all items ahead of it in the queue. The total expected response time has three components.

- Expected service time of arriving process = T_s
- Expected service time of all processes currently waiting to be served. This value is simply $w \times T_s$, where w is the mean number of items waiting to be served.
- Remaining service time for the item currently in service, if there is an item currently in service. This value can be expressed as $\rho \times T_s$, where ρ is the utilization and therefore the probability that an item is currently in service and T_s , as we have demonstrated, is the expected remaining service time.

Thus, we have

$$R = T_s \times (1 + w + \rho) = T_s \times \left(1 + \frac{\rho^2}{1 - \rho} + \rho\right) = \frac{T_s}{1 - \rho}$$

9.10 Let us denote the time slice, or quantum, used in round robin scheduling as δ . In this problem, δ is assumed to be very small compared to the service time of a process. Now, consider a newly arrived process, which is placed at the end of the ready queue for service. We are assuming that this particular process has a service time of x , which is some multiple of δ :

$$x = m\delta$$

To begin, let us ask the question, how much time does the process spend in the queue before it receives its first quantum of service. It must wait until all q processes waiting in line ahead of it have been serviced. Thus the initial wait time = $q\delta$, where q is the average number of items in the system (waiting and being served). We can now calculate the total time this process will spend waiting before

it has received x seconds of service. Since it must pass through the active queue m times, and each time it waits $q\delta$ seconds, the total wait time is as follows:

$$\begin{aligned}\text{Wait time} &= m(q\delta) \\ &= \frac{x}{\delta}(q\delta) \\ &= qx\end{aligned}$$

Then, the response time is the sum of the wait time and the total service time

$$\begin{aligned}R_x &= \text{wait time} + \text{service time} \\ &= qx + x = (q + 1)x\end{aligned}$$

Referring to the queuing formulas in Appendix A, the mean number of items in the system, q , can be expressed as

$$q = \frac{\rho}{1 - \rho}$$

Thus,

$$R_x = \left(\frac{\rho}{1 - \rho} + 1 \right) \times x = \frac{x}{1 - \rho} \quad R_x = \left(\frac{\rho}{1 - \rho} + 1 \right) \times x = \frac{x}{1 - \rho}$$

9.11 An argument in favor of a small quantum: Using a small quantum will enhance responsiveness by frequently running all processes for a short time slice. When the ready queue has many processes that are interactive, responsiveness is very important e.g. general-purpose computer system.

An argument in favor of a large quantum: Using a large quantum will enhance the throughput, and the CPU utilization measured with respect to real work, because there is less context switching and therefore less overhead. e.g. batch jobs.

A system for which both might be appropriate: There are some systems for which both small and large quanta are reasonable. e.g. a long job that requires just a few user interactions. Although this type of job can be considered as a batch job, in some sense, it still has to interact with the user. Therefore, during the times when there is no user interaction, the quantum might be increased to optimize the throughput and during interactive time, the quantum might be lowered to provide better responsiveness.

9.12 First, we need to clarify the significance of the parameter λ' . The rate at which items arrive at the first box (the "queue" box) is λ . Two adjacent arrivals to the second box (the "service" box) will arrive at a slightly slower rate, since the second item is delayed in its chase of the first item. We may calculate the vertical offset y in the figure in two different ways, based on the geometry of the diagram:

$$y = \frac{\beta}{\lambda'} \quad y = \frac{\beta}{\lambda'}$$

$$y = \left(\frac{1}{\lambda'} - \frac{1}{\lambda} \right) \alpha$$

which therefore gives

$$\lambda' = \lambda \left(1 - \frac{\beta}{\alpha}\right)$$

The total number of jobs q waiting or in service when the given job arrives is given by:

$$q = \frac{\rho}{1 - \rho}$$

independent of the scheduling algorithm. Using Little's formula (see Appendix A):

$$R = \frac{q}{\lambda} = \frac{s}{1 - \rho}$$

Now let W and V_x denote the mean times spent in the queue box and in the service box by a job of service time x . Since priorities are initially based only on elapsed waiting times, W is clearly independent of the service time x . Evidently we have

$$R_x = W + V_x$$

From problem 9.10, we have

$$V_x = \frac{t}{1 - \rho'}, \quad \text{where } \rho' = \lambda's$$

By taking the expected values of R_x and S_x , we have $R = W + V$. We have already developed the formula for R . For V , observe that the arrival rate to the service box is λ' , and therefore the utilization is ρ' . Accordingly, from our basic M/M/1 formulas, we have

$$V = \frac{s}{1 - \rho'}$$

Thus,

$$W = R - V = s \left(\frac{1}{1 - \rho} - \frac{1}{1 - \rho'} \right)$$

which yields the desired result for R_x .

- 9.13** Only as long as there are comparatively few users in the system. When the quantum is decreased to satisfy more users rapidly two things happen: (1) processor utilization decreases, and (2) at a certain point, the quantum becomes too small to satisfy most trivial requests. Users will then experience a sudden increase of response times because their requests must pass through the round-robin queue several times. Source: [BRIN73].
- 9.14** If a process uses too much processor time, it will be moved to a lower-priority queue. This leaves I/O-bound processes in the higher-priority queues.
- 9.15** Dekker's algorithm relies on the fairness of the hardware and the OS. The use of priorities risks starvation as follows. It may happen if P0 is a very fast repetitive process which, as it constantly finds flag [1] = false, keeps entering its critical section, while P1, leaving the internal loop in which it was waiting for its turn,

cannot set flag [1] to true, being prevented from doing so by P0's reading of the variable (remember that access to the variable takes place under mutual exclusion).

9.16 a. Sequence with which processes will get 1 min of processor time:

1	2	3	4	5	Elapsed time
A	B	C	D	E	5
A	B	C	D	E	10
A	B	C	D	E	15
A	B		D	E	19
A	B		D	E	23
A	B		D	E	27
A	B			E	30
A	B			E	33
A	B			E	36
A				E	38
A				E	40
A				E	42
A					43
A					44
A					45

The turnaround time for each process:

A = 45 min, B = 35 min, C = 13 min, D = 26 min, E = 42 min

The average turnaround time is $= (45+35+13+26+42) / 5 = 32.2$ min

b.

Priority	Job	Turnaround Time
3	B	9
4	E	$9 + 12 = 21$
6	A	$21 + 15 = 36$
7	C	$36 + 3 = 39$
9	D	$39 + 6 = 45$

The average turnaround time is: $(9+21+36+39+45) / 5 = 30$ min

c.

Job	Turnaround Time
A	15
B	$15 + 9 = 24$
C	$24 + 3 = 27$
D	$27 + 6 = 33$
E	$33 + 12 = 45$

The average turnaround time is: $(15+24+27+33+45) / 5 = 28.8$ min

d.

Running Time	Job	Turnaround Time
3	C	3
6	D	$3 + 6 = 9$
9	B	$9 + 9 = 18$
12	E	$18 + 12 = 30$
15	A	$30 + 15 = 45$

The average turnaround time is: $(3+9+18+30+45) / 5 = 21$ min

Please Do Not
Post on Web

CHAPTER 10 MULTIPROCESSOR AND REAL-TIME SCHEDULING

ANSWERS TO QUESTIONS

- 10.1 Fine:** Parallelism inherent in a single instruction stream. **Medium:** Parallel processing or multitasking within a single application. **Coarse:** Multiprocessing of concurrent processes in a multiprogramming environment. **Very Coarse:** Distributed processing across network nodes to form a single computing environment. **Independent:** Multiple unrelated processes.
- 10.2 Load sharing:** Processes are not assigned to a particular processor. A global queue of ready threads is maintained, and each processor, when idle, selects a thread from the queue. The term load sharing is used to distinguish this strategy from load-balancing schemes in which work is allocated on a more permanent basis. **Gang scheduling:** A set of related threads is scheduled to run on a set of processors at the same time, on a one-to-one basis. **Dedicated processor assignment:** Each program is allocated a number of processors equal to the number of threads in the program, for the duration of the program execution. When the program terminates, the processors return to the general pool for possible allocation to another program. **Dynamic scheduling:** The number of threads in a program can be altered during the course of execution.
- 10.3 First Come First Served (FCFS):** When a job arrives, each of its threads is placed consecutively at the end of the shared queue. When a processor becomes idle, it picks the next ready thread, which it executes until completion or blocking. **Smallest Number of Threads First:** The shared ready queue is organized as a priority queue, with highest priority given to threads from jobs with the smallest number of unscheduled threads. Jobs of equal priority are ordered according to which job arrives first. As with FCFS, a scheduled thread is run to completion or blocking. **Preemptive Smallest Number of Threads First:** Highest priority is given to jobs with the smallest number of unscheduled threads. An arriving job with a smaller number of threads than an executing job will preempt threads belonging to the scheduled job.
- 10.4 A hard real-time task** is one that must meet its deadline; otherwise it will cause undesirable damage or a fatal error to the system. A **soft real-time task** has an associated deadline that is desirable but not mandatory; it still makes sense to schedule and complete the task even if it has passed its deadline.

- 10.5** An **aperiodic task** has a deadline by which it must finish or start, or it may have a constraint on both start and finish time. In the case of a **periodic task**, the requirement may be stated as "once per period T " or "exactly T units apart."
- 10.6** **Determinism:** An operating system is deterministic to the extent that it performs operations at fixed, predetermined times or within predetermined time intervals. **Responsiveness:** Responsiveness is concerned with how long, after acknowledgment, it takes an operating system to service the interrupt. **User control:** The user should be able to distinguish between hard and soft tasks and to specify relative priorities within each class. A real-time system may also allow the user to specify such characteristics as the use of paging or process swapping, what processes must always be resident in main memory, what disk transfer algorithms are to be used, what rights the processes in various priority bands have, and so on. **Reliability:** Reliability must be provided in such a way as to continue to meet real-time deadlines. **Fail-soft operation:** Fail-soft operation is a characteristic that refers to the ability of a system to fail in such a way as to preserve as much capability and data as possible.
- 10.7** **Static table-driven approaches:** These perform a static analysis of feasible schedules of dispatching. The result of the analysis is a schedule that determines, at run time, when a task must begin execution. **Static priority-driven preemptive approaches:** Again, a static analysis is performed, but no schedule is drawn up. Rather, the analysis is used to assign priorities to tasks, so that a traditional priority-driven preemptive scheduler can be used. **Dynamic planning-based approaches:** Feasibility is determined at run time (dynamically) rather than offline prior to the start of execution (statically). An arriving task is accepted for execution only if it is feasible to meet its time constraints. One of the results of the feasibility analysis is a schedule or plan that is used to decide when to dispatch this task. **Dynamic best effort approaches:** No feasibility analysis is performed. The system tries to meet all deadlines and aborts any started process whose deadline is missed.
- 10.8** **Ready time:** time at which task becomes ready for execution. In the case of a repetitive or periodic task, this is actually a sequence of times that is known in advance. In the case of an aperiodic task, this time may be known in advance, or the operating system may only be aware when the task is actually ready. **Starting deadline:** Time by which a task must begin. **Completion deadline:** Time by which task must be completed. The typical real-time application will either have starting deadlines or completion deadlines, but not both. **Processing time:** Time required to execute the task to completion. In some cases, this is supplied. In others, the operating system measures an exponential average. For still other scheduling systems, this information is not used. **Resource requirements:** Set of resources (other than the processor) required by the task while it is executing. **Priority:** Measures relative importance of the task. Hard real-time tasks may have an "absolute" priority, with the system failing if a deadline is missed. If the system is to continue to run no matter what, then both hard and soft real-time tasks may be assigned relative priorities as a guide to the scheduler. **Subtask structure:** A task may be decomposed into a mandatory subtask and an optional subtask. Only the mandatory subtask possesses a hard deadline.

ANSWERS TO PROBLEMS

10.1 For fixed priority, we do the case in which the priority is A, B, C. Each square represents five time units; the letter in the square refers to the currently running process. The first row is fixed priority; the second row is earliest deadline scheduling using completion deadlines.

A	A	B	B	A	A	C	C	A	A	B	B	A	A	C	C	A	A		
A	A	B	B	A	C	C	A	C	A	A	B	B	A	A	C	C	C	A	A

For fixed priority scheduling, process C always misses its deadline.

10.2 Each square represents 10 time units.

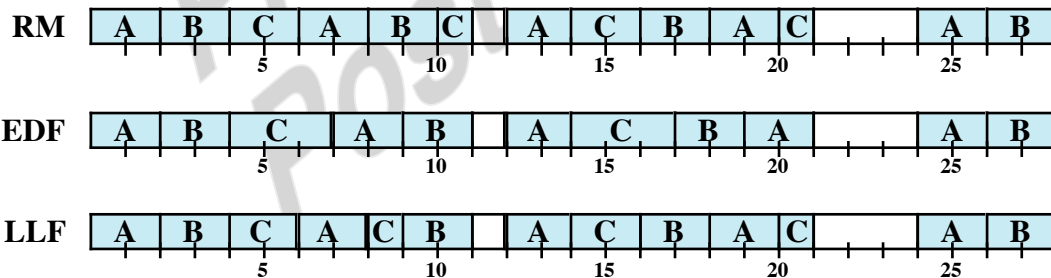
Earliest deadline

Earliest deadline with unforced idle times

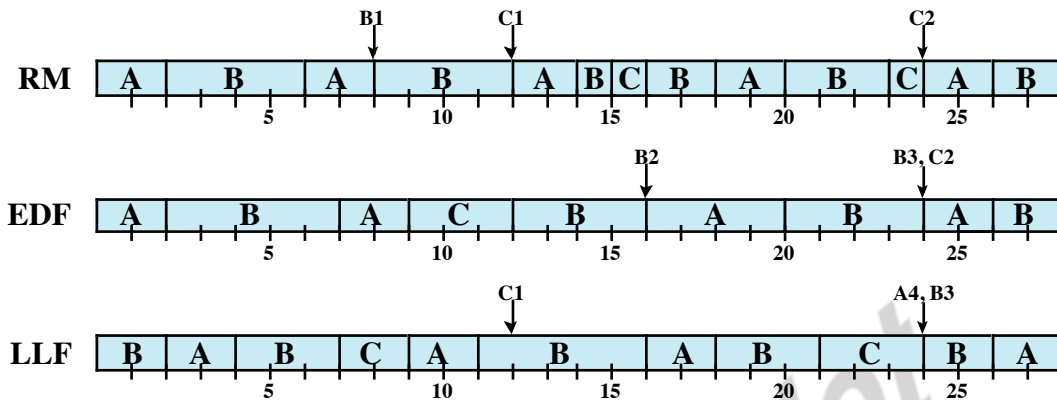
FCFS

	A	A		C	C	E	E	D	D		
		B	B	C	C	E	E	D	D	A	A
	A	A		C	C	D	D				

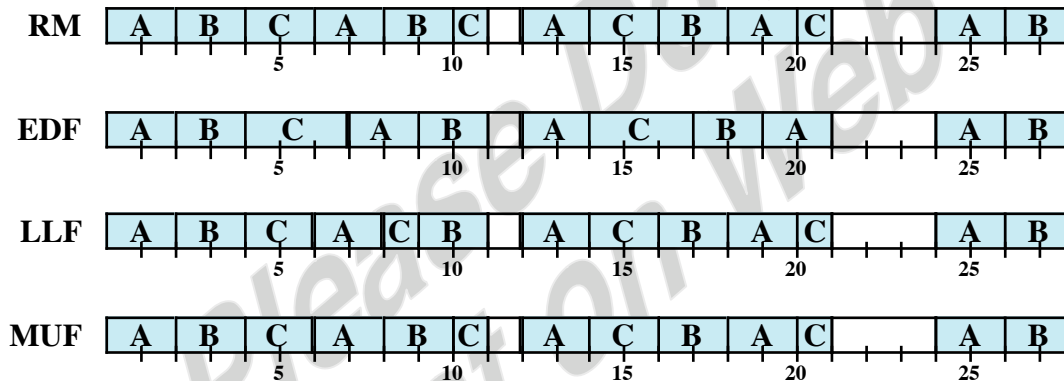
- 10.3**
- The task may be delayed up to an interval of t and still meet its deadline.
 - A laxity of 0 means that the task must be executed now or will fail to meet its deadline.
 - A task with negative laxity cannot meet its deadline.
 - The task set has a total load of 83%. The resulting schedules differ in the number of (potentially costly) context switches: 13, 11, and 13, respectively. Any timeline repeats itself every 24 time units. Although the total load (83%) is higher than the RM limit for this case (78%), RM can schedule the set satisfactorily.



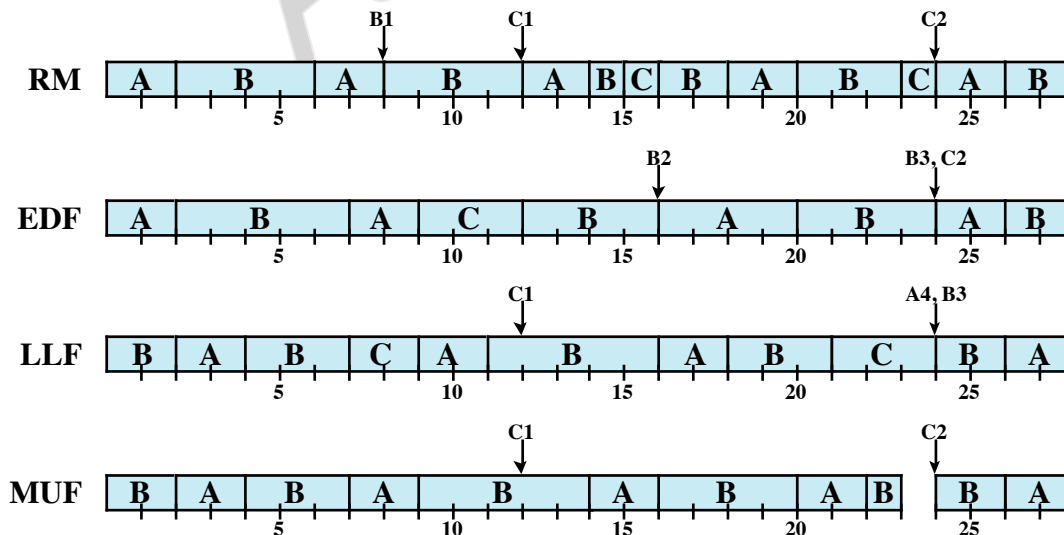
10.4 The total load now exceeds 100%. None of the methods can handle the load. The tasks that fail to complete vary for the various methods.



10.5 On this task profile, MUF behavior is identical to RM. Note that EDF has fewer context switches (11) than the other methods, all of which have 13.

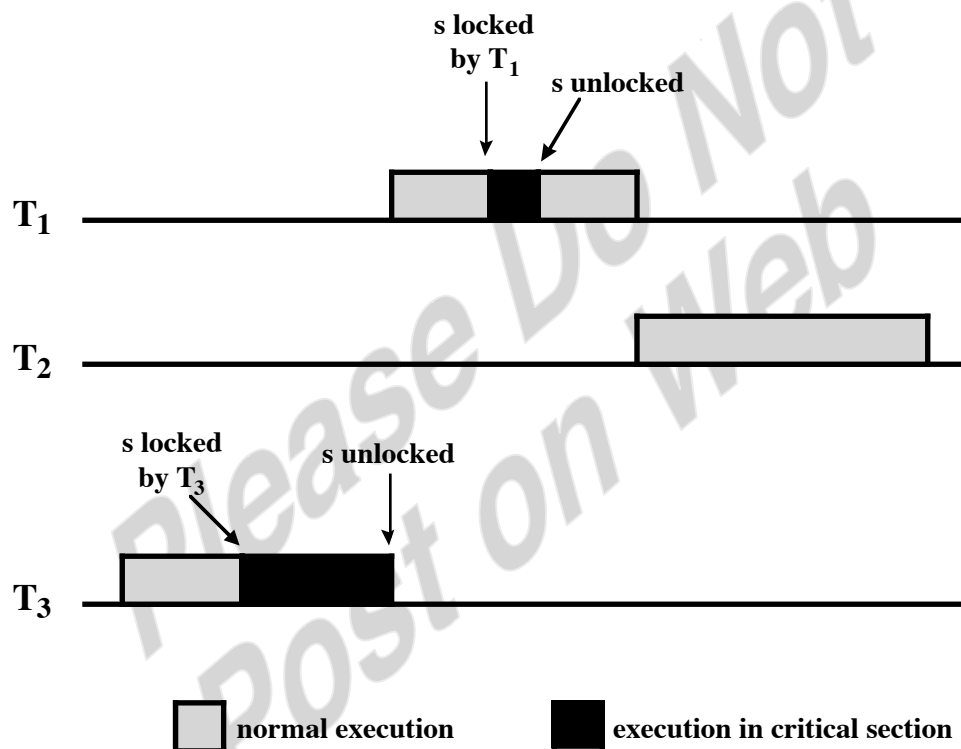


10.6 The critical task set consists of A and B. Note that only MUF is the only algorithm that gets the critical set (A and B) scheduled in time.



- 10.7 a. The total utilization of P_1 and P_2 is 0.41, which is less than 0.828, the bound given for two tasks by Equation 10.2. Therefore, these two tasks are schedulable.
- b. The utilization of all the tasks is 0.86, which exceeds the bound of 0.779.
- c. Observe that P_1 and P_2 must execute at least once before P_3 can begin executing. Therefore, the completion time of the first instance of P_3 can be no less than $20 + 30 + 68 = 118$. However, P_1 is initiated one additional time in the interval $(0, 118)$. Therefore, P_3 does not complete its first execution until $118 + 20 = 138$. This is within the deadline for P_3 . By continuing this reasoning, we can see that all deadlines of all three tasks can be met.

10.8



Once T_3 enters its critical section, it is assigned a priority higher than T_1 . When T_3 leaves its critical section, it is preempted by T_1 .

CHAPTER 11 I/O MANAGEMENT AND DISK SCHEDULING

ANSWERS TO QUESTIONS

- 11.1 Programmed I/O:** The processor issues an I/O command, on behalf of a process, to an I/O module; that process then busy-waits for the operation to be completed before proceeding. **Interrupt-driven I/O:** The processor issues an I/O command on behalf of a process, continues to execute subsequent instructions, and is interrupted by the I/O module when the latter has completed its work. The subsequent instructions may be in the same process, if it is not necessary for that process to wait for the completion of the I/O. Otherwise, the process is suspended pending the interrupt and other work is performed. **Direct memory access (DMA):** A DMA module controls the exchange of data between main memory and an I/O module. The processor sends a request for the transfer of a block of data to the DMA module and is interrupted only after the entire block has been transferred.
- 11.2 Logical I/O:** The logical I/O module deals with the device as a logical resource and is not concerned with the details of actually controlling the device. The logical I/O module is concerned with managing general I/O functions on behalf of user processes, allowing them to deal with the device in terms of a device identifier and simple commands such as open, close, read, write. **Device I/O:** The requested operations and data (buffered characters, records, etc.) are converted into appropriate sequences of I/O instructions, channel commands, and controller orders. Buffering techniques may be used to improve utilization.
- 11.3 Block-oriented** devices stores information in blocks that are usually of fixed size, and transfers are made one block at a time. Generally, it is possible to reference data by its block number. Disks and tapes are examples of block-oriented devices. **Stream-oriented** devices transfer data in and out as a stream of bytes, with no block structure. Terminals, printers, communications ports, mouse and other pointing devices, and most other devices that are not secondary storage are stream oriented.
- 11.4** Double buffering allows two operations to proceed in parallel rather than in sequence. Specifically, a process can transfer data to (or from) one buffer while the operating system empties (or fills) the other.
- 11.5** Seek time, rotational delay, access time.

- 11.6 FIFO:** Items are processed from the queue in sequential first-come-first-served order. **SSTF:** Select the disk I/O request that requires the least movement of the disk arm from its current position. **SCAN:** The disk arm moves in one direction only, satisfying all outstanding requests en route, until it reaches the last track in that direction or until there are no more requests in that direction. The service direction is then reversed and the scan proceeds in the opposite direction, again picking up all requests in order. **C-SCAN:** Similar to SCAN, but restricts scanning to one direction only. Thus, when the last track has been visited in one direction, the arm is returned to the opposite end of the disk and the scan begins again.
- 11.7 0:** Non-redundant **1:** Mirrored; every disk has a mirror disk containing the same data. **2:** Redundant via Hamming code; an error-correcting code is calculated across corresponding bits on each data disk, and the bits of the code are stored in the corresponding bit positions on multiple parity disks. **3:** Bit-interleaved parity; similar to level 2 but instead of an error-correcting code, a simple parity bit is computed for the set of individual bits in the same position on all of the data disks. **4:** Block-interleaved parity; a bit-by-bit parity strip is calculated across corresponding strips on each data disk, and the parity bits are stored in the corresponding strip on the parity disk. **5:** Block-interleaved distributed parity; similar to level 4 but distributes the parity strips across all disks. **6:** Block-interleaved dual distributed parity; two different parity calculations are carried out and stored in separate blocks on different disks.

11.8 512 bytes.

ANSWERS TO PROBLEMS

- 11.1** If the calculation time exactly equals the I/O time (which is the most favorable situation), both the processor and the peripheral device running simultaneously will take half as long as if they ran separately. Formally, let C be the calculation time for the entire program and let T be the total I/O time required. Then the best possible running time with buffering is $\max(C, T)$, while the running time without buffering is $C + T$; and of course $((C + T)/2) \leq \max(C, T) \leq (C + T)$. Source: [KNUT97].
- 11.2** The best ratio is $(n + 1):n$. Source: [KNUT97].

11.3 Disk head is initially moving in the direction of decreasing track number:

FIFO		SSTF		SCAN		C-SCAN	
Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed
27	73	110	10	64	36	64	36
129	102	120	10	41	23	41	23
110	19	129	9	27	14	27	14
186	76	147	18	10	17	10	17
147	39	186	39	110	100	186	176
41	106	64	122	120	10	147	39
10	31	41	23	129	9	129	18
64	54	27	14	147	18	120	9
120	56	10	17	186	39	110	10
Average	61.8	Average	29.1	Average	29.6	Average	38

If the disk head is initially moving in the direction of increasing track number, only the SCAN and C-SCAN results change:

SCAN		C-SCAN	
Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed
110	10	110	10
120	10	120	10
129	9	129	9
147	18	147	18
186	39	186	39
64	122	10	176
41	23	27	17
27	14	41	14
10	17	64	23
Average	29.1	Average	35.1

11.4 It will be useful to keep the following representation of the N tracks of a disk in mind:

0	1	• • •	j - 1	• • •	N - j	• • •	N - 2	N - 1
---	---	-------	-------	-------	-------	-------	-------	-------

- Let us use the notation $P_s[j/t] = \Pr[\text{seek of length } j \text{ when head is currently positioned over track } t]$. Recognize that each of the N tracks is equally likely to be requested. Therefore the unconditional probability of selecting any particular track is $1/N$. We can then state:

$$Ps[j / t] = \frac{1}{N} \quad \text{if} \quad t \leq j-1 \text{ OR } t \geq N-j$$

$$Ps[j / t] = \frac{2}{N} \quad \text{if} \quad j-1 < t < N-j$$

In the former case, the current track is so close to one end of the disk (track 0 or track $N-1$) that only one track is exactly j tracks away. In the second case, there are two tracks that are exactly j tracks away from track t , and therefore the probability of a seek of length j is the probability that either of these two tracks is selected, which is just $2/N$.

- b. Let $Ps[K] = \Pr[\text{seek of length } K, \text{ independent of current track position}]$. Then:

$$\begin{aligned} Ps[K] &= \sum_{t=0}^{N-1} Ps[K/t] \times \Pr[\text{current track is track } t] \\ &= \frac{1}{N} \sum_{t=0}^{N-1} Ps[K/t] \end{aligned}$$

From part (a), we know that $Ps[K/t]$ takes on the value $1/N$ for $2K$ of the tracks, and the value $2/N$ for $(N-2K)$ of the tracks. So

$$Ps[K] = \frac{1}{N} \left(\frac{2K}{N} + \frac{2(N-K)}{N} \right) = \frac{2}{N^2} (N-K)$$

- c.

$$\begin{aligned} E[K] &= \sum_{K=0}^{N-1} K \times Ps[K] = \sum_{K=0}^{N-1} \frac{2K(N-K)}{N^2} \\ &= \frac{2}{N} \sum_{K=0}^{N-1} K - \frac{2}{N^2} \sum_{K=0}^{N-1} K^2 \\ &= (N+1) - \frac{2(N+1)(2N+1)}{6N} \\ &= \frac{N^2-1}{3N} \end{aligned}$$

- d. This follows directly from the last equation.

11.5 Define

A_i = Time to retrieve a word from memory level i

H_i = Probability that a word is in memory level i and no higher-level memory

B_i = Time to transfer a block of data from memory level $(i+1)$ to memory level i

Let cache be memory level 1; main memory, memory level 2, and so on, for a total of N levels of memory. Then, we can write

$$T_s = \sum_{i=1}^N A_i H_i$$

Now, recognize that if a word is in M_1 (cache), it is read immediately. If it is in M_2 but not in M_1 , then a block of data is transferred from M_2 to M_1 and then read.

Thus $A_2 = B_1 + A_1$

Further $A_3 = B_2 + A_2 = B_1 + B_2 + A_1$

Generalizing, $A_i = A_1 + \sum_{j=1}^{i-1} B_j$

So $T_s = T_1 \sum_{i=1}^N H_i + \sum_{L=2}^N \sum_{j=1}^{L-1} B_j H_i$

But $\sum_{i=1}^N H_i = 1$

Finally $T_s = T_1 + \sum_{L=2}^N \sum_{j=1}^{L-1} B_j H_i$

11.6 a. The middle section in Figure 11.11b is empty. Thus, this reduces to the strategy of Figure 11.11a.

b. The old section consists of one block, and we have the LRU replacement policy.

11.7 Each sector can hold 4 logical records. The required number of sectors is $303,150 / 4 = 75,788$ sectors. This requires $75,788 / 96 = 790$ tracks, which in turn requires $790 / 110 = 8$ surfaces.

11.8 There are 512 bytes/sector. Since each byte generates an interrupt, there are 512 interrupts. Total interrupt processing time = $2.5 \times 512 = 1280 \mu s$. The time to read one sector is:

$$\begin{aligned} & ((60 \text{ sec/min}) / (360 \text{ rev/min})) / (96 \text{ sectors/track}) \\ & = 0.001736 \text{ sec} = 1736 \mu s \end{aligned}$$

Percentage of time processor spends handling I/O:

$$(100) \times (1280 / 1736) = 74\%$$

11.9 With DMA, there is only one interrupt of $2.5 \mu s$. Therefore, the percentage of time the processor spends handling I/O is

$$(100) \times (2.5 / 1736) = 0.14\%$$

11.10 Only one device at a time can be serviced on a selector channel. Thus,

$$\text{Maximum rate} = 800 + 800 + 2 \times 6.6 + 2 \times 1.2 + 10 \times 1 = 1625.6 \text{ KBytes/sec}$$

11.11 It depends on the nature of the I/O request pattern. On one extreme, if only a single process is doing I/O and is only doing one large I/O at a time, then disk striping improves performance. If there are many processes making many small I/O requests, then a nonstriped array of disks should give comparable performance to RAID 0.

11.12	RAID 0: 800 GB	RAID 4: 600 GB
	RAID 1: 400 GB	RAID 5: 600 GB
	RAID 3: 600 GB	RAID 6: 400 GB

Please Do Not
Post on Web

CHAPTER 12 FILE MANAGEMENT

ANSWERS TO QUESTIONS

- 12.1 A **field** is the basic element of data containing a single value. A **record** is a collection of related fields that can be treated as a unit by some application program.
- 12.2 A **file** is a collection of similar records, and is treated as a single entity by users and applications and may be referenced by name. A **database** is a collection of related data. The essential aspects of a database are that the relationships that exist among elements of data are explicit and that the database is designed for use by a number of different applications.
- 12.3 A file management system is that set of system software that provides services to users and applications in the use of files.
- 12.4 Rapid access, ease of update, economy of storage, simple maintenance, reliability.
- 12.5 **Pile:** Data are collected in the order in which they arrive. Each record consists of one burst of data. **Sequential file:** A fixed format is used for records. All records are of the same length, consisting of the same number of fixed-length fields in a particular order. Because the length and position of each field is known, only the values of fields need to be stored; the field name and length for each field are attributes of the file structure. **Indexed sequential file:** The indexed sequential file maintains the key characteristic of the sequential file: records are organized in sequence based on a key field. Two features are added; an index to the file to support random access, and an overflow file. The index provides a lookup capability to reach quickly the vicinity of a desired record. The overflow file is similar to the log file used with a sequential file, but is integrated so that records in the overflow file are located by following a pointer from their predecessor record. **Indexed file:** Records are accessed only through their indexes. The result is that there is now no restriction on the placement of records as long as a pointer in at least one index refers to that record. Furthermore, variable-length records can be employed. **Direct, or hashed, file:** The direct file makes use of hashing on the key value.
- 12.6 In a sequential file, a search may involve sequentially testing every record until the one with the matching key is found. The indexed sequential file provides a structure that allows a less exhaustive search to be performed.
- 12.7 Search, create file, delete file, list directory, update directory.

- 12.8 The pathname is an explicit enumeration of the path through the tree-structured directory to a particular point in the directory. The working directory is a directory within that tree structure that is the current directory that a user is working on.
- 12.9 None, knowledge of, read, write, execute, change protection, delete.
- 12.10 **Fixed blocking:** Fixed-length records are used, and an integral number of records are stored in a block. There may be unused space at the end of each block. This is referred to as internal fragmentation. **Variable-length spanned blocking:** Variable-length records are used and are packed into blocks with no unused space. Thus, some records must span two blocks, with the continuation indicated by a pointer to the successor block. **Variable-length unspanned blocking:** Variable-length records are used, but spanning is not employed. There is wasted space in most blocks because of the inability to use the remainder of a block if the next record is larger than the remaining unused space.
- 12.11 **Contiguous allocation:** a single contiguous set of blocks is allocated to a file at the time of file creation. **Chained allocation:** allocation is on an individual block basis. Each block contains a pointer to the next block in the chain. **Indexed allocation:** the file allocation table contains a separate one-level index for each file; the index has one entry for each portion allocated to the file.

ANSWERS TO PROBLEMS

- 12.1 **Fixed blocking:** $F = \text{largest integer} \leq \frac{B}{R}$

When records of variable length are packed into blocks, data for marking the record boundaries within the block has to be added to separate the records. When spanned records bridge block boundaries, some reference to the successor block is also needed. One possibility is a length indicator preceding each record. Another possibility is a special separator marker between records. In any case, we can assume that each record requires a marker, and we assume that the size of a marker is about equal to the size of a block pointer [WEID87]. For spanned blocking, a block pointer of size P to its successor block may be included in each block, so that the pieces of a spanned record can easily be retrieved. Then we have

Variable-length spanned blocking:
$$F = \frac{B - P}{R + P}$$

With unspanned variable-length blocking, an average of R/2 will be wasted because of the fitting problem, but no successor pointer is required:

Variable-length unspanned blocking:
$$F = \frac{B - \frac{R}{2}}{R}$$

- 12.2 a. $\log_2 \frac{N}{F}$
 b. Less than half the allocated file space is unused at any time.
- 12.3 a. Indexed
 b. Indexed sequential
 c. Hashed or indexed

12.4	File size	41,600 bytes	640,000 bytes	4,064,000 bytes
	No. of blocks	3 (rounded up to whole number)	40	249
	Total capacity	$3 \times 16K = 48K = 49,152$ bytes	$40 \times 16K = 640K = 655,360$ bytes	$249 \times 16K = 3984K = 4,079,616$ bytes
	Wasted space	7,552 bytes	15,360 bytes	15,616 bytes
	% of wasted space	15.36%	2.34%	0.38%

12.5 Directories enable files to be organized and clearly separated on the basis of ownership, application, or some other criterion. This improves security, integrity (organizing backups), and avoids the problem of name clashes.

12.6 Clearly, security is more easily enforced if directories are easily recognized as special files by the operating system. Treating a directory as an ordinary file with certain assigned access restrictions provides a more uniform set of objects to be managed by the operating system and may make it easier to create and manage user-owned directories.

12.7 This is a rare feature. If the operating system structures the file system so that subdirectories are allowed underneath a master directory, there is little or no additional logic required to allow arbitrary depth of subdirectories. Limiting the depth of the subdirectory tree places an unnecessary limitation on the user's organization of file space.

- 12.8 a. Yes, the method employed is very similar to that used by many LISP systems for garbage collection. First we would establish a data structure representing every block on a disk supporting a file system. A bit map would be appropriate here. Then, we would start at the root of the file system (the "/" directory), and mark every block used by every file we could find through a recursive descent through the file system. When finished, we would create a free list from the blocks remaining as unused. This is essentially what the UNIX utility `fsck` does.
- b. Keep a "backup" of the free-space list pointer at one or more places on the disk. Whenever this beginning of the list changes, the "backup" pointers are also updated. This will ensure you can always find a valid pointer value even if there is a memory or disk block failure.

12.9	Level	Number of Blocks	Number of Bytes
	Direct	10	10K
	Single indirect	256	256K
	Double indirect	$256 \times 256 = 65K$	65M
	Triple indirect	$256 \times 65K = 16M$	16G

The maximum file size is over 16G bytes.

- 12.10 a. Find the number of disk block pointers that fit in one block by dividing the block size by the pointer size:

$$8K/4 = 2K \text{ pointers per block}$$

The maximum file size supported by the inode is thus:

12	+ 2K	+ (2K × 2K)	+ (2K × 2K × 2K)
Direct	Indirect – 1	Indirect – 2	Indirect – 3
12	+ 2K	+ 4M	+ 8G blocks

Which, when multiplied by the block size (8K), is

$$96KB + 16MB + 32GB + 64TB$$

Which is HUGE.

- b. There are 24 bits for identifying blocks within a partition, so that leads to:

$$2^{24} \times 8K = 16M \times 8K = 128 \text{ GB}$$

- c. Using the information from (a), we see that the direct blocks only cover the first 96KB, while the first indirect block covers the next 16MB. the requested file position is 13M and change, which clearly falls within the range of the first indirect block. There will thus be two disk accesses. One for the first indirect block, and one for the block containing the required data.

CHAPTER 13 EMBEDDED OPERATING SYSTEMS

ANSWERS TO QUESTIONS

- 13.1 A combination of computer hardware and software, and perhaps additional mechanical or other parts, designed to perform a dedicated function. In many cases, embedded systems are part of a larger system or product, as in the case of an antilock braking system in a car.
- 13.2
- Small to large systems, implying very different cost constraints, thus different needs for optimization and reuse
 - Relaxed to very strict requirements and combinations of different quality requirements, for example, with respect to safety, reliability, real-time, flexibility, and legislation
 - Short to long life times
 - Different environmental conditions in terms of, for example, radiation, vibrations, and humidity
 - Different application characteristics resulting in static versus dynamic loads, slow to fast speed, compute versus interface intensive tasks, and/or combinations thereof
 - Different models of computation ranging from discrete-event systems to those involving continuous time dynamics (usually referred to as hybrid systems)
- 13.3 An embedded OS is an OS designed for the embedded system environment.
- 13.4
- **Real time operation:** In many embedded systems, the correctness of a computation depends, in part, on the time at which it is delivered. Often, real-time constraints are dictated by external I/O and control stability requirements.
 - **Reactive operation:** Embedded software may execute in response to external events. If these events do not occur periodically or at predictable intervals, the embedded software may need to take into account worst-case conditions and set priorities for execution of routines.
 - **Configurability:** Because of the large variety of embedded systems, there is a large variation in the requirements, both qualitative and quantitative, for embedded OS functionality. Thus, an embedded OS intended for use on a variety of embedded systems lends itself to flexible configuration so that only the functionality needed for a specific application and hardware suite is provided.
 - **I/O device flexibility:** There is virtually no device that needs to be supported by all versions of the OS, and the range of I/O devices is large.
 - **Streamlined protection mechanisms:** Embedded systems are typically designed for a limited, well-defined functionality. Untested programs are rarely added to the software. After the software has been configured and tested, it can be assumed to be reliable. Thus, apart from security measures, embedded systems

have limited protection mechanisms. For example, I/O instructions need not be privileged instructions that trap to the OS; tasks can directly perform their own I/O. Similarly, memory protection mechanisms can be minimized.

- **Direct use of interrupts:** General-purpose operating systems typically do not permit any user process to use interrupts directly.

- 13.5 An advantage of an embedded OS based on an existing commercial OS compared to a purpose-built embedded OS is that the embedded OS derived from a commercial general-purpose OS is based on a set of familiar interfaces, which facilitates portability. The disadvantage of using a general-purpose OS is that it is not optimized for real-time and embedded applications. Thus, considerable modification may be required to achieve adequate performance.
- 13.6
- **Low interrupt latency:** the time it takes to respond to an interrupt and begin executing an ISR.
 - **Low task switching latency:** the time it takes from when a thread becomes available to when actual execution begins.
 - **Small memory footprint:** memory resources for both program and data are kept to a minimum by allowing all components to configure memory as needed.
 - **Deterministic behavior:** throughout all aspect of execution, the kernels performance must be predictable and bounded to meet real-time application requirements.
- 13.7 An ISR is invoked in response to a hardware interrupt. Hardware interrupts are delivered with minimal intervention to an ISR. The HAL decodes the hardware source of the interrupt and calls the ISR of the attached interrupt object. This ISR may manipulate the hardware but is only allowed to make a restricted set of calls on the driver API. When it returns, an ISR may request that its DSR should be scheduled to run. A DSR is invoked in response to a request by an ISR. A DSR will be run when it is safe to do so without interfering with the scheduler. Most of the time the DSR will run immediately after the ISR, but if the current thread is in the scheduler, it will be delayed until the thread is finished. A DSR is allowed to make a larger set of driver API calls, including, in particular, being able to call `cyg_drv_cond_signal()` to wake up waiting threads.
- 13.8 The eCos kernel can be configured to include one or more of six different thread synchronization mechanisms. These include the classic synchronization mechanisms: mutexes, semaphores, and condition variables. In addition, eCos supports two synchronization/communication mechanisms that are common in real-time systems, namely event flags and mailboxes. Finally, the eCos kernel supports spinlocks, which are useful in SMP (symmetric multiprocessing) systems.
- 13.9 A wireless sensor network.
- 13.10
- **Allow high concurrency:** In a typical wireless sensor network application, the devices are concurrency intensive. Several different flows of data must be kept moving simultaneously. While sensor data is input in a steady stream, processed results must be transmitted in a steady stream. In addition, external controls from remote sensors or base stations must be managed.

- **Operate with limited resources:** The target platform for TinyOS will have limited memory and computational resources and run on batteries or solar power. A single platform may offer only kilobytes of program memory and hundreds of bytes of RAM. The software must make efficient use of the available processor and memory resources while enabling low power communication.
- **Adapt to hardware evolution:** Mote hardware is in constant evolution; applications and most system services must be portable across hardware generations. Thus, should be possible to upgrade the hardware with little or no software change, if the functionality is the same.
- **Support a wide range of applications:** Applications exhibit a wide range of requirements in terms of lifetime, communication, sensing, and so on. A modular, general-purpose embedded OS is desired so that a standardized approach leads to economies of scale in developing applications and support software.
- **Support a diverse set of platforms:** As with the preceding point, a general-purpose embedded OS is desirable.
- **Be robust:** Once deployed, a sensor network must run unattended for months or years. Ideally, there should be redundancy both within a single system and across the network of sensors. However, both types of redundancy require additional resources. One software characteristic that can improve robustness is to use highly modular, standardized software components.

13.11 An embedded software system built using TinyOS consists of a set of small modules, called components, each of which performs a simple task or set of tasks, and which interface with each other and with hardware in limited and well-defined ways.

13.12 The scheduler plus a number of components.

13.13 The default scheduler in TinyOS is a simple FIFO (first-in-first-out) queue.

ANSWERS TO PROBLEMS

13.1 This ensures proper exclusion with code running on both other processors and this processor.

13.2 Wherever this is called from, this operation effectively pauses the processor until it succeeds. This operation should therefore be used sparingly, and in situations where deadlocks cannot occur.

13.3 There should be no processors attempting to claim the lock at the time this function is called, otherwise the behavior is undefined.

13.4 The mutex should be unlocked and there should be no threads waiting to lock it when this call is made.

13.5 Timeslicing within a given priority level is irrelevant since there can be only one thread at each priority level.

- 13.6** If a thread has locked a mutex and then attempts to lock the mutex again, typically as a result of some recursive call in a complicated call graph, then either an assertion failure will be reported or the thread will deadlock. This behavior is deliberate. When a thread has just locked a mutex associated with some data structure, it can assume that that data structure is in a consistent state. Before unlocking the mutex again it must ensure that the data structure is again in a consistent state. Recursive mutexes allow a thread to make arbitrary changes to a data structure, then in a recursive call lock the mutex again while the data structure is still inconsistent. The net result is that code can no longer make any assumptions about data structure consistency, which defeats the purpose of using mutexes.
- 13.7**
- a. This listing is an example using a condition variable. Thread A is acquiring data that are processed by Thread B. First, B executes. On line 24, B acquires the mutex associated with the condition variable. There is no data in the buffer and `buffer_empty` is initialized so `true`, so B calls `cyg_cond_wait` on line 26. This call suspends B waiting for the condition variable to be set and it unlocks the mutex `mut_cond_var`. Once A has acquired data, it sets `buffer_empty` to `false` (line 11). A then locks the mutex (line 13), signals the condition variable (line 15), and then unlocks the mutex (line 17). B can now run. Before returning from `cyg_cond_wait`, the mutex `mut_cond_var` is locked and owned by B. B can now get the data buffer (line 28) and set `buffer_empty` to `true` (line 31). Finally, the mutex is released by B (line 33) and the data in the buffer is processed (line 35)
 - b. If this code were not atomic, it would be possible for B to miss the signal call from A even though the buffer contained data. This is because `cyg_cond_wait` first checks to see if the condition variable is set and, in this case, it is not. Next, the mutex is released in the `cyg_cond_wait` call. Now, A executes, putting data into the buffer and then signals the condition variable (line 15). Then, B returns to waiting. However, the condition variable has been set.
 - c. This ensures that the condition that B is waiting on is still true after returning from the condition wait call. This is needed in case there are other threads waiting on the same condition. Source: [MASS03]
- 13.8** Even if the two threads were running at the same priority, the one attempting to claim the spinlock would spin until it was timesliced and a lot of processor time would be wasted. If an interrupt handler tried to claim a spinlock owned by a thread, the interrupt handler would loop forever.
- 13.9** The basic reason is that the system is almost completely idle. You only need intelligent scheduling when a resource is under contention.

- 13.10** a. Clients are not able to monopolize the resource queue by making multiple requests.
- b. The basic reason is that a component may still hold a lock after it calls release. It may still hold the lock because transferring control to the next holder requires reconfiguring the hardware in a split-phase operation. The lock can be granted to the next holder only after this reconfiguration occurs. This means that technically, a snippet of code such as
- ```
release()
request()
```
- can be seen as the lock as re-request. There are other solutions to the problem (e.g., give the lock a special "owner" who holds it while reconfiguring), but this raises issue when accounting for CPU cycles or energy.

Please Do Not  
Post on Web



# CHAPTER 14 COMPUTER SECURITY THREATS

## ANSWERS TO QUESTIONS

- 14.1** The protection afforded to an automated information system in order to attain the applicable objectives of preserving the integrity, availability and confidentiality of information system resources (includes hardware, software, firmware, information/ data, and telecommunications).
- 14.2** Confidentiality, integrity, and availability.
- 14.3** Passive attacks have to do with eavesdropping on, or monitoring, transmissions. Electronic mail, file transfers, and client/server exchanges are examples of transmissions that can be monitored. Active attacks include the modification of transmitted data and attempts to gain unauthorized access to computer systems.
- 14.4** **Masquerader:** An individual who is not authorized to use the computer and who penetrates a system's access controls to exploit a legitimate user's account.  
**Misfeasor:** A legitimate user who accesses data, programs, or resources for which such access is not authorized, or who is authorized for such access but misuses his or her privileges. **Clandestine user:** An individual who seizes supervisory control of the system and uses this control to evade auditing and access controls or to suppress audit collection.
- 14.5** **Hackers:** those who hack into computers do so for the thrill of it or for status. The hacking community is a strong meritocracy in which status is determined by level of competence. Thus, attackers often look for targets of opportunity and then share the information with others.  
**Criminals:** organized groups of hackers, typically with specific target or target classes in mind.  
**Insider attacks:** carried out by those within an organization.
- 14.6** A virus may use compression so that the infected program is exactly the same length as an uninfected version.
- 14.7** A portion of the virus, generally called a *mutation engine*, creates a random encryption key to encrypt the remainder of the virus. The key is stored with the virus, and the mutation engine itself is altered. When an infected program is invoked, the virus uses the stored random key to decrypt the virus. When the virus replicates, a different random key is selected.
- 14.8** A dormant phase, a propagation phase, a triggering phase, and an execution phase

- 14.9** 1. Search for other systems to infect by examining host tables or similar repositories of remote system addresses. 2. Establish a connection with a remote system. 3. Copy itself to the remote system and cause the copy to be run.
- 14.10** A **bot** (robot), also known as a zombie or drone, is a program that secretly takes over another Internet-attached computer and then uses that computer to launch attacks that are difficult to trace to the bot's creator. A **rootkit** is a set of programs installed on a system to maintain administrator (or root) access to that system. Root access provides access to all the functions and services of the operating system. The rootkit alters the host's standard functionality in a malicious and stealthy way.

## ANSWERS TO PROBLEMS

- 14.1** a.  $T = \frac{26^4}{2}$  seconds = 63.5 hours  
 b. Expect 13 tries for each digit.  $T = 13 \times 4 = 52$  seconds.
- 14.2** The program will loop indefinitely once all of the executable files in the system are infected.
- 14.3** D is supposed to examine a program P and return TRUE if P is a computer virus and FALSE if it is not. But CV calls D. If D says that CV is a virus, then CV will not infect an executable. But if D says that CV is not a virus, it infects an executable. D always returns the wrong answer.
- 14.4** a. When the program is executed, it produces the following output:  
       begin print (); end.  
       The program was probably intended to produce, upon execution, an exact listing of the original program text. Clearly, it fails.
- b. This works. Basically, it is a three-step process: (1) declare a character string that corresponds to the main body of the program; (2) print each character of the defined string individually; (3) print the value of the array as a defined character string.
- c. The problem shows a self-replicating program, the type of functionality used in a virus.
- 14.5** Logic bomb.
- 14.6** Backdoor.
- 14.7** The original code has been altered to disrupt the signature without affecting the semantics of the code. The ineffective instructions in the metamorphic code are the second, third, fifth, sixth, and eighth.

# CHAPTER 15 COMPUTER SECURITY TECHNIQUES

## ANSWERS TO QUESTIONS

- 15.1 Something the individual knows:** Examples include a password, a personal identification number (PIN), or answers to a prearranged set of questions.  
**Something the individual possesses:** Examples include electronic keycards, smart cards, and physical keys. This type of authenticator is referred to as a token.  
**Something the individual is (static biometrics):** Examples include recognition by fingerprint, retina, and face.  
**Something the individual does (dynamic biometrics):** Examples include recognition by voice pattern, handwriting characteristics, and typing rhythm.
- 15.2** The salt is combined with the password at the input to the one-way encryption routine.
- 15.3** Memory cards can store but not process data. Smart cards have a microprocessor.
- 15.4 Facial characteristics:** Facial characteristics are the most common means of human-to-human identification; thus it is natural to consider them for identification by computer. The most common approach is to define characteristics based on relative location and shape of key facial features, such as eyes, eyebrows, nose, lips, and chin shape. An alternative approach is to use an infrared camera to produce a face thermogram that correlates with the underlying vascular system in the human face.
- Fingerprints:** Fingerprints have been used as a means of identification for centuries, and the process has been systematized and automated particularly for law enforcement purposes. A fingerprint is the pattern of ridges and furrows on the surface of the fingertip. Fingerprints are believed to be unique across the entire human population. In practice, automated fingerprint recognition and matching system extract a number of features from the fingerprint for storage as a numerical surrogate for the full fingerprint pattern.
- Hand geometry:** Hand geometry systems identify features of the hand, including shape, and lengths and widths of fingers.
- Retinal pattern:** The pattern formed by veins beneath the retinal surface is unique and therefore suitable for identification. A retinal biometric system obtains a digital image of the retinal pattern by projecting a low-intensity beam of visual or infrared light into the eye.
- Iris:** Another unique physical characteristic is the detailed structure of the iris.
- Signature:** Each individual has a unique style of handwriting and this is reflected especially in the signature, which is typically a frequently written sequence.

However, multiple signature samples from a single individual will not be identical. This complicates the task of developing a computer representation of the signature that can be matched to future samples.

**Voice:** Whereas the signature style of an individual reflects not only the unique physical attributes of the writer but also the writing habit that has developed, voice patterns are more closely tied to the physical and anatomical characteristics of the speaker. Nevertheless, there is still a variation from sample to sample over time from the same speaker, complicating the biometric recognition task.

- 15.5 Discretionary access control (DAC)** controls access based on the identity of the requestor and on access rules (authorizations) stating what requestors are (or are not) allowed to do. This policy is termed *discretionary* because an entity might have access rights that permit the entity, by its own volition, to enable another entity to access some resource. **Role-based access control (RBAC)** controls access based on the roles that users have within the system and on rules stating what accesses are allowed to users in given roles. RBAC may have a discretionary or mandatory mechanism.
- 15.6 Statistical anomaly detection** involves the collection of data relating to the behavior of legitimate users over a period of time. Then statistical tests are applied to observed behavior to determine with a high level of confidence whether that behavior is not legitimate user behavior. **Signature intrusion detection** involves an attempt to define a set of rules that can be used to decide that a given behavior is that of an intruder.
- 15.7** A digital immune system provides a general-purpose emulation and virus-detection system. The objective is to provide rapid response time so that viruses can be stamped out almost as soon as they are introduced. When a new virus enters an organization, the immune system automatically captures it, analyzes it, adds detection and shielding for it, removes it, and passes information about that virus to systems running a general antivirus program so that it can be detected before it is allowed to run elsewhere.
- 15.8** Behavior-blocking software integrates with the operating system of a host computer and monitors program behavior in real-time for malicious actions. The behavior blocking software then blocks potentially malicious actions before they have a chance to affect the system.
- 15.9 Signature-based worm scan filtering:** This type of approach generates a worm signature, which is then used to prevent worm scans from entering/leaving a network/host. Typically, this approach involves identifying suspicious flows and generating a worm signature. This approach is vulnerable to the use of polymorphic worms: Either the detection software misses the worm or, if it is sufficiently sophisticated to deal with polymorphic worms, the scheme may take a long time to react.
- Filter-based worm containment:** This approach is similar to class A but focuses on worm content rather than a scan signature. The filter checks a message to determine if it contains worm code.
- Payload-classification-based worm containment:** These network-based techniques examine packets to see if they contain a worm. Various anomaly

detection techniques can be used, but care is needed to avoid high levels of false positives or negatives.

**Threshold random walk (TRW) scan detection:** TRW exploits randomness in picking destinations to connect to as a way of detecting if a scanner is in operation [JUNG04]. TRW is suitable for deployment in high-speed, low-cost network devices. It is effective against the common behavior seen in worm scans.

**Rate limiting:** This class limits the rate of scanlike traffic from an infected host. Various strategies can be used, including limiting the number of new machines a host can connect to in a window of time, detecting a high connection failure rate, and limiting the number of unique IP addresses a host can scan in a window of time.

**Rate halting:** This approach immediately blocks outgoing traffic when a threshold is exceeded either in outgoing connection rate or diversity of connection attempts.

- 15.10 The programming languages vulnerable to buffer overflows are those without a very strong notion of the type of variables, and what constitutes permissible operations on them. They include assembly language, and C and similar languages. Strongly typed languages such as Java, ADA, Python, and many others are not vulnerable to these attacks.
- 15.11 Two broad categories of defenses against buffer overflows are: compile-time defenses which aim to harden programs to resist attacks in new programs; and run-time defenses which aim to detect and abort attacks in existing programs.
- 15.12 Compile-time defenses include: writing programs using a modern high-level programming language that is not vulnerable to buffer overflow attacks; using safe coding techniques to validate buffer use; using language safety extensions and/or safe library implementations; or using stack protection mechanisms.
- 15.13 Run-time defenses that provide some protection for existing vulnerable programs include: using “Executable Address Space Protection” that blocks execution of code on the stack, heap, or in global data; using “Address Space Randomization” to manipulate the location of key data structures such as the stack and heap in the processes address space; or by placing **guard pages** between critical regions of memory in a processes address space.

## ANSWERS TO PROBLEMS

- 15.1
  - a. If this is a license plate number, that is easily guessable.
  - b. suitable
  - c. easily guessable
  - d. easily guessable
  - e. easily guessable
  - f. suitable
  - g. very unsuitable
  - h. This is bigbird in reverse; not suitable.
- 15.2 The number of possible character strings of length 8 using a 36-character alphabet is  $36^8 \approx 2^{41}$ . However, only  $2^{15}$  of them need be looked at, because that is the



number of possible outputs of the random number generator. This scheme is discussed in [MORR79].

15.3 a.  $T = \frac{26^4}{2}$  seconds = 63.5 hours

b. Expect 13 tries for each digit.  $T = 13 \times 4 = 52$  seconds.

15.4 a.  $p = r^k$

b.  $p = \frac{r^k - r^p}{r^{k+p}}$

c.  $p = r^p$

15.5 There are  $95^{10} \approx 6 \times 10^{19}$  possible passwords. The time required is:

$$\frac{6 \times 10^{19} \text{ passwords}}{6.4 \times 10^6 \text{ passwords / second}} = 9.4 \times 10^{12} \text{ seconds} \\ = 300,000 \text{ years}$$

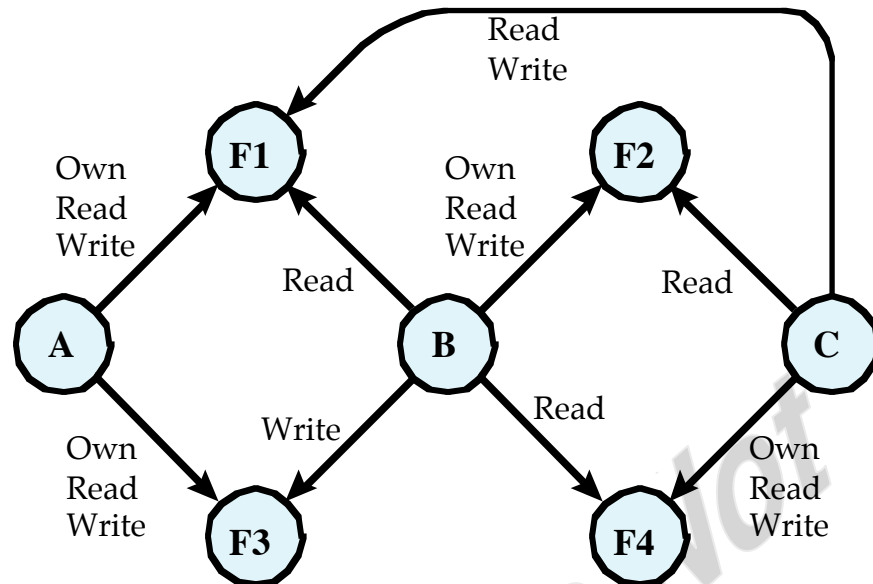
15.6 a. Since  $PU_a$  and  $PR_a$  are inverses, the value  $PR_a$  can be checked to validate that  $P_a$  was correctly supplied: Simply take some arbitrary block  $X$  and verify that  $X = D(PR_a, E[PU_a, X])$ .

b. Since the file /etc/publickey is publicly readable, an attacker can guess  $P$  (say  $P'$ ) and compute  $PR_a = D(P', E[P, PR_a])$ . now he can choose an arbitrary block  $Y$  and check to see if  $Y = D(PR_a, E[PU_a, Y])$ . If so, it is highly probable that  $P' = P$ . Additional blocks can be used to verify the equality.

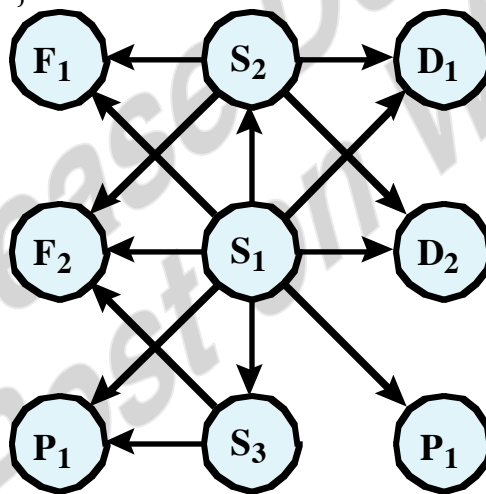
15.7 Without the salt, the attacker can guess a password and encrypt it. If ANY of the users on a system use that password, then there will be a match. With the salt, the attacker must guess a password and then encrypt it once for each user, using the particular salt for each user.

15.8 It depends on the size of the user population, not the size of the salt, since the attacker presumably has access to the salt for each user. The benefit of larger salts is that the larger the salt, the less likely it is that two users will have the same salt. If multiple users have the same salt, then the attacker can do one encryption per password guess to test all of those users.

15.9 a.



- b. For simplicity and clarity, the labels are omitted. Also, there should be arrowed lines from each subject node to itself.



- c. A given access matrix generates only one directed graph, and a given directed graph yields only one access matrix, so the correspondence is one-to-one.

**15.10** Suppose that the directory **d** and the file **f** have the same owner and group and that **f** contains the text *something*. Disregarding the superuser, no one besides the owner of **f** can change its contents, because only the owner has write permission. However, anyone in the owner's group has write permission for **d**, so that any such person can remove **f** from **d** and install a different version, which for most purposes is the equivalent of being able to modify **f**. This example is from Grampp, F., and Morris, R. "UNIX Operating System Security." *AT&T Bell Laboratories Technical Journal*, October 1984.

**15.11** A default UNIX file access of full access for the owner combined with no access for group and other means that newly created files and directories will only be accessible by their owner. Any access for other groups or users must be explicitly granted. This is the most common default, widely



used by government and business where the assumption is that a person's work is assumed private and confidential.

A default of full access for the owner combined with read/execute access for group and none for other means newly created files and directories are accessible by all members of the owner's group. This is suitable when there is a team of people working together on a server, and in general most work is shared with the group. However there are also other groups on the server for which this does not apply. An organization with cooperating teams may choose this.

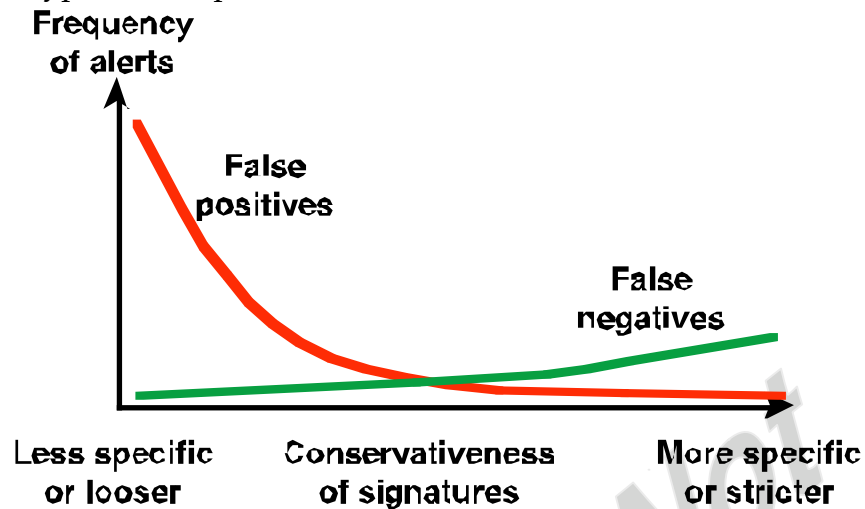
A default of full access for the owner combined with read/execute access for both group and other means newly created files and directories are accessible by all users on the server. This is appropriate for organization's where users trust each other in general, and assume that their work is a shared resource. This used to be the default for University staff, and in some research labs. It is also often the default for small businesses where people need to rely on and trust each other.

- 15.12** In order to provide the Web server access to a user's 'public\_html' directory, then search (execute) access must be provided to the user's home directory (and hence to all directories in the path to it), read/execute access to the actual Web directory, and read access to any Web pages in it, for others (since access cannot easily be granted just to the user that runs the web server). However this access also means that any user on the system (not just the web server) has this same access. Since the contents of the user's web directory are being published on the web, local public access is not unreasonable (since they can always access the files via the web server anyway). However in order to maintain these required permissions, if the system default is one of the more restrictive (and more common) options, then the user must set suitable permissions every time a new directory or file is created in the user's web area. Failure to do this means such directories and files are not accessible by the server, and hence cannot be access over the web. This is a common error. As well the fact that at least search access is granted to the user's home directory means that some information can be gained on its contents by other users, even if it is not readable, by attempting to access specific names. It also means that if the user accidentally grants too much access to a file, it may then be accessible to other users on the system. If the user's files are sufficiently sensitive, then the risk of accidental leakage due to inappropriate permissions being set may be too serious to allow such a user to have their own web pages.

**15.13** a. 
$$\sum_{i=1}^N (U_i \times P_i)$$

b. 
$$\sum_{i=1}^N (U_i + P_i)$$

15.14 This is a typical example:



15.15 Corrected version of the program shown in Figure 11.1a (see bold text):

```
int main(int argc, char *argv[]) {
 int valid = FALSE;
 char str1[8];
 char str2[8];

 next_tag(str1);
 fgets(str2, sizeof(str2), stdin);
 if (strncmp(str1, str2, sizeof(str2)) == 0)
 valid = TRUE;
 printf("buffer1: str1(%s), str2(%s), valid(%d)\n", str1, str2, valid);
}
```

# Chapter 16 DISTRIBUTED PROCESSING, CLIENT/SERVER, AND CLUSTERS

## ANSWERS TO QUESTIONS

- 16.1 A networked environment that includes client machines that make requests of server machines.
- 16.2 1. There is a heavy reliance on bringing user-friendly applications to the user on his or her own system. This gives the user a great deal of control over the timing and style of computer usage and gives department-level managers the ability to be responsive to their local needs. 2. Although applications are dispersed, there is an emphasis on centralizing corporate databases and many network management and utility functions. This enables corporate management to maintain overall control of the total capital investment in computing and information systems, and to provide interoperability so that systems are tied together. At the same time it relieves individual departments and divisions of much of the overhead of maintaining sophisticated computer-based facilities, but enables them to choose just about any type of machine and interface they need to access data and information. 3. There is a commitment, both by user organizations and vendors, to open and modular systems. This means that the user has greater choice in selecting products and in mixing equipment from a number of vendors. 4. Networking is fundamental to the operation. Thus, network management and network security have a high priority in organizing and operating information systems.
- 16.3 It is the communications software that enables client and server to interoperate.
- 16.4 **Server-based processing:** The rationale behind such configurations is that the user workstation is best suited to providing a user-friendly interface and that databases and applications can easily be maintained on central systems. Although the user gains the advantage of a better interface, this type of configuration does not generally lend itself to any significant gains in productivity or to any fundamental changes in the actual business functions that the system supports. **Client-based processing:** This architecture enables the user to employ applications tailored to local needs. **Cooperative processing:** This type of configuration may offer greater user productivity gains and greater network efficiency than other client/server approaches.
- 16.5 **Fat client:** Client-based processing, with most of the software at the client. The main benefit of the fat client model is that it takes advantage of desktop power, offloading application processing from servers and making them more efficient and less likely to be bottlenecks. **Thin client:** Server-based processing, with most

of the software at the server. This approach more nearly mimics the traditional host-centered approach and is often the migration path for evolving corporate wide applications from the mainframe to a distributed environment.

- 16.6 Fat client:** The main benefit is that it takes advantage of desktop power, offloading application processing from servers and making them more efficient and less likely to be bottlenecks. The addition of more functions rapidly overloads the capacity of desktop machines, forcing companies to upgrade. If the model extends beyond the department to incorporate many users, the company must install high-capacity LANs to support the large volumes of transmission between the thin servers and the fat clients. Finally, it is difficult to maintain, upgrade, or replace applications distributed across tens or hundreds of desktops. **Thin client:** This approach more nearly mimics the traditional host-centered approach and is often the migration path for evolving corporate wide applications from the mainframe to a distributed environment. It does not provide the flexibility of the fat client approach.
- 16.7** The middle tier machines are essentially gateways between the thin user clients and a variety of backend database servers. The middle tier machines can convert protocols and map from one type of database query to another. In addition, the middle tier machine can merge/integrate results from different data sources. Finally, the middle tier machine can serve as a gateway between the desktop applications and the backend legacy applications by mediating between the two worlds.
- 16.8** Middleware is a set of standard programming interfaces and protocols that sit between the application above and communications software and operating system below. It provides a uniform means and style of access to system resources across all platforms
- 16.9** TCP/IP does not provide the APIs and the intermediate-level protocols to support a variety of applications across different hardware and OS platforms.
- 16.10 Nonblocking primitives** provide for efficient, flexible use of the message-passing facility by processes. The disadvantage of this approach is that it is difficult to test and debug programs that use these primitives. Irreproducible, timing-dependent sequences can create subtle and difficult problems. **Blocking primitives** have the opposite advantages and disadvantages.
- 16.11 Nonpersistent binding:** Because a connection requires the maintenance of state information on both ends, it consumes resources. The nonpersistent style is used to conserve those resources. On the other hand, the overhead involved in establishing connections makes nonpersistent binding inappropriate for remote procedures that are called frequently by the same caller. **Persistent binding:** For applications that make many repeated calls to remote procedures, persistent binding maintains the logical connection and allows a sequence of calls and returns to use the same connection.
- 16.12** The **synchronous RPC** is easy to understand and program because its behavior is predictable. However, it fails to exploit fully the parallelism inherent in distributed applications. This limits the kind of interaction the distributed application can have, resulting in lower performance. To provide greater

flexibility, **asynchronous RPC** facilities achieve a greater degree of parallelism while retaining the familiarity and simplicity of the RPC. Asynchronous RPCs do not block the caller; the replies can be received as and when they are needed, thus allowing client execution to proceed locally in parallel with the server invocation.

- 16.13 Passive Standby:** A secondary server takes over in case of primary server failure. **Separate Servers:** Separate servers have their own disks. Data is continuously copied from primary to secondary server. **Servers Connected to Disks:** Servers are cabled to the same disks, but each server owns its disks. If one server fails, its disks are taken over by the other server. **Servers Share Disks:** Multiple servers simultaneously share access to disks.

## ANSWERS TO PROBLEMS

- 16.1** a. MIPS rate =  $[n\alpha + (1 - \alpha)] x = (n\alpha - \alpha + 1)x$   
 b.  $\alpha = 0.6$
- 16.2** a. One computer executes for a time T. Eight computers execute for a time T/4, which would take a time 2T on a single computer. Thus the total required time on a single computer is 3T. Effective speedup = 3.  $\alpha = 0.75$ .  
 b. New speedup = 3.43  
 c.  $\alpha$  must be improved to 0.8.
- 16.3** a. Sequential execution time = 1,051,628 cycles  
 b. Speedup = 16.28  
 c. Each computer is assigned 32 iterations balanced between the beginning and end of the I-loop.  
 d. The ideal speedup of 32 is achieved.

# CHAPTER 17 NETWORKING

## ANSWERS TO QUESTIONS

- 17.1 The network access layer is concerned with the exchange of data between a computer and the network to which it is attached.
- 17.2 The transport layer is concerned with data reliability and correct sequencing.
- 17.3 A protocol is the set of rules or conventions governing the way in which two entities cooperate to exchange data.
- 17.4 The software structure that implements the communications function. Typically, the protocol architecture consists of a layered set of protocols, with one or more protocols at each layer.
- 17.5 Transmission Control Protocol/Internet Protocol (TCP/IP) are two protocols originally designed to provide low level support for internetworking. The term is also used generically to refer to a more comprehensive collection of protocols developed by the U.S. Department of Defense and the Internet community.
- 17.6 A sockets interface is an API that enables programs to be writing that make use of the TCP/IP protocol suite to establish communication between a client and server.

## ANSWERS TO PROBLEMS

17.1 a.



The PMs speak as if they are speaking directly to each other. For example, when the French PM speaks, he addresses his remarks directly to the Chinese PM. However, the message is actually passed through two translators via the phone system. The French PM's translator translates his remarks into English and telephones these to the Chinese PM's translator, who translates these remarks into Chinese.

b.





An intermediate node serves to translate the message before passing it on.

- 17.2** Perhaps the major disadvantage is the processing and data overhead. There is processing overhead because as many as seven modules (OSI model) are invoked to move data from the application through the communications software. There is data overhead because of the appending of multiple headers to the data. Another possible disadvantage is that there must be at least one protocol standard per layer. With so many layers, it takes a long time to develop and promulgate the standards.
- 17.3** Data plus transport header plus internet header equals 1820 bits. This data is delivered in a sequence of packets, each of which contains 24 bits of network header and up to 776 bits of higher-layer headers and /or data. Three network packets are needed. Total bits delivered =  $1820 + 3 \times 24 = 1892$  bits.
- 17.4** UDP has a fixed-sized header. The header in TCP is of variable length.
- 17.5** Suppose that A sends a data packet  $k$  to B and the ACK from B is delayed but not lost. A resends packet  $k$ , which B acknowledges. Eventually A receives 2 ACKs to packet  $k$ , each of which triggers transmission of packet  $(k + 1)$ . B will ACK both copies of packet  $(k + 1)$ , causing A to send two copies of packet  $(k + 2)$ . From now on, 2 copies of every data packet and ACK will be sent.
- 17.6** TFTP can transfer a maximum of 512 bytes per round trip (data sent, ACK received). The maximum throughput is therefore 512 bytes divided by the round-trip time.
- 17.7** The "netascii" transfer mode implies the file data are transmitted as lines of ASCII text terminated by the character sequence {CR, LF}, and that both systems must convert between this format and the one they use to store the text files locally. This means that when the "netascii" transfer mode is employed, the file sizes of the local and the remote file may differ, without any implication of errors in the data transfer. For example, UNIX systems terminate lines by means of a single LF character, while other systems, such as Microsoft Windows, terminate lines by means of the character sequence {CR, LF}. This means that a given text file will usually occupy more space in a Windows host than in a UNIX system.
- 17.8** If the same TIDs are used in twice in immediate succession, there's a chance that packets of the first instance of the connection that were delayed in the network arrive during the life of the second instance of the connection, and, as they would have the correct TIDs, they could be (mistakenly) considered as valid.



- 17.9** TFTP needs to keep a copy of only the last packet it has sent, since the acknowledgement mechanism it implements guarantees that all the previous packets have been received, and thus will not need to be retransmitted.
- 17.10** This could trigger an "error storm". Suppose host A receives an error packet from host B, and responds it by sending an error packet back to host B. This packet could trigger another error packet from host B, which would (again) trigger an error packet at host A. Thus, error messages would bounce from one host to the other, indefinitely, congesting the network and consuming the resources of the participating systems.
- 17.11** The disadvantage is that using a fixed value for the retransmission timer means the timer will not reflect the characteristics of the network on which the data transfer is taking place. For example, if both hosts are on the same local area network, a 5-second timeout is more than enough. On the other hand, if the transfer is taking place over a (long delay) satellite link, then a 5-second timeout might be too short, and could trigger unnecessary retransmissions. On the other hand, using a fixed value for the retransmission timer keeps the TFTP implementation simple, which is the objective the designers of TFTP had in mind.
- 17.12** TFTP does not implement any error detection mechanism for the transmitted data. Thus, reliability depends on the service provided by the underlying transport protocol (UDP). While the UDP includes a checksum for detecting errors, its use is optional. Therefore, if UDP checksums are not enabled, data could be corrupted without being detected by the destination host.

# Chapter 18 DISTRIBUTED PROCESS MANAGEMENT

## ANSWERS TO QUESTIONS

- 18.1 Load sharing:** By moving processes from heavily loaded to lightly loaded systems, the load can be balanced to improve overall performance. **Communications performance:** Processes that interact intensively can be moved to the same node to reduce communications cost for the duration of their interaction. Also, when a process is performing data analysis on some file or set of files larger than the process's size, it may be advantageous to move the process to the data rather than vice versa. **Availability:** Long-running processes may need to move to survive in the face of faults for which advance notice can be achieved or in advance of scheduled downtime. If the operating system provides such notification, a process that wants to continue can either migrate to another system or ensure that it can be restarted on the current system at some later time. **Utilizing special capabilities:** A process can move to take advantage of unique hardware or software capabilities on a particular node.
- 18.2** The following alternative strategies may be used. **Eager (all):** Transfer the entire address space at the time of migration. **Precopy:** The process continues to execute on the source node while the address space is copied to the target node. Pages modified on the source during the precopy operation have to be copied a second time. **Eager (dirty):** Transfer only those pages of the address space that are in main memory and have been modified. Any additional blocks of the virtual address space will be transferred on demand only. **Copy-on-reference:** This is a variation of eager (dirty) in which pages are only brought over when referenced. **Flushing:** The pages of the process are cleared from the main memory of the source by flushing dirty pages to disk. Then pages are accessed as needed from disk instead of from memory on the source node.
- 18.3** Nonpreemptive process migration can be useful in load balancing. It has the advantage that it avoids the overhead of full-blown process migration. The disadvantage is that such a scheme does not react well to sudden changes in load distribution.
- 18.4** Because of the delay in communication among systems, it is impossible to maintain a system wide clock that is instantly available to all systems. Furthermore, it is also technically impractical to maintain one central clock and to keep all local clocks synchronized precisely to that central clock; over a period of time, there will be some drift among the various local clocks that will cause a loss of synchronization.

- 18.5** In a fully **centralized algorithm**, one node is designated as the control node and controls access to all shared objects. When any process requires access to a critical resource, it issues a Request to its local resource-controlling process. This process, in turn, sends a Request message to the control node, which returns a Reply (permission) message when the shared object becomes available. When a process has finished with a resource, a Release message is sent to the control node. In a distributed algorithm, the mutual exclusion algorithm involves the concurrent cooperation of distributed peer entities.
- 18.6** Deadlock in resource allocation, deadlock in message communication.

## ANSWERS TO PROBLEMS

- 18.1** a. Eager (dirty)  
b. Copy on reference
- 18.2** Process  $P_1$  begins with a clock value of 0. To transmit message  $a$ , it increments its clock by 1 and transmits  $(a, 1, 1)$ , where the first numerical value is the timestamp and the second is the identity of the site. Similarly,  $P_4$  increments its clock by 1 and transmits issues  $(q, 1, 4)$ . Both messages are received by the other three sites. Both  $a$  and  $q$  have the same timestamp, but  $P_1$ 's numerical identifier is less than  $P_4$ 's numerical identifier ( $1 < 4$ ). Therefore, the ordering is  $\{a, q\}$  at all four sites.
- 18.3**  $P_i$  can save itself the transmission of a Reply message to  $P_j$  if  $P_i$  has sent a Request message but has not yet received the corresponding Release message.
- 18.4** a. If a site  $i$ , which has asked to enter its critical section, has received a response from all the others, then (1) its request is the oldest (in the sense defined by the timestamp ordering) of all the requests that may be waiting; and (2) all critical sections requested earlier have been completed. If a site  $j$  has itself sent an earlier request, or if it was in its critical section, it would not have sent a response to  $i$ .  
b. As incoming requests are totally ordered, they are served in that order; every request will at some stage become the oldest, and will then be served.
- 18.5** The algorithm makes no allowance for resetting the time stamping clocks with respect to each other. For a given process,  $P_i$  for example, *clock* is only used to update, on the one hand, *request* [ $i$ ] variables in the other processes by way of request messages, and, on the other hand, *token* [ $i$ ] variables, when messages of the token type are transferred. So the clocks are not used to impose a total ordering on requests. They are used simply as counters that record the number of times the various processes have asked to use the critical section, and so to find whether or not the number of times that  $P_i$  has been given this access, recorded as the value of *token* [ $i$ ], is less than the number of requests it has made, known to  $P_j$  by the value of *request* <sub>$j$</sub>  [ $i$ ]. The function *max* used in the processing associated with the reception of requests results in only the last request from  $P_j$  being considered if several had been delivered out of sequence.

- 18.6 a.** Mutual exclusion is guaranteed if at any one time the number of variables *token\_present* that have the value true cannot exceed 1. Since this is the initial condition, it suffices to show that the condition is conserved throughout the procedure. Consider first the prelude. The variable for  $P_i$ , which we write *token\_present<sub>i</sub>*, changes its value from false to true when  $P_i$  receives the token. If we now consider the postlude for process  $P_j$  that has issued the token, we see that  $P_j$  has been able to do so only if *token\_present<sub>j</sub>* had the value true and  $P_j$  had changed this to false before sending the token.
- b.** Suppose that all the processes wish to enter the critical section but none of them has the token, so they are all halted, awaiting its arrival. The token is therefore in transit. It will after some finite time arrive at one of the processes, and so unblock it.
- c.** Fairness follows from the fact that all messages are delivered within a finite time of issue. The postlude requires that  $P_i$  transfers the token to the first process  $P_j$ , found in scanning the set in the order  $j = i+1, i+2, \dots, n, 1, \dots, i-1$ , whose request has reached  $P_i$ ; if the transmission delays for all messages are finite (i.e., no message is lost), all the processes will learn of the wish for some  $P_j$  to enter the critical section and will agree to this when its turn comes.
- 18.7** The receipt of a request from  $P_j$  has the effect of updating the local variable, *request (j)*, which records the time of  $P_j$ 's last request. The max operator assures that the correct order is maintained.

# Appendix A TOPICS IN CONCURRENCY

## ANSWERS TO QUESTIONS

- A.1 a.** Process P1 will only enter its critical section if  $\text{flag}[0] = \text{false}$ . Only P1 may modify  $\text{flag}[1]$ , and P1 tests  $\text{flag}[0]$  only when  $\text{flag}[1] = \text{true}$ . It follows that when P1 enters its critical section we have:

$$(\text{flag}[1] \text{ and } (\text{not } \text{flag}[0])) = \text{true}$$

Similarly, we can show that when P0 enters its critical section:

$$(\text{flag}[1] \text{ and } (\text{not } \text{flag}[0])) = \text{true}$$

- b. Case 1:** A single process P(i) is attempting to enter its critical section. It will find  $\text{flag}[1-i]$  set to false, and enters the section without difficulty.

**Case 2:** Both process are attempting to enter their critical section, and  $\text{turn} = 0$  (a similar reasoning applies to the case of  $\text{turn} = 1$ ). Note that once both processes enter the **while** loop, the value of  $\text{turn}$  is modified only after one process has exited its critical section.

**Subcase 2a:**  $\text{flag}[0] = \text{false}$ . P1 finds  $\text{flag}[0] = 0$ , and can enter its critical section immediately.

**Subcase 2b:**  $\text{flag}[0] = \text{true}$ . Since  $\text{turn} = 0$ , P0 will wait in its external loop for  $\text{flag}[1]$  to be set to false (without modifying the value of  $\text{flag}[0]$ ). Meanwhile, P1 sets  $\text{flag}[1]$  to false (and will wait in its internal loop because  $\text{turn} = 0$ ). At that point, P0 will enter the critical section.

Thus, if both processes are attempting to enter their critical section, there is no deadlock.

- A.2** It doesn't work. There is no deadlock; mutual exclusion is enforced; but starvation is possible if  $\text{turn}$  is set to a non-contending process.

- A.3 a.** There is no variable that is both read and written by more than one process (like the variable  $\text{turn}$  in Dekker's algorithm). Therefore, the bakery algorithm does not require atomic load and store to the same global variable.
- b.** Because of the use of  $\text{flag}$  to control the reading of  $\text{turn}$ , we again do not require atomic load and store to the same global variable.

- A.4** The answer is no for both questions.

- A.5 a.** Change *receipt* to an array of semaphores all initialized to 0 and use *enqueue2*, *queue2*, and *dequeue2* to pass the customer numbers.

- b. Change *leave\_b\_chair* to an array of semaphores all initialized to 0 and use *enqueue1(custnr)*, *queue1*, and *dequeue1(b\_cust)* to release the right barber.

The figure below shows the program with both of the above modifications. Note: The barbershop example in the book and this problem are based on the following article, used with permission:

Hilzer, P. "Concurrency with Semaphores." *SIGSCE Bulletin*, September 1992.

Please Do Not  
Post on Web

```

program barbershop2;
var max_capacity: semaphore (:= 20);
 sofa: semaphore (:= 4);
 barber_chair, coord: semaphore (:= 3);
 mutex1, mutex2, mutex3: semaphore (:=1);
 cust_ready, payment: semaphore (:= 0);
 finished, leave_b_chair, receipt: array[1..50] of semaphore (:=0);
 count: integer;

procedure customer;
var custnr: integer;
begin
 wait(max_capacity);
 enter shop;
 wait(mutex1);
 count := count + 1;
 custnr := count;
 signal(mutex1);
 wait(sofa);
 sit on sofa;
 wait(barber_chair);
 get up from sofa;
 signal(sofa);
 sit in barber chair;
 wait(mutex2);
 enqueue1(custnr);
 signal(cust_ready);
 signal(mutex2);
 wait(finished[custnr]);
 signal(leave_b_chair[custnr]);
 pay;
 wait(mutex3);
 enqueue2(custnr);
 signal(payment);
 signal(mutex3);
 wait(receipt[custnr]);
 exit shop;
 signal(max_capacity)
end;

procedure barber;
var b_cust: integer;
begin
 repeat
 wait(cust_ready);
 wait(mutex2);
 dequeue1(b_cust);
 signal(mutex2);
 wait(coord);
 cut hair;
 signal(coord);
 signal(finished[b_cust]);
 wait(leave_b_chair[custnr]);
 signal(barber_chair);
 forever
end;

procedure cashier;
var b_cust: integer;
begin
 repeat
 wait(payment);
 wait(mutex3);
 dequeue2(c_cust);
 signal(mutex3);
 wait(coord);
 accept pay;
 signal(coord);
 signal(receipt[b_cust]);
 forever
end;

begin (*main program*)
 count := 0;
 parbegin
 customer; . . . 50 times; . . . customer;
 barber; barber; barber;
 cashier
 parend
end.

```